

TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 2 (Foundation) — How To Build It

The decisions you will make on the new platform, how to set each one up, and why

18 HOURS

24 LESSONS

PREREQUISITE: MODULE 1



AUTHOR

Surender Sara

COMPANY

Northbay Solutions

ISSUED

August 2026

Design the Engine Before the Pipelines

Eight source systems and roughly a hundred tables. Decide now what the unit of work is — because you cannot change it later.

1 · THE DECISION

Is the unit of work a job, or a spec?

Whatever you pick in week one is what the next hundred tables have to live inside.

✗ One job per table

✓ One engine · many specs

2 · DO THIS

BUILD FOUR SEAMS, THEN WRITE JOBS THAT ONLY WIRE THEM

SPEC

What the table is: schema, natural key, source config — validated before Spark starts.

glue_engine/spec.py

CONNECTOR

Where the rows come from. One Protocol; one class per kind of source, never per table.

sources/protocol.py

WRITER

Where the rows go to. Land raw immutably, then MERGE into the table on its key.

writers/s3_tables.py

CONTROL PLANE

What actually happened: runs per stage, watermarks, live state, lineage. Queryable.

glue_engine/control_plane.py

add table N+1 = 1 spec file + 1 catalog row – no new code, no deploy

3 · WHY

One codebase serves a hundred tables, so a fix lands once, everywhere. And the config path is diffable, reviewable and testable without ever starting a cluster.

IF YOU DON'T

A hundred near-identical jobs drift apart.

4 · ON APPAREL GROUP

RMS · SIM · XStore → one Oracle class

Epsilon · MoEngage → one API class

Magento · Vemco · Irisys → drop / API

8 sources · 3 classes · ~100 specs

IN PLAIN ENGLISH

Build the printer once, not a hundred hand-written letters. After that, every new table is just another page you feed it.

L01 · Design the Engine Before the Pipelines

Slide: [_render/L01-design-the-engine.html](#)

The decision

You are about to onboard eight source systems and roughly a hundred tables. Before you write a single pipeline, you have to answer one question:

Is the unit of work a job, or a spec?

Either every table gets its own Python job — its own file, its own tests, its own deployment — or you build **one engine** that reads a **declarative spec** and treats every table as data.

You pick this in week one, and the next hundred tables have to live inside whatever you picked. It is not a refactor you get to do later: by the time the cost is obvious, there are ninety jobs to unpick.

Choose the engine.

Do this

- 1. **Define the spec model first, in code.** A typed, validated object — Pydantic or equivalent — with fields for: source system, object name, column list with types, natural key (`merge_key`), read mode, landing prefix, and any per-table overrides. Nothing about a table lives anywhere else.
- 2. **Make an invalid spec fail at parse time, not at run time.** Add model validators that reject the classes of mistake you cannot detect later: a `merge_key` naming a column that is not in the declared schema; a column marked as sensitive being used as a key; a watermark column missing from the projection. These are cheap to write and each one removes a whole family of production bugs. *Worked example:* [src/glue/glue_engine/spec.py](#) — read the `@model_validator` block.

3. Cut the engine along four seams, and only four.

SEAM	OWNS	REFERENCE FILE
Spec	What the table <i>is</i>	glue_engine/spec.py
Connector	Where the rows come <i>from</i>	sources/protocol.py
Writer	Where the rows go <i>to</i>	writers/raw_landing.py · writers/s3_tables.py
Control plane	What actually <i>happened</i>	glue_engine/control_plane.py

- 4. **Write jobs that only wire the seams together.** A download job should read: resolve the table's routing → parse its spec → ask the registry for a connector → call one read method → hand the frame to a writer → record the outcome. If a job contains business logic, that logic belongs behind a seam. *Worked example:* [jobs/source_download.py](#) is exactly that sequence, end to end.
- 5. **Package the engine as one artifact.** One wheel, one version, deployed once. Specs are configuration that ships alongside it, not code that ships inside it.
- 6. **Keep routing in the control plane, not in the spec.** Which tables run today, in which phase, at what concurrency — that is operational state and it changes without a deploy. The spec describes the table; the control plane decides the schedule.
- 7. **Prove the claim before you scale.** Onboard the second table and check that you wrote **zero** new Python. If you had to write some, the seam it crossed is in the wrong place — fix it now, at two tables, not at fifty.

The target shape, stated as an acceptance test:

add table N+1 = 1 spec file + 1 catalog row - no new code, no deploy

L01 · Design the Engine Before the Pipelines

Why

- **A fix lands once.** One `MERGE` implementation, one retry policy, one metrics emitter. When you improve it, every table gets the improvement on the next run.
- **The config path is reviewable.** A YAML spec diffs in a pull request and a non-Python reviewer can read it. A hundred near-identical Python files diff badly and nobody reads them.
- **The config path is testable without a cluster.** Spec parsing and validation are pure functions. You get a fast unit-test suite over the thing that actually breaks — the per-table declarations.
- **Onboarding is bounded.** "How do I add a table?" has a three-line answer that a new joiner can execute on day two.

What breaks if you don't: a hundred near-identical jobs drift apart, and every fix has to be made a hundred times — with the ones you miss failing silently.

On Apparel Group

Eight sources, three connector classes, roughly a hundred specs.

SOURCE	KIND	CONNECTOR IT NEEDS
Oracle Retail (RMS)	Oracle DB	Oracle JDBC
Oracle SIM	Oracle DB	Oracle JDBC
Oracle XStore	Oracle DB	Oracle JDBC
Epsilon	SaaS API (PII)	API / cursor
MoEngage	SaaS API	API / cursor
Magento	DB or API	Oracle JDBC pattern, or API
Vemco Footfall	file / API	File drop
Irisys Footfall	file / API	File drop

Three of the eight are Oracle and share one connector class outright. Two are cursor-paged SaaS APIs and share a second. The footfall feeds are small file drops and share a third. **The variety is in the specs, not in the code** — which is the whole reason the engine is worth building on week one rather than week nine.

Sequence the build: engine and spec model first, then the Oracle connector, then RMS as the first real table. Every source after RMS is a spec-writing exercise.

Checklist

- ☐ Spec model is typed and validated; an invalid spec raises before any compute starts
- ☐ Validators cover: key not in schema, watermark not in projection, sensitive column used as key
- ☐ The four seams exist as separate modules with no cross-imports between them
- ☐ No job file contains an `if source == ...` or a table name
- ☐ Adding the second table required zero new Python — verified, not assumed
- ☐ Engine is one versioned artifact; specs ship as configuration
- ☐ Routing and schedule live in the control plane, not in the spec files
- ☐ A new joiner can add a table from the README alone

You've got it when you can...

...be handed a ninth source system and estimate the work as *"one connector class, then one spec per table"* — and defend that estimate by naming which of the four seams each piece of the work lands in, and why none of it touches the jobs.

Define the Source Contract First

Write the interface every source must satisfy before you write any source. The engine must never learn a vendor's name.

1 · THE DECISION

How does system number nine plug in?

Either the engine grows a branch for every vendor, or it never hears of them at all.

✗ `if src == "oracle"`

✓ **Protocol + registry**

2 · DO THIS

WRITE THE CONTRACT, THEN REGISTER IMPLEMENTATIONS BY NAME

```
class SourceConnector(Protocol):
    def configure(cfg)      -> None
    def read_full()         -> DataFrame
    def read_incremental(wm)-> df, new_wm
    def read_range(a, b)    -> DataFrame
    def emit_metrics()      -> dict
```

`read_incremental` returns the new watermark.

The engine persists it — never the connector.

the test: `grep -ri oracle glue_engine/` → hits `sources/oracle_jdbc.py` and nothing else

THE REGISTRY

```
@register("oracle_jdbc")
```

A decorator fills a dict; the job looks the class up by name at run time.

ADDING SOURCE #9

1 new file · `sources/oracle_jdbc.py`

1 import line · `sources/__init__.py`

3 · WHY

A JDBC pull, a paged REST call and a nightly file drop all satisfy the same five methods. Adding a source becomes additive — and a connector may decline a mode it cannot do.

IF YOU DON'T

Source #9 edits twenty unrelated files.

4 · ON APPAREL GROUP

RMS · SIM · XStore — full + incremental

Epsilon · MoEngage — incremental by cursor

Magento · Vemco · Irisys — full only

The Oracle branch lands in ONE file

IN PLAIN ENGLISH

A wall socket. The kettle knows nothing about the power station; it just fits the plug. Define the socket before you buy appliances.

L02 · Define the Source Contract First

Slide: [_render/L02-source-contract.html](#)

The decision

Source number nine will arrive. It always does — a new acquisition, a new loyalty platform, a system nobody mentioned in the kickoff.

How does it plug in?

Two answers. Either the engine grows a branch for every vendor (`if source == "oracle": ... elif source == "epsilon": ...` , spreading through jobs, writers and barriers), or the engine **never hears a vendor's name at all** and looks a class up by string at run time.

Write the interface before you write any source. The first connector you write will otherwise become the interface by accident, shaped entirely around whichever system you happened to start with.

Do this

1. Write the **Protocol** first — five methods, no more.

```
class SourceConnector(Protocol):
    def configure(cfg)      -> None
    def read_full()         -> DataFrame
    def read_incremental(wm) -> tuple[DataFrame, str]  # (rows, new_watermark)
    def read_range(a, b)    -> DataFrame
    def emit_metrics()      -> dict
```

Mark it `@runtime_checkable` so a test can assert `isinstance(conn, SourceConnector)` before the class ever runs in production. *Worked example:* [sources/protocol.py](#) — the whole contract is about a hundred lines including docstrings.

2. **Put credentials and validation in `configure` , once per run.** Secret fetch, driver setup, connection-string assembly, and every precondition check the connector wants to make. It runs once; the read methods stay cheap and stateless.

3. **Make `read_incremental` return the new watermark — never persist it.** The connector hands back `(df, new_watermark)` ; the engine writes it, atomically with the run-success record. Ordering matters: a connector that advanced its own watermark would move the bookmark for a run that then failed to land, and the rows in the gap are lost forever.

4. **Build a registry keyed by string.**

```
@register("oracle_jdbc")
class OracleJdbcConnector: ...

conn = get_connector(source_type)()      # look up by name
conn.configure(cfg)
```

Populate it with import side effects — one import block that pulls in every connector module so its decorator runs. Note that this block is load-bearing: delete it and the registry is empty. *Worked example:* [sources/__init__.py](#) .

L02 · Define the Source Contract First

- 5. **Make both registry failure modes loud.** Registering the same name twice raises immediately ("refusing to overwrite"). Asking for an unregistered name raises an error that **lists every registered name** — that list is the fastest debugging aid you will ship all quarter.
- 6. **Let a connector decline a mode.** A snapshot-only source raising `NotImplementedError` from `read_range` is correct behaviour, and the contract should say so explicitly. Declining is part of the interface, not a bug. *Worked example:* `sources/excel_landing.py` .
- 7. **Allow a per-mode driver override, and keep it in config.** A table may want one connector for its full load and another for its deltas. Resolve it with a pure function — no cloud SDK imports, no Spark — so it is unit-testable anywhere. *Worked example:* `driver_select.py` .
- 8. **Where the Oracle branch lands.** Onboarding Oracle is: - **1 new file** — `sources/oracle_jdbc.py` , holding the driver JAR reference, the connection URL, the partitioning strategy, the type quirks, everything Oracle-shaped. - **1 import line** — added to `sources/__init__.py` . - **0 changes** to jobs, writers, barriers, the control plane or any other connector.
- 9. **Enforce it with a test you can run in CI.**

```
grep -ri oracle glue_engine/ → hits sources/oracle_jdbc.py and nothing else
```

If that grep ever returns a second file, a vendor name has leaked into the engine. Fail the build.

Why

- **One interface, many shapes.** A JDBC pull, a paged REST call and a nightly file drop all satisfy the same five methods. The engine cannot tell them apart, and does not need to.
- **Onboarding becomes additive.** New behaviour is a new file. Nothing existing is edited, so nothing existing can regress — which means the change is reviewable in isolation and cheap to roll back.
- **The seam is free.** A `Protocol` and a dictionary cost you an afternoon. You will pay that afternoon back the first time a source is added, and again every time after.
- **It tells you when the engine genuinely must change.** If a source cannot express *any* of full / range / incremental, the contract is wrong and you should change the contract deliberately — rather than smuggling the special case into a job.

What breaks if you don't: source number nine means edits to twenty unrelated files, each one a chance to regress a source that was working fine yesterday.

On Apparel Group

All eight sources reduce to three connector classes and a handful of declined modes.

L02 · Define the Source Contract First

SOURCE	SOURCE_TYPE	READ_FULL	READ_INCREMENTAL	READ_RANGE
Oracle Retail (RMS)	oracle_jdbc	✓	✓	✓
Oracle SIM	oracle_jdbc	✓	✓	✓
Oracle XStore	oracle_jdbc	✓	✓	✓ (the giant needs it)
Epsilon	api_cursor	✓	✓ (their cursor)	decline
MoEngage	api_cursor	✓	✓ (their cursor)	decline
Magento	oracle_jdbc or api_cursor	✓	✓	depends
Vemco Footfall	file_drop	✓	decline	decline
Irisys Footfall	file_drop	✓	decline	decline

Two things to notice. **Declining is the common case, not the exception** — five of the eight decline at least one mode, and that is fine. And **Epsilon carries PII**, so its connector is also the natural place to enforce masking-at-read; keeping that inside the class rather than in a job is what stops the rule being forgotten on source number ten.

Write the Oracle connector once and three of your eight sources are covered. That is the return on defining the contract first.

Checklist

- ☐ `Protocol` written and `@runtime_checkable` , with all five method signatures
- ☐ Docstrings state the watermark contract: the connector **returns**, the engine **persists**
- ☐ Registry populated by decorator; the side-effect import block is present and commented as load-bearing
- ☐ Duplicate registration raises; unknown name raises and lists what *is* registered
- ☐ Declining a mode is documented as legal, with a real example
- ☐ Per-mode driver override resolves in a pure, importable-anywhere function
- ☐ A contract test asserts every registered class satisfies the `Protocol`
- ☐ CI runs the vendor-name grep over the engine package
- ☐ Adding a source touched exactly one new file and one import line — verified

You've got it when you can...

...explain to a sceptical colleague why onboarding Oracle needs no change to the download job, point at the exact line where the class is chosen by name — and then say what *would* legitimately force an engine change: a source that cannot express any of full, range or incremental.

Size JDBC Parallelism Properly

Too few readers and the job times out. Too many and the source refuses every connection. Here is the arithmetic you reuse.

1 · THE DECISION

How many parallel readers for a giant table?

The number is bounded below by row volume and above by what the source will let you open at once.

✗ Guess, then tune

✓ Compute, then clamp

2 · DO THIS

FOUR NUMBERS, IN THIS ORDER — THE CEILING ALWAYS WINS

WORKED EXAMPLE · XSTORE POS · 300 M ROWS · 4 LANES · CEILING 160

1	partitions	=	rows ÷ 8 M	300 M ÷ 8 M	->	38
2	connections	=	lanes × partitions	4 × 38	->	152
3	budget	=	ceiling × 50 %	160 × 50 %	->	80
4	CLAMP	=	budget ÷ lanes	80 ÷ 4	->	16 partitions

WHEN THE CLAMP BITES

16 partitions × 19 M rows is over the 5–10 M target — cut lanes, or split the giant into date windows.

connect timeout 60 s · read timeout 120 s - the network hop is slower than the query

3 · WHY

One reader on a 300 M-row table is one socket and one very long wait, ending in a timeout. Too many and the source's connect pool saturates — then every read fails at once.

IF YOU DON'T
You learn the ceiling from a 2-hour timeout.

4 · ON APPAREL GROUP

XStore POS is the giant — size it first
RMS · SIM facts — 4 to 8 partitions
RMS masters · lookups — 1, unpartitioned
Ask the DBA for the ceiling

IN PLAIN ENGLISH

Checkout lanes: too few and the queue never clears, too many and the doorway jams. Size against the doorway, not the queue.

L03 · Size JDBC Parallelism Properly

Slide: [_render/L03-sizing-jdbc-parallelism.html](#)

The decision

You have a 300-million-row table on the far side of a network hop.

How many parallel readers?

One connection reading 300 M rows is one socket, one executor and one very long wait, which ends in a job timeout. So you add readers. But readers are simultaneous connections, and the source database has a hard ceiling on those — a ceiling you share with every other job, every other lane, and the source system's own users.

The number is **bounded below by row volume and above by the connection ceiling**. This lesson gives you the arithmetic to find it before you run anything.

Do this

1 · Get the two inputs, from evidence

- rows** — the real row count of the table, measured with the same filter your extract will use (client/tenant filter, date horizon, everything). Not the vendor's headline figure.
- ceiling** — the source's maximum simultaneous connections. **Ask the DBA**. Do not infer it, and do not discover it by saturating production.

Record both in the spec so the next person does not have to re-derive them:

```
expected_row_count: 300_000_000
# source connection ceiling: 160 (confirmed with DBA)
```

Worked example: [specs/download/sap_mbew.yaml](#) — the partition count, the row count, and the reasoning are all in comments beside the values.

2 · Run the four numbers, in this order

Target $\approx$ **5–10 M rows per partition**. Use 8 M as the working midpoint.

```
1 partitions = rows ÷ 8 M      300 M ÷ 8 M      -> 38
2 connections = lanes × partitions 4 × 38      -> 152
3 budget      = ceiling × 50 %   160 × 50 %   -> 80
4 CLAMP       = budget ÷ lanes   80 ÷ 4        -> 16 partitions
```

- lanes** is your orchestrator's concurrency — how many download jobs run at the same time (a Step Functions Map `MaxConcurrency`, a thread pool size, a task count). Every lane opens its own set of partitions, so **total simultaneous connections = lanes × partitions**. This is the number that matters, and it is the one people forget.
- Budget at 50 % of the ceiling, not 100 %**. You are not the only client. Leave headroom for the source's own workload, for a retry that overlaps the original, and for the next table you onboard.
- Step 4 always wins**. If the clamp is lower than step 1, take the clamp.

3 · Handle the case where the clamp bites

In the example above, 16 partitions means **19 M rows per partition** — over the 5–10 M target. Do **not** raise the partition count past the ceiling. Instead, in this order of preference:

L03 · Size JDBC Parallelism Properly

- 1. **Cut lanes.** Run the giant with fewer concurrent tables and give it more partitions each. `2 lanes × 40 partitions = 80` connections, 7.5 M rows per partition — inside both targets.
- 2. **Split the table into date windows** (a chunked / windowed load — see [L05](#)). Each window is a smaller read, and windows are scheduled rather than simultaneous.
- 3. **Only then**, negotiate a higher ceiling with the DBA — with the arithmetic in hand.

4 · Set timeouts for a network hop, not a local socket

Default JDBC timeouts assume a database on the same network. Yours is not.

```
connect timeout    60 s
read timeout      120 s
```

When many partitions open at once, the **connect handshake** is what saturates first — the query itself may be fine. A 15-second connect budget will fail under exactly the parallelism you just computed. Put these in the connection URL, with a comment saying why.

5 · Add a guard so nobody ships an unpartitioned giant by accident

In `configure()` , before a single row is read:

```
if expected_row_count > single_partition_max_rows and no partition strategy configured:
    raise
```

Default `single_partition_max_rows` to 5 M. A table that legitimately needs a higher ceiling raises it **in its own spec, with a comment** — which turns "we never thought about it" into "we thought about it and here is

why". *Worked examples:* [sap_konp.yaml](#) (a deliberate single-partition read with an explicitly raised ceiling) and [sap_vbrp.yaml](#) (the lanes × partitions arithmetic written out in a comment next to the value).

6 · Choose the partitioning mechanism to match the column

- **Predicate partitioning** — hand the reader a list of `WHERE` clauses (one contiguous sub-range each) and it opens one connection per predicate, each pushed down to the source. Works on any orderable column, including text dates. This is the default choice.
- **Range partitioning** (`partitionColumn + lowerBound / upperBound / numPartitions`) — requires a genuinely numeric column with a known, roughly uniform range. Beautiful when it applies; it often does not.
- **Never partition on a column that is not indexed at the source.** Each partition then becomes its own full table scan and you have multiplied the work, not divided it.

Why

- **Too few readers:** one socket, one long-running query, and a job timeout — usually after burning most of a batch window, and usually retried automatically so it burns the window again.
- **Too many readers:** the source's connect pool saturates. The failure is not graceful degradation — *every* read fails at once, including the small tables that were nothing to do with the giant. And the error text ("cannot connect to host", socket timeout) points at the network, so the first hour of debugging goes in the wrong direction.
- **The ceiling is the hard constraint, row volume is the soft one.** You can always deliver a giant in more windows over more time. You cannot open more connections than the source will accept.

What breaks if you don't: you learn the ceiling from a two-hour timeout in a live batch window, instead of from a five-minute conversation with the DBA.

L03 · Size JDBC Parallelism Properly

On Apparel Group

XStore POS is the giant — size it first, and size everything else in the space it leaves.

TABLE CLASS	EXAMPLE	ROWS (ORDER)	PARTITIONS	NOTES
POS transactions	XStore	100 M – 1 B	clamp, then window	Almost certainly exceeds the ceiling at any sensible lane count. Plan a windowed load from day one.
Retail facts	RMS sales, SIM positions	10 – 100 M	4 – 8	The normal case. Straight predicate partitioning on the watermark column.
Master data	RMS item, supplier, location	1 – 10 M	1 – 2	Well under the single-partition ceiling; do not partition for the sake of it.
Lookups / code tables	RMS reference tables	< 100 K	1	Partitioning costs more in connections than it saves in time.
SaaS APIs	Epsilon , MoEngage	n/a	n/a	Not JDBC. Their limit is a rate limit , not a connection ceiling — same discipline, different number.
File drops	Vemco , Irisys	tiny	n/a	Single reader. Nothing to size.
Magento	orders, customers, products	1 – 50 M	1 – 4	Size once you know whether it is DB or API.

Two Apparel-Group-specific cautions:

- **RMS, SIM and XStore may share an Oracle instance or a listener.** If they do, the ceiling is shared across all three, and your budget must be split across them — not applied three times.
- **Three Oracle sources running in the same nightly window multiply lanes.** Compute `lanes` as the total concurrent download jobs across *all* sources, not per source.

Checklist

- ☐ Real row count measured with the production filter, and recorded in the spec
- ☐ Connection ceiling confirmed with the DBA, and recorded in a comment
- ☐ `lanes` identified as the orchestrator's actual concurrency across all sources
- ☐ Four-step arithmetic run and written down; the clamp applied
- ☐ Rows per partition inside 5–10 M — or the table is windowed instead
- ☐ Budget is ≤ 50 % of the ceiling
- ☐ Connect timeout 60 s, read timeout 120 s, set in the connection URL with a comment
- ☐ Guard raises on an unpartitioned table above `single_partition_max_rows`
- ☐ Any raised ceiling is per-spec and carries a written reason
- ☐ Partition column is indexed at the source

You've got it when you can...

...read *"cannot connect to host (socket timeout)"* in a job log and immediately ask the right question — **"how many partitions, times how many lanes, against what ceiling?"** — instead of the wrong one, *"is the network down?"*

Choose a Watermark You Can Trust

One column decides which rows are new. Pick it on evidence, validate it before use, and never store one you would refuse.

1 · THE DECISION

Which column means "this row is new"?

Every future run trusts that column without question. Qualify it now, or pay for it every night.

✗ Any date-ish column

✓ Monotonic & indexed

2 · DO THIS

QUALIFY THE COLUMN — ALL FOUR TESTS, OR IT IS NOT A WATERMARK

MONOTONIC

only ever goes up;
an update moves it

INDEXED

the delta predicate
must not table-scan

NON-NULL

a NULL row stays
invisible forever

PARSEABLE

you know its exact
type and format

VALIDATE BEFORE YOU SPLICE

Check the value against the declared
format before it goes near SQL. On
failure keep the old mark and re-read.

NO COLUMN QUALIFIES?

Declare the table full-refresh in its
spec. Faking CDC on a column you
cannot trust loses rows in silence.

forward-only: a candidate below the stored watermark is discarded, never persisted

3 · WHY

A watermark is a bookmark every future run
trusts without question. A bad one fails in
one of two ways: it skips rows in silence,
or it wedges the table at the same guard.

IF YOU DON'T

The gap surfaces months later, unexplained.

4 · ON APPAREL GROUP

RMS · SIM · XStore — prove one watermark
per table before any spec is written
Epsilon · MoEngage — use their cursor

Lookups & footfall — full-refresh

IN PLAIN ENGLISH

A bookmark. Put it on the wrong page and you skip chapters — or can't open the book at all. Check the page before you place it.

L04 · Choose a Watermark You Can Trust

Slide: [_render/L04-choosing-a-watermark.html](#)

The decision

An incremental load is one stored value interpolated into one `WHERE` clause. That value is the **watermark**, and it decides which rows the platform believes are new.

Which column means "this row is new" — and what do you do when no column qualifies?

Every future run will trust that column without asking questions. It is a bookmark you place once and then rely on, unattended, for years. So the decision is not "which column looks like a date" — it is **which column can you prove is safe to trust**, and what you do with the tables where the honest answer is *none of them*.

Do this

1 · Qualify the column against four tests. All four, or it is not a watermark.

TEST	WHAT IT MEANS	HOW TO CHECK IT
MONOTONIC	Only ever increases, and an update moves it	Does the application write it on <code>UPDATE</code> , or only on <code>INSERT</code> ? A creation-date column cannot see edits or hard deletes.
INDEXED	The delta predicate must not table-scan	Read the execution plan for <code>WHERE col > :wm</code> . If it scans, your "cheap" delta costs more than a full read.
NON-NULL	A NULL row is invisible forever	<code>SELECT count(*) FROM t WHERE col IS NULL</code> . Any non-zero answer disqualifies the column, or requires a <code>COALESCE</code> with a documented sentinel.
PARSEABLE	You know its exact type and format	Is it a real <code>DATE</code> / <code>TIMESTAMP</code> , or text? What format is the text? Is it timezone-aware? Never assume.

Do this **before** you write the spec, and record the answers next to the column name. A watermark chosen after the spec is written is a watermark chosen to fit the code.

2 · Prefer an update stamp, then an id, then a creation date

- `last_updated` / `row_version` / `SCN` — sees inserts *and* updates. The right answer when the source offers one.
- A **monotonic surrogate id** — sees inserts only, but is dense, indexed and unambiguous.
- A **creation date** — sees inserts only, and **cannot see updates or hard deletes**. Usable, but write the limitation down: this table will need a periodic full refresh to stay correct.

3 · Declare the format explicitly in the spec

```
watermark_column: last_update_date
watermark_type:   timestamp           # or: integer_id | text_date
watermark_format: "YYYY-MM-DD HH24:MI:SS"
safety_buffer:    1d                  # widen the READ window; never move the stored value
```

If the column is text, the format field is load-bearing: a mismatch produces a predicate that matches nothing, the run succeeds, and zero rows land. A silent zero is much worse than a loud failure.

4 · Validate the value before it goes near SQL

Check the stored watermark against the declared format **every time**, immediately before interpolation. Anything that does not match raises rather than being spliced. This is an injection defence and a corruption detector in the same three lines.

Distinguish two cases carefully:

L04 · Choose a Watermark You Can Trust

- **Empty / absent** — the "no watermark yet" sentinel for a first-ever run. Legal. Fall back to the full-load path.
- **Present but unparseable** — corruption. Raise.

Treating the first as the second sends every first run straight into the guard.

5 · Never persist a watermark you would refuse to reuse

This is the rule the whole lesson exists for.

```
candidate = MAX(watermark_column) over the rows just read
if candidate would fail your own validation      -> keep the previous watermark
if candidate < stored watermark                  -> keep the previous watermark    (forward-only)
if rows_with_errors > 0                         -> keep the previous watermark
```

Compute `MAX()` over **plausible rows only** — inside the declared format, inside a sane range (not before the system existed, not in the future). Rows that fail those bounds still land; they simply cannot become the bookmark.

Keeping the old watermark costs you one repeated delta, and repeating a delta is free because your load is idempotent (see [L07](#) — the merge on the natural key absorbs it). Storing a bad watermark costs you **every future run** until a human notices.

The **forward-only** rule matters more than it looks: with a safety buffer, the read window starts *before* the stored watermark, so a hard delete upstream can make the window's `MAX()` come back lower than what you already have. Persisting that rewinds progress a little every single night.

Worked example: the refusal-and-forward-only sequence in [sources/sap_hana.py](#) , `read_incremental` .

6 · Guard the projection

If the spec's column list omits the watermark column, `MAX(watermark_column)` fails — *after* the entire extract has been paid for. Validate at spec-parse time that the watermark column is in the projection. One validator, one whole class of expensive late failure removed.

7 · No column qualifies? Declare full-refresh. Do not fake CDC.

Some tables genuinely have no trustworthy change column: reference and code tables, tables keyed on a constant, tables where the "date" column is populated by hand. For those, set the read mode to full in the spec and route the table so the delta path never runs at all.

This is a **legitimate, documented design choice**, not a compromise. Faking CDC on a column you cannot trust produces a pipeline that succeeds every night while quietly losing rows — and nobody finds out for months, because there is no error to find.

Why

- **A watermark is trusted unconditionally.** No downstream consumer re-checks it. Whatever it says is what the platform believes.
- **It has exactly two failure modes, and both are nasty.** It either **skips rows in silence** — the run succeeds, the data is wrong, and no alarm exists for "rows that were never asked for" — or it **wedges the table**, failing at the same guard every night until someone edits the stored value by hand.
- **Re-reading is free; wedging is not.** Because the load is idempotent, the conservative choice (keep the old mark, read a bit more) has almost no cost. The aggressive choice (store it and hope) has an unbounded one.
- **A loud failure is a gift.** An exception costs an hour. A silent gap costs a quarter, and the credibility of the whole platform along with it.

L04 · Choose a Watermark You Can Trust

What breaks if you don't: the gap surfaces months later, in a report nobody can reconcile, with no way to tell which nights are affected.

On Apparel Group

Do the watermark survey before any spec is written. For each table on each Oracle source, one row: column name, type, format, indexed?, NULL count, does it move on update?

SOURCE	WATERMARK STRATEGY
Oracle Retail (RMS)	Facts: prefer <code>LAST_UPDATE_DATETIME</code> / <code>LAST_UPDATE_ID</code> where the RMS table offers one. Masters and code tables: expect several with no usable change column — full-refresh them, they are small.
Oracle SIM	Inventory positions churn constantly; a true update stamp is essential , not a creation date. A creation-date watermark on a position table will silently miss every quantity change — which is the only thing anyone cares about.
Oracle XStore	POS transactions are append-heavy, so a business date or transaction id works well. Confirm the timezone: store-local vs UTC will shift a day's takings across the date boundary.
Epsilon	Use their cursor/token. Do not derive a watermark from a payload field — persist the vendor's cursor exactly as issued, opaquely. Same refusal rules apply.
MoEngage	As Epsilon: their cursor, persisted verbatim, forward-only.
Magento	<code>updated_at</code> is usually present and usually trustworthy. Verify it is indexed before you rely on it.
Vemco / Irisys Footfall	Small per-store counts. Full-refresh . Do not build CDC for a table you can re-read in seconds.

Two Apparel-Group traps worth naming up front:

- **Retail systems love a soft-delete flag.** A row "deleted" by a status change is an *update*. If your watermark only sees inserts, deletions never propagate and your stock and sales figures drift upward forever.
- **Timezones across a multi-country estate.** Store-local timestamps across several countries mean "yesterday" is not a single interval. Decide — and write down — whether watermarks are UTC or store-local, once, for the whole platform.

Checklist

- ☐ Every table has a watermark decision recorded **before** its spec is written
- ☐ Candidate column passes all four tests: monotonic, indexed, non-null, parseable
- ☐ Column type and format declared explicitly in the spec
- ☐ It is known and written down whether the column moves on `UPDATE` — and if not, a periodic full refresh is scheduled
- ☐ Empty watermark is handled as "first run", not as corruption
- ☐ Value is validated against its declared format immediately before interpolation
- ☐ `MAX()` is computed over plausible rows only
- ☐ A candidate that fails validation is **discarded**, keeping the previous value
- ☐ Forward-only: a candidate below the stored value is discarded
- ☐ Rows-with-errors > 0 blocks the watermark advance
- ☐ Spec validator asserts the watermark column is in the projection
- ☐ Tables with no qualifying column are declared full-refresh and routed away from the delta path
- ☐ Timezone convention decided once, platform-wide

L04 · Choose a Watermark You Can Trust

You've got it when you can...

...answer "*why didn't the watermark move even though the job succeeded?*" with a specific reason — empty delta, unparseable candidate, or a value behind the stored one — name the log line each case emits, and explain

why refusing to move it was the **correct** outcome in all three.

Plan the Load Strategy per Table

Full-only master, incremental fact or windowed giant. Classify first — then make day one complete before CDC ever starts.

1 · THE DECISION

Full, incremental or windowed — and what does day one load?

Classify every table before you write its spec — the class decides day one and every day after.

✗ Let CDC catch up

✓ Classify, then load

2 · DO THIS

CLASSIFY EVERY TABLE INTO ONE OF THREE SHAPES

FULL-ONLY MASTER

No trustworthy change column, or small enough not to care. Read it all.

`read_mode: full`

INCREMENTAL FACT

Trusted watermark, high churn. Day 1 is ONE full load, then daily deltas.

`read_mode: incremental`

WINDOWED GIANT

Too big for one pull. Day 1 is N date windows in parallel — all finish.

`read_mode: range (chunked)`

HAND OVER ONCE, NOT PER CHUNK

(none) → initial_loaded → cdc. The phase gate flips it when every window is done.

EMPTY MEANS TWO THINGS

Empty FULL pull = FAIL, before landing.
Empty DELTA = success. A quiet day.

3 · WHY

An incomplete day one is invisible: every later reconciliation is wrong by an amount nobody can name. And an empty full pull that lands replaces good data with none.

IF YOU DON'T

The error surfaces months after the build.

4 · ON APPAREL GROUP

XStore POS — windowed giant
RMS · SIM facts — incremental
RMS masters · Magento — full-only
Epsilon · MoEngage — cursor delta
Vemco · Irisys — full drops

IN PLAIN ENGLISH

Moving house: everything moves once, then only the new post. Forward the post too early and the furniture never arrives.

L05 · Plan the Load Strategy per Table

Slide: [_render/L05-load-strategy.html](#)

The decision

For every single table you onboard:

Full, incremental or windowed — and what exactly does day one load?

These are two questions and people usually only answer the first. The steady-state mode is the easy part. The expensive part is **day one**: how the table gets from empty to complete, and how the platform knows it got there before the nightly delta takes over.

Classify the table first. The class determines the read mode, the day-one plan, and the empty-result rule — all three fall out of one decision, which is why it is worth making deliberately and writing down.

Do this

1 · Put every table in exactly one of three boxes

CLASS	WHEN	STEADY STATE	DAY ONE
FULL-ONLY MASTER	No trustworthy change column (see L04), or small enough that a full read is cheaper than the machinery	Read all of it, every run	Just a normal run
INCREMENTAL FACT	Trusted watermark, high volume, high churn	Daily delta from the stored watermark	One complete full load
WINDOWED GIANT	Too big to pull in one go, even partitioned (see L03)	Daily delta	N date windows, run in parallel, all must finish

```
read_mode: full           # full-only master
read_mode: incremental    # incremental fact
read_mode: range          # windowed giant (chunked initial load)
```

Worked example: one row per table in [config/ingestion_tables.yaml](#) , with the per-table detail in [specs/download/](#) .

2 · Make the initial load complete — never a window

An initial load must cover the table's **full declared scope**. If you define a horizon ("the last five years"), the initial load pulls the whole horizon in one go; it does not sub-window below it and call itself done.

And note the exception that catches people out: **master data and creation-date-keyed tables get no horizon at all**. A customer master keyed on a creation date, filtered to five years, drops every customer created before that — including the active ones. Master tables load *truly* full.

State the invariant you are protecting, in one sentence, and then test it:

After the initial load, the landed row count equals the source's row count **measured with the same filter**.

Check it on **both** sides. Run the count against the source and against the target and compare. Do not assert it from reading the code.

3 · Derive full where the config already implies it

If a table's routing declares only a full mode and no incremental mode, the job should force full and **clear any date window** it was passed. A full-refresh table has no real date column, so a uniform initial-load window would filter on a placeholder and match nothing — a run that succeeds and lands zero rows.

L05 · Plan the Load Strategy per Table

One exception: a table that is *both* full-refresh *and* enormous is delivered as many windows on purpose. Make that an explicit opt-in flag (`chunk_full_reload` or similar) so the derivation does not silently un-window it. *Worked example:* the derivation and its exception in [jobs/source_download.py](#) .

4 · Hand over to CDC once — and let the phase gate own the flip

Track an explicit lifecycle per table:

```
(none) → initial_loaded → cdc
```

- `(none)` — never loaded. Not admitted to the nightly cycle.
- `initial_loaded` — the complete initial load succeeded.
- `cdc` — the first plain daily delta after that has run.

The phase barrier owns the transition, not the job. A windowed load's chunks run concurrently across lanes and finish out of order, so no single chunk can know whether the load is complete. If a chunk flips the state, the table gets admitted to the next nightly cycle **mid-reload**, with a watermark from whichever window happened to finish first — and the delta then starts from the middle of history. Let the barrier flip it, once, when every window of every table is terminal. *Worked example:* the completion check in [src/lambdaas/download_barrier/handler.py](#) .

5 · Seed the watermark from a windowed initial load

A windowed initial load still has to leave a watermark behind — seeded forward-only from the **end of the window range**. Otherwise a giant that never gets a single full pull reaches `initial_loaded` with an empty watermark, and every subsequent delta dies at the validation guard from [LO4](#).

6 · Define the empty-result policy — it is not one rule

An empty result means opposite things in the two directions:

	MEANING	ACTION
Empty FULL pull	Almost always a broken extract, a wrong filter, or a connection that returned nothing	FAIL — and fail before landing , so no success marker is written and no watermark moves
Empty DELTA	A quiet window. Perfectly normal	SUCCEED — record zero rows and move on

Allow a per-spec `on_empty: succeed | warn | fail` override for the genuine exceptions, but make the **defaults** the two rules above. The critical half is "fail before landing": an empty full pull that lands replaces good data with nothing, and the failure looks like a successful run.

7 · Treat a backfill as a side trip, not progress

A bounded `[date_from, date_to]` re-pull returns rows and **does not advance the watermark**. Write each chunk under its own sub-prefix (`chunk=<from>_<to>`) so concurrent chunks cannot clobber each other, and so a failed chunk can be re-run alone.

L05 · Plan the Load Strategy per Table

Why

- **Day one is invisible when it is wrong.** There is no error. There is just a table that is quietly short, and every reconciliation built on it is wrong by an amount nobody can name — usually discovered when a business user disputes a number, months after the build.
- **An empty full pull that lands is worse than a crash.** It replaces good data with none, marks itself successful, and leaves the watermark intact so the next run sees nothing unusual.
- **Flipping to CDC early is the more dangerous of the two mistakes.** A late flip costs you a day. An early flip leaves a permanent hole in the middle of history, at a watermark that looks entirely plausible.
- **Classification is cheap and one-way.** Deciding the class takes ten minutes per table at design time. Discovering it was wrong takes a re-load of a giant table and a hard conversation.

What breaks if you don't: the error surfaces months after the build, in a total that cannot be reconciled against the system of record — and by then nobody remembers what day one looked like.

On Apparel Group

The classification exercise for all eight sources. **Do this as a table, in a document, before writing any specs.**

#	SOURCE	TABLE CLASS	LOAD STRATEGY
1	Oracle Retail (RMS) — facts (sales, receipts, transfers)	Incremental fact	Full initial load within horizon, then daily delta
1	Oracle Retail (RMS) — masters (item, supplier, location)	Full-only master	Truly full, no horizon , every run
1	Oracle Retail (RMS) — code/lookup tables	Full-only master	Full every run; cheap
2	Oracle SIM — inventory positions	Incremental fact	High churn — requires a true update stamp, not a creation date
3	Oracle XStore — POS transactions	Windowed giant	N date windows on day one, all must complete before CDC
4	Epsilon — loyalty / customer master (PII)	Incremental fact (cursor)	Full initial pull, then vendor cursor. Classify and mask before it reaches a report
5	MoEngage — campaign & engagement	Incremental fact (cursor)	Vendor cursor; events are append-only
6	Magento — orders, customers, products	Incremental fact	<code>updated_at</code> delta; products may be small enough for full-only
7	Vemco Footfall	Full-only master	Small per-store counts — full-refresh the file drop
8	Irisys Footfall	Full-only master	As Vemco

XStore is the one to plan first. It is the giant, it is the table that forces the windowed path to exist, and its day-one load will dominate the ingestion schedule for the first weeks of the build. Everything else is comparatively routine.

Two Apparel-Group notes:

- **Apparel dimensions multiply.** Style / colour / size / season means "master data" is bigger here than in most retail estates. Check the row counts before assuming a master table is small enough for full-only.
- **Epsilon is PII.** Its initial load is also the first time customer data lands in the lake. Have the masking and classification decision made *before* day one, not after.

L05 · Plan the Load Strategy per Table

Checklist

- ❑ Every table classified: full-only master / incremental fact / windowed giant
- ❑ Classification recorded in config, not in someone's head
- ❑ Initial load covers the full declared scope — never sub-windowed below the horizon
- ❑ Master and creation-date-keyed tables load with **no** horizon
- ❑ Row-count parity verified on **both** sides after the initial load
- ❑ Full-derivation clears any date window it was passed
- ❑ Chunked full-reload is an explicit opt-in flag
- ❑ Lifecycle state tracked: `(none)` → `initial_loaded` → `cdc`

- ❑ The **phase barrier** flips the state, once, when every window is terminal — no chunk can flip it
- ❑ A windowed initial load seeds its watermark forward-only from the window end
- ❑ Empty FULL = fail, **before landing**; empty DELTA = succeed
- ❑ `on_empty` override available per spec, with the two defaults above
- ❑ Backfills do not advance the watermark and write under their own chunk prefix

You've got it when you can...

...say why an empty pull is a **success** for one table and a **failure** for another on the same morning — and explain why declaring a windowed chunk complete is the more dangerous of the two possible mistakes.

Lay Out the Lake

Zones, buckets, namespaces and retention — decided once, before the first terabyte lands.

1 · THE DECISION

Zones, buckets, namespaces — how is storage carved up?

You get one attempt. The layout is fixed the moment the first terabyte lands.

DO · ZONE PER BUCKET

AVOID · ONE BIG BUCKET

2 · DO THIS

THE SETUP · FIVE DECISIONS, MADE ONCE

- 1 Cut four zones — raw, bronze, silver, gold — and give each one its own bucket.
a separate bucket is a separate policy, key and blast radius
- 2 Make raw immutable and replayable: write it once per cycle and never edit it.
`s3://ag-raw-<env>/<source>/<table>/cycle=<cycle_id>/`
- 3 Give every source system its own namespace. Never mix two systems in one.
`bronze.rms · bronze.sim · bronze.xstore · bronze.epsilon · bronze.magento`
- 4 Set lifecycle and retention when the bucket is created, not when it fills up.
`raw: long retention, then archive · bronze + silver: expire noncurrent`
- 5 Put the zone and the environment in every bucket name, so a path explains itself.
`ag-<zone>-<env>` — you can read the blast radius of a grant off the name

3 · WHY

Each zone carries a different retention, cost and access profile. A bucket boundary is the only place you can enforce all three at once.

IF YOU DON'T

Fixing the layout later means rewriting every path, spec, job and grant you have written.

4 · ON APPAREL GROUP

8 SOURCES → 8 NAMESPACES

`rms · sim · xstore · epsilon · moengage · magento · vemco · irisys`. XStore sets the size. Epsilon is PII: own prefix, own key.

IN PLAIN ENGLISH

Four separate warehouses, not four shelves in one room. Each one gets its own lock, its own bill and its own clear-out schedule.

L06 · Lay Out the Lake

Slide: `_render/L06-lay-out-the-lake.html`

The decision

Zones, buckets, namespaces, retention — how is storage carved up?

You will make this decision in week one, on a whiteboard, before a single row has landed. It looks like a naming argument. It is not. Every path you choose here gets baked into ingestion specs, transformation models, IAM policies, KMS key policies, lifecycle rules, catalog entries and the Power BI connection strings — and it gets baked in *permanently*, because moving terabytes between buckets means re-running everything that ever wrote them.

The two candidate answers:

	ONE BUCKET, PREFIXES INSIDE	FOUR ZONES, FOUR BUCKETS
Boundary	a string prefix	a real resource boundary
Access control	one policy, conditioned on paths	one policy per zone
Encryption	one key for everything	one key per zone
Cost visibility	one line on the bill	four lines on the bill
Retention	one lifecycle rule, or per-prefix rules	one rule per zone, obviously correct
Changing your mind	rewrite every path everywhere	still a migration, but a smaller one

Choose **four zones, four buckets**. The single-bucket layout is not wrong on day one; it is wrong on day four hundred, and by then it cannot be undone.

Do this

1. **Cut four zones — raw, bronze, silver, gold — and give each one its own bucket.** A separate bucket is a separate policy, a separate encryption key and a separate blast radius. `ag-raw-<env>` · `ag-bronze-<env>` · `ag-silver-<env>` · `ag-gold-<env>` . Put the zone *and* the environment in the name, so anyone reading a path or a policy can see what it touches without opening a console.
2. **Make raw immutable and replayable.** Raw is written exactly once per source, per table, per cycle, and never edited afterwards. Its job is to let you rebuild every downstream layer from scratch without going back to the source system — which matters enormously when the source is someone else's production Oracle database and you get one extraction window a night.

```
s3://ag-raw-<env>/<source>/<table>/cycle=<cycle_id>/part-*.parquet
```

The `cycle=<cycle_id>` component is doing real work: it makes a re-run overwrite *exactly one* cycle's worth of files. Without it, a retried download writes a second copy alongside the first and the duplication surfaces one layer up (see [L07](#)).

3. **Give every source system its own namespace, and never mix two systems in one.** One namespace per source, one table per source table:

```
bronze.rms · bronze.sim · bronze.xstore · bronze.epsilon  
bronze.moengage · bronze.magento · bronze.vemco · bronze.irisys
```

This is what makes "add a ninth source" an additive change rather than a refactor. It also gives you a natural grant boundary: a team that needs loyalty data gets `epsilon` , not "the bronze bucket".

L06 · Lay Out the Lake

4. **Set lifecycle and retention rules when the bucket is created**, in the same infrastructure code that creates it — not as a follow-up ticket. Raw gets a long retention with an archive transition (it is your replay tape and it is the cheapest data you own). Bronze and silver get noncurrent-version expiry so old copies do not accumulate. Gold is small and usually keeps everything.
5. **Decide the zone contract in one sentence each, and write it in the repo README:**
- **raw** — exactly what the source gave us, unmodified, replayable, partitioned by cycle.
 - **bronze** — one table per source table, typed, deduped, upserted on the declared key.
 - **silver** — conformed and joined; business rules live here; source-neutral names.
 - **gold** — the star schema the BI tool reads. No raw column names survive to this layer.

The value is not the prose. It is that a reviewer can now reject a pull request with "that transformation belongs in silver, not bronze" and point at a line.

Worked example of the pattern: the Tamimi lakehouse uses exactly this shape — a raw S3 bucket partitioned by cycle, then Bronze and Silver as separate Iceberg table buckets with per-bucket lifecycle settings declared in `infra/modules/s3-data-lake/main.tf` , and one Bronze namespace per source system. Read the namespace list next to the spec directory (`src/glue/specs/bronze/`) and the one-to-one mapping is immediately visible.

Why

Each zone carries a genuinely different **retention, cost and access profile**. Raw is large, cold, rarely read and must be kept for a long time. Gold is small, hot, read constantly and can be rebuilt in an hour. Bronze sits in between. A bucket boundary is the only place where you can enforce retention, encryption and access for a zone *at once*, in one declarative place, and prove it to an auditor by pointing at a single resource.

The namespace rule buys something different: **independence**. When Magento's schema changes, nothing in the RMS namespace can be affected, because there is no shared object between them. Adding the ninth source touches nothing that already exists.

The lifecycle rule buys the thing nobody thinks about in week one: **you never get a good moment to add retention later**. By the time storage cost is noticeable, the bucket contains data whose deletability nobody is sure about, and the safe answer is always "keep it".

What breaks if you don't: fixing the layout later means rewriting every path, spec, job, policy and grant you have written — so it does not get fixed, and you live with it.

On Apparel Group

Eight sources, eight namespaces, one layout:

L06 · Lay Out the Lake

NAMESPACE	SOURCE	WHAT DRIVES THE LAYOUT DECISION
rms	Oracle Retail RMS	Large masters + transaction tables. Mostly full refreshes; modest storage.
sim	Oracle SIM	Inventory positions, high churn. Many writes, moderate volume.
xstore	Oracle XStore	The giant. POS transactions. Size raw retention and the storage bill against this one, not against RMS.
epsilon	Epsilon	Loyalty + customer master. PII — own prefix, own KMS key, own grant, from day one, not after a classification review.
moengage	MoEngage	Campaign and engagement events. API/file drop cadence.
magento	Magento	E-commerce orders, customers, products.
vemco	Vemco Footfall	Small per-store counts, frequent files.
irisys	Irisys Footfall	Same shape as Vemco. Keep them separate anyway — they are different vendors with different formats.

Three Apparel-Group-specific calls to make in week one:

- **Size raw retention against XStore.** Whatever retention you can afford for XStore is the retention policy; the other seven are rounding errors next to it. Decide the number before you sign off the storage estimate, not after the first invoice.
- **Epsilon is PII and it is not negotiable.** Its own prefix and its own key are what let you grant "loyalty analytics" to a team without granting them sales, and what let you delete a customer's data without touching seven

other namespaces.

- **Keep Vemco and Irisys apart even though they do the same job.** They are optional sources that may be swapped or dropped. Merging them into a `footfall` namespace saves one line of config and costs you the ability to remove one vendor cleanly.

Checklist

- ☐ Four zones exist as four buckets, each with the zone and the environment in its name
- ☐ Raw is write-once, partitioned by cycle id, and nothing ever edits it in place
- ☐ Every source system has its own namespace; no namespace holds two systems
- ☐ Lifecycle and retention are declared in the same infrastructure code that creates the bucket
- ☐ Each zone's contract is written down in one sentence, in the repo
- ☐ Epsilon (PII) has its own prefix and its own key, decided before first ingestion
- ☐ Raw retention was sized against XStore, not against an average table
- ☐ I can name the blast radius of any grant by reading the bucket name

You've got it when you can...

...draw the four zones and their buckets from memory, state each zone's one-sentence contract, explain why the cycle id belongs in the raw path, and answer "can we just use one bucket and sort it out with prefixes?" with **"we can, and in eighteen months we will not be able to un-do it"** — then name the three things a bucket boundary gives you that a prefix does not.

Make Every Load Idempotent

Replays, overlapping windows and retries are normal. Re-running must not change the row count.

1 · THE DECISION

How do you re-run a load without changing the row count?

Replays, overlapping windows and automatic retries are routine, not exceptional.

DO · DEDUPE, THEN MERGE

AVOID · PLAIN APPEND

2 · DO THIS

THE SETUP · FIVE STEPS TO AN IDEMPOTENT LOAD

- 1 Find the real primary key in the source DDL. Never infer it from a column name.
`prove it once: count(*) must equal count(distinct key)`
- 2 Declare that key on the spec as `merge_key`. No declared key, no load.
`merge_key: [store_id, business_date, transaction_no]`
- 3 Dedupe the source to one row per key before the MERGE ever sees the frame.
`row_number() over (partition by key order by ingested_at desc) = 1`
- 4 Upsert with MERGE — matched updates, unmatched inserts. Never a plain append.
`MERGE INTO t USING s ON t.key = s.key WHEN MATCHED / WHEN NOT MATCHED`
- 5 Make the landing write overwrite its own cycle prefix, so a retry replaces itself.
`one cycle id · one prefix · one write — a rerun overwrites, never adds`

3 · WHY

Retries and overlapping windows are routine, and append-only turns every one of them into duplicate rows. A one-match rule is the engine protecting you.

IF YOU DON'T

The very first write can land duplicate keys, and once they are in, they are permanent.

4 · ON APPAREL GROUP

PROVE THE PK BEFORE THE SPEC

RMS, SIM and XStore all use composite keys — read the DDL. Magento keys order plus line; Epsilon and MoEngage on event id. Prove each.

IN PLAIN ENGLISH

Idempotent means pressing the button twice changes nothing. The key is the promise; MERGE is how the engine keeps it.

L07 · Make Every Load Idempotent

Slide: [_render/L07-idempotent-loads.html](#)

The decision

How do you re-run a load without changing the row count?

Not *if* you re-run it. You will re-run it. A cross-network extraction from someone else's production Oracle database times out; the orchestrator retries; the extraction window overlaps with yesterday's by an hour because someone widened it to catch late data; a backfill re-processes a month; a developer runs the same job twice while testing. Every one of those is a normal Tuesday, and every one of them delivers the same rows to your writer a second time.

So the design question is not "how do we avoid re-running" — it is "**what does the load do the second time?**"

	APPEND-ONLY	DEDUPE, THEN MERGE
Second run	adds rows	changes nothing
Row count after retry	doubled	identical
Failure signal	none — it looks fine	a loud cardinality error if the source is dirty
Fixing a bad day	delete-and-reload, by hand	re-run the job
Cost of getting it wrong	permanent, silent duplication	one failed job

Choose **dedupe, then MERGE**. An append-only load is a load you can never safely retry, which means it is a load nobody can operate.

Do this

1. **Find the real primary key in the source DDL**. Not the column called `id` , not the column that "looks unique", not the one the source team says is unique. Pull the constraint definition, and then prove it on real data before you write anything:

```
SELECT COUNT(*), COUNT(DISTINCT (col_a, col_b, col_c)) FROM source_table;
```

If those two numbers differ, you have not found the key yet — keep going. This step takes ten minutes per table and it is the single highest-value ten minutes in the whole ingestion build.

2. **Declare that key on the spec as `merge_key`** . It belongs in configuration, next to the table definition, where a reviewer can see it:

```
table: xstore.transactions
merge_key: [store_id, business_date, transaction_no]
```

Make the engine **refuse to run** a table that has no declared key rather than fall back to a guess. A guessed key produces a load that appears to work and silently corrupts.

3. **Dedupe the source frame to exactly one row per key, before the MERGE ever sees it**. Rank and keep the winner deterministically — do not use an arbitrary "drop duplicates", because on a re-run you want the *same* row to win both times:

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY store_id, business_date, transaction_no
    ORDER BY _ingested_at DESC, _run_id DESC) AS _rn
  FROM source
) WHERE _rn = 1
```

L07 · Make Every Load Idempotent

Say out loud *why* this is safe for your load shape. For a watermark-range delta or a full snapshot, the source carries one current-state row per key, so collapsing duplicates only ever drops identical copies. For a true change stream carrying delete markers it is **not** safe, because collapsing would discard a delete. Different shape, different rule.

4. **Upsert with MERGE** . Matched rows update, unmatched rows insert. Never a plain append on a keyed table:

```
MERGE INTO target t USING source s ON t.key = s.key
WHEN MATCHED      THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

If the feed carries delete markers, add the third arm — `WHEN MATCHED AND s.op = 'D' THEN DELETE` , and exclude already-absent deletes from the insert arm.

5. **Make the landing write overwrite its own cycle prefix**. The raw layer writes to `.../cycle=<cycle_id>/` and a retry of that cycle *replaces* what is there rather than adding beside it. One cycle id, one prefix, one write. This is the same discipline as the MERGE, one layer earlier — and it is where duplication is cheapest to stop.

6. **Watch out for the first write**. On a table that does not exist yet, `MERGE` cannot run: the engine creates the table and appends the frame verbatim. If you deduped in step 3, that is correct. If you skipped step 3 because "the first load is a clean snapshot", the duplicates become permanent from row one and the MERGE will never remove them. **Dedupe on the first load too**.

Worked example of the pattern: the Tamimi engine declares `merge_key` per Bronze spec, dedupes in the job immediately before the writer call, dedupes again inside `writers/s3_tables.py::merge_into()` with a

ranked window, and ranks a third time in the dbt staging models. Three layers, same rule, because the cost of a duplicate rises at every hop.

Why

Replays, overlapping windows and automatic retries are routine. **Append-only turns every one of them into duplicate rows**. And duplicates in a keyed table are not a storage problem — they are a correctness problem, because the very next join fans out. One duplicated row in a dimension multiplies every fact row that joins to it, and the resulting number is plausible, not obviously broken.

The MERGE's one-match constraint is worth understanding properly, because you will meet it as an error message. A target row may be claimed by **at most one** source row. Two source rows matching the same target is a cardinality violation and the job stops. That is not the engine being fragile; it is the engine refusing to guess which of two conflicting rows should win. **A cardinality violation is never a bug in the target — it is a duplicate that arrived with the source**, and the fix is always upstream.

There is a second reason, and it is operational rather than mathematical. A load you can safely re-run is a load an on-call engineer can fix at 3am by pressing one button. A load that is not idempotent requires someone to work out *what already landed* before they dare touch it — which is why non-idempotent pipelines are the ones that stay broken until morning.

What breaks if you don't: the very first write can land duplicate keys, and once they are in, they are permanent — the MERGE will faithfully update both of them forever.

On Apparel Group

Find the true primary key before you write the spec. Do not assume it.

L07 · Make Every Load Idempotent

SOURCE	WHERE THE KEY COMES FROM	WATCH OUT FOR
Oracle Retail RMS	Composite keys on almost every table — item/location, item/supplier, and so on. Read the DDL.	The "obvious" single-column key is usually the master key of a different table.
Oracle SIM	Inventory position keyed by item + location (+ status or bucket on some tables).	High churn — the same key is legitimately updated many times a day, which is exactly what MERGE is for.
Oracle XStore	Transaction identity is composite: store + business date + transaction number. Line detail adds a line sequence.	Transaction numbers recycle per store per day. A number alone is not unique.
Magento	Orders on order id; order lines on order id + line item id.	Guest checkouts and re-created carts can reuse identifiers across environments.
Epsilon	Loyalty/customer master on the customer identifier; events on an event id.	PII — dedupe and merge behaviour has to be consistent with your deletion process.
MoEngage	Campaign/engagement events on an event id.	At-least-once delivery is the vendor's normal mode — assume repeats.
Vemco / Irisys Footfall	Store + interval start timestamp.	Files are frequently re-delivered for the same day. Overwrite the cycle, then merge.

The three Oracle sources are where this lesson earns its keep: **RMS, SIM and XStore all use composite keys**, all three are being ingested by people who did not build them, and all three will happily return duplicate rows for a

re-run window. Produce the key table for all eight sources as the exercise, with a `COUNT(*)` vs `COUNT(DISTINCT key)` result next to each row. That table is a deliverable, not a note.

Checklist

- ☐ The primary key came from the source DDL, not from a column name
- ☐ `COUNT(*) = COUNT(DISTINCT key)` was proven on real data, and the result is recorded
- ☐ `merge_key` is declared in the spec; a table without one refuses to load
- ☐ The source is deduped to one row per key **before** the MERGE, including on the first load
- ☐ The dedupe picks a winner deterministically (ranked), not arbitrarily
- ☐ I can say why deduping is safe for *this* load shape
- ☐ Delete markers, if any, are handled by a third MERGE arm — not by dedupe
- ☐ The landing write overwrites its cycle prefix rather than adding beside it
- ☐ Running the job twice in Dev leaves the row count unchanged — verified, not assumed

You've got it when you can...

...read a cardinality-violation error and say, without opening the target table, "**two source rows share a merge key — something upstream landed twice**"; name the three places a mature pipeline dedupes and why each one exists; explain why deduping is safe for a snapshot delta but dangerous for a change stream carrying deletes; and prove your load is idempotent by running it twice and showing the row count did not move.

Plan Table Maintenance on Day One

Every write leaves litter behind. Decide now who sweeps it — the platform, or a job you ship.

1 · THE DECISION

Who compacts, and who expires the snapshots?

Maintenance has no error message. If nobody owns it, nobody notices it is missing.

DO · SHIP THE SCHEDULE

AVOID · WIRE IT LATER

2 · DO THIS

THE SETUP · FIVE THINGS TO SETTLE BEFORE GO-LIVE

- 1 Split maintenance into three jobs: compaction, snapshot expiry, file cleanup.
`three different jobs – never one vague word called "maintenance"`
- 2 Write down which of the three the platform runs for you, and which you own.
`managed compaction is common · scheduled snapshot expiry rarely is`
- 3 Schedule snapshot expiry weekly, keeping a retain floor so rollback survives.
`expire_snapshots(older_than_days=7, retain_last=100)`
- 4 Declare unreferenced-file removal on the bucket, in Terraform, at creation.
`unreferenced_days = 7 on bronze · 30 on silver – declared, not clicked`
- 5 Ship that schedule in the same pull request as the pipeline, or it never ships.
`count snapshots per table weekly · alarm when the count stops falling`

3 · WHY

Frequent small writes commit a new snapshot every cycle, and query planning walks that metadata chain before it reads a single row of data.

IF YOU DON'T

Reads decay for a year and nothing fails — there is no log line anyone can grep for.

4 · ON APPAREL GROUP

XSTORE AND SIM DECAY FIRST

Both commit on every cycle, so their metadata grows fastest — put them on the weekly expiry job. Vemco and Irisys are the small-file source.

IN PLAIN ENGLISH

Nothing errors — everything just gets slower. Book the cleaner before you move in, because nobody ever books one afterwards.

L08 · Plan Table Maintenance on Day One

Slide: `__render/L08-table-maintenance.html`

The decision

Who compacts, and who expires the snapshots?

Every write to a modern table format leaves two kinds of litter behind: **small data files**, and **a longer history**. Neither is a bug. Both are the direct consequence of committing frequently, which is exactly what a daily pipeline across eight sources does.

Somebody has to sweep up. The decision you are making is not *whether* — it is **who, and when do we wire it**.

	WIRE IT LATER	SHIP THE SCHEDULE WITH THE PIPELINE
When it gets done	never, because nothing fails	in the same pull request as the load
What prompts it	a performance complaint, twelve months in	the definition of done
Who can do it then	whoever can prove it is safe to delete history	the person who just wrote the table
Symptom in the meantime	queries slowly getting worse	none

Choose **ship the schedule with it**. The reason this decision is worth a whole lesson is the entry in the third column: maintenance is the one part of a data platform with **no error message**. If nobody owns it on day one, nobody notices it is missing, and the platform degrades for a year before anyone connects the symptom to the cause.

Do this

1. **Split "maintenance" into three named jobs**. The word is useless as a single item on a backlog. There are three distinct activities with three distinct owners:

JOB	WHAT IT DOES	TYPICALLY OWNED BY
Compaction	Rewrites many small data files into fewer large ones	the storage platform, if it offers managed maintenance
Snapshot expiry	Drops old table versions so metadata stops growing	you — this is the one that gets forgotten
Unreferenced-file removal	Deletes objects no snapshot points at any more	the platform, but <i>you</i> configure it

2. **Write down which of the three the platform runs for you, and which one you own**. Put it in the repo, next to the storage module, in a table exactly like the one above with a real owner in each row. Managed compaction is common on modern managed table storage; **scheduled snapshot expiry rarely is**. If you delegate compaction, also decide *not* to schedule your own compaction job — two compactors fighting over the same files is worse than none.

3. **Schedule snapshot expiry weekly, with a retain floor**. Expiry needs two parameters and both matter:

```
expire_snapshots(older_than_days=7, retain_last=100)
```

`older_than_days` is the age cut. `retain_last` is the safety net: it guarantees a minimum number of recent snapshots survive regardless of age, so time travel and rollback still work, and so a long-running query that resolved an older snapshot is not reading files you are deleting underneath it. Shape: a weekly cron → a job that iterates every table in every namespace → build a writer → call expire.

L08 · Plan Table Maintenance on Day One

4. **Declare unreferenced-file removal on the bucket, in infrastructure code, at creation.** This is a bucket-level setting, so it belongs in Terraform next to the bucket itself — shorter for the hot, high-churn zone, longer for the conformed zone. Declared, not clicked:

```
unreferenced_days = 7    # bronze
unreferenced_days = 30   # silver
```

Note the structural trap: if your tables register themselves on first write from a job (rather than being declared in Terraform), there is **no Terraform resource on which to set per-table maintenance**. Those settings stay at the service default and nobody notices. Know which of your settings are per-bucket and which are per-table, and know which ones you have no declarative place to put.

5. **Ship the schedule in the same pull request as the pipeline.** This is the whole lesson compressed into one habit. A maintenance job that is "the next ticket" is a maintenance job that does not exist. Add the observability with it: count snapshots per table weekly, and alarm when the count **stops falling** — that is the signal that your expiry job died silently, and it is the only signal you will get.

Worked example of the pattern: the Tamimi platform delegates compaction to managed storage maintenance, declares `iceberg_unreferenced_file_removal` per table bucket in `infra/modules/s3-data-lake/main.tf` (7 days on Bronze, 30 on Silver), and implements `expire_snapshots(older_than_days=7, retain_last=100)` in `writers/s3_tables.py`. Read it as three separate concerns with three separate homes — that separation is the pattern to copy.

Why

Frequent small writes are the cause, and they are unavoidable. Every cycle, every table, a Bronze load commits a new version; a Silver transform commits another. Each commit produces new data files and a new entry in the table's metadata chain. Eight sources × dozens of tables × daily × months is thousands of small files and a metadata history that only ever grows.

Query planning walks that metadata chain before it reads a single row of data. That is where the cost lands. The engine opens the table's root metadata, resolves the current snapshot, reads the manifest inventory, prunes on statistics — and all of that gets slower as the chain lengthens and the file count rises. Then the scan itself pays per-file overhead on thousands of tiny files instead of hundreds of large ones.

Note what does *not* happen: **nothing fails**. There is no exception, no failed job, no alarm, no log line. The symptom is "the dashboard feels slower than it used to", reported by a business user, twelve months after the cause was introduced. That is the worst possible failure shape — it survives indefinitely because nothing surfaces it.

One related trap worth carrying with you: because managed compaction **rewrites files**, the last-modified timestamp on a storage object is not a build timestamp. It can read newer than your last real write. Triage data staleness from your job-run history or your control plane, never from an object timestamp.

What breaks if you don't: reads decay for a year and nothing fails — there is no log line anyone can grep for.

On Apparel Group

XStore and SIM decay first. Rank the eight sources by commit frequency, because commit frequency — not data volume — is what drives metadata growth:

L08 · Plan Table Maintenance on Day One

SOURCE	COMMIT CADENCE	MAINTENANCE PRIORITY
Oracle XStore	Every cycle, high volume	First. Biggest tables <i>and</i> frequent commits. On the weekly expiry job before go-live.
Oracle SIM	Every cycle, high churn	First. Inventory positions change constantly; the metadata chain grows fastest here.
Vemco / Irisys Footfall	Frequent small files	The classic small-file source. Tiny payloads, many writes. Exactly the shape managed compaction exists for — confirm it is actually enabled on their tables.
Magento	Daily, moderate	Standard weekly expiry.
Epsilon / MoEngage	Daily API or file drop	Standard weekly expiry. Note that PII deletion obligations interact with snapshot retention — old snapshots still contain deleted rows. Decide the retention number with that in mind.
Oracle Retail RMS	Masters refreshed rarely	Lowest priority. Full-refresh masters commit less often and can wait.

Two Apparel-Group-specific calls:

- **Enable and verify managed compaction on the footfall tables specifically.** They are optional, small, and therefore the ones most likely to be set up quickly and never checked. They are also the ones that generate the most files per byte.
- **Set the expiry window with Epsilon in mind.** Snapshot history is a copy of deleted rows. A retention window that is fine for sales data is a compliance question for loyalty data — decide it once, deliberately, with

whoever owns data protection.

Checklist

- ☐ "Maintenance" is written down as three named jobs, never as one word
- ☐ Each of the three has a named owner: the platform, infrastructure code, or a person
- ☐ Snapshot expiry runs on a schedule that exists **now**, not in a ticket
- ☐ Expiry has both an age cut and a `retain_last` floor, and I can say what the floor defends
- ☐ I am not running my own compaction against managed compaction
- ☐ Unreferenced-file removal is declared in infrastructure code, per bucket, at creation
- ☐ I know which settings have no declarative home because tables self-register
- ☐ Snapshot counts per table are recorded weekly, with an alarm when the count stops falling
- ☐ Staleness triage uses job-run history, never an object's last-modified timestamp
- ☐ The maintenance job shipped in the same pull request as the pipeline

You've got it when you can...

...split "maintenance" into **compaction / unreferenced-file removal / snapshot expiry**, name the owner of each on your platform without looking it up, state what `retain_last` is protecting and from what, and describe the symptom of skipping expiry as **"query planning gets slower the longer a table lives"** rather than as any error a person would find in a log.

Make Schema Change a Controlled Event

The source will add a column. Decide now whether your pipeline stops, or quietly absorbs it.

1 · THE DECISION

What happens when a source adds or widens a column?

It will happen. The only question is whether the pipeline stops or absorbs it silently.

DO · FAIL LOUDLY

AVOID · SILENT DRIFT

2 · DO THIS

THE SETUP · FIVE RULES FOR A CONTROLLED CHANGE

- 1 Set the drift policy to fail on every model. Unexpected change stops the build.
`on_schema_change: fail – never ignore, never sync_all_columns`
- 2 Classify the change: a new last column is additive, everything else is a migration.
`drop · rename · retype · reorder · narrow – all migrations, all planned`
- 3 Allow a schema merge only on writes that rewrite every row, never a partial one.
`full refresh and create-or-replace only · never append, never merge`
- 4 Pre-cast numerics to one stable width at the edge of every model and union leg.
`cast(amount as numeric(38,4)) – the resolved type can then never drift`
- 5 Set the partition spec when the table is created; a later edit changes nothing.
`partitioned by (months(business_date)) – old data never moves`

3 · WHY

Silent absorption produces a plausible number, not an error — and nobody reviews a report that still renders. A loud failure costs an hour.

IF YOU DON'T

A widened column shifts totals quietly, and finance finds it long before engineering does.

4 · ON APPAREL GROUP

ORACLE NUMBER IS THE ONE TO WATCH

Oracle NUMBER carries no fixed width: pin RMS, SIM and XStore amounts to numeric(38,4) at staging. Magento and MoEngage add fields often.

IN PLAIN ENGLISH

A build that goes red costs an hour. A number that goes wrong costs a quarter, and nobody notices until someone asks why.

L09 · Make Schema Change a Controlled Event

Slide: [_render/L09-schema-change.html](#)

The decision

What happens when a source adds or widens a column?

It will happen. Oracle Retail gets a patch; the Magento team ships a feature and a new field appears on the order; MoEngage adds an attribute to an event payload; someone widens a numeric column from six digits to twelve because a currency conversion overflowed. None of these people know your pipeline exists.

So the decision is not whether to handle schema change. It is **what your pipeline does at the moment of the mismatch**:

	ABSORB IT SILENTLY	FAIL LOUDLY, THEN DECIDE
The run	succeeds	goes red
The table	quietly changes shape	unchanged
Downstream	keeps serving numbers	keeps serving <i>known-stale</i> numbers
Who finds out	a business user, months later	you, in the next ten minutes
Cost	a quarter of wrong reporting	an hour

Choose **fail loudly**. Silent absorption is seductive because it keeps the build green — and that is precisely the problem. It converts a schema change into a data-quality problem with no timestamp and no owner.

There is a subtlety that makes loud failure even more clearly correct. When a write fails, **the old table stays readable**. Downstream reports keep rendering, keep serving pre-change numbers, and nothing tells the reader

they are stale. So the choice is not "broken vs working" — it is "loud and stale" vs "quiet and wrong". Loud and stale is the one you can act on.

Do this

1. **Set the drift policy to fail, on every model.** Make it a project-level default so nobody has to remember it per model, and comment it so nobody "fixes" it later:

```
models:
  +on_schema_change: fail    # NEVER silently absorb schema drift
```

The alternatives — ignore, append-new-columns, sync-all-columns — each let a mismatch pass quietly into a table your BI tool is reading. Do the same at the ingestion layer: guard the write, and raise an error that **names the table and the columns**, so the person reading the log knows what to do without opening the code.

2. **Classify the change before you touch anything.** There are exactly two categories and the sorting rule is one sentence:
 - **Additive** — a new column at the end. Cheap: it is a metadata edit, existing data files stay valid, nothing is rewritten, and old rows read back null. **Backfill it before anyone depends on it**, because a null meaning "not populated yet" will be read as a business value.
 - **Everything else is a migration** — drop, rename, retype, reorder, narrow. Each one changes what existing rows *mean*, and each needs a plan, a window and a verification query.

Say which category you are in out loud before writing code. The common failure is treating a migration as if it were additive.

L09 · Make Schema Change a Controlled Event

3. **Allow a schema merge only on writes that rewrite every row.** Full-refresh and create-or-replace paths may carry a merge-schema option, because they replace the entire table and the new column lands populated everywhere. Append, partition-overwrite and merge paths must **not** get it: they touch a subset of rows and would leave the rest null in the new column — a table that is half-migrated and reports no error. Pin that distinction with a unit test so a future change cannot quietly grant the option to a partial-write path.

Refuse drops by name. A schema merge is a *union*, so a column the frame stopped emitting survives on the target and gets nulls written over real values. Raise before the write, naming the missing columns.

4. **Pre-cast numerics to one stable width at the edge of every model.** This is the trap that catches people who did everything else right. A model that unions several legs resolves its column types from **whichever legs carried rows on that build**. Build once with only the integer legs present and the column materialises as an integer type; the next run, once a fractional leg has data, the type widens — and your fail-on-drift policy trips, correctly, on a model nobody changed.

```
CAST(amount AS NUMERIC(38,4)) AS amount  -- every leg, every time
```

Cast every leg to the same explicit type, at the edge of the model, not in the middle. Then the resolved type cannot drift regardless of which legs have rows. The same rule prevents the mirror-image failure — "union types numeric and character varying cannot be matched" — when a text-typed source column meets a numeric one.

5. **Set the partition spec when the table is created.** Partitioning is consulted **only** on first write, when the create-table statement runs. Later writes inherit the existing layout, so editing the partition setting in a spec for a live table changes nothing at all, and no existing data ever moves.

```
PARTITIONED BY (months(business_date))
```

That means: get it right at creation, and treat "we need to re-partition" as a table rebuild with a migration plan, not a config edit. Write the intended partition column into the spec review checklist for every new table.

Worked example of the pattern: the Tamimi lakehouse sets `on_schema_change: fail` as a Gold default in `src/dbt/dbt_project.yml` , guards non-additive drift in `writers/s3_tables.py` with an error that names the table and columns, passes the merge-schema option only on the full-rewrite writer paths, and pre-casts every union leg in `unified_customer_count.sql` to a single `NUMERIC(38,4)` . Four separate mechanisms, one policy.

Why

Silent absorption produces a plausible number, not an error. That is the entire argument. An error is reviewed by an engineer within minutes because a build is red and someone is paged. A plausible number is reviewed by nobody, because the report still renders, the totals still look like totals, and there is no mechanism anywhere in the stack that flags "this figure is 4% lower than it should be".

The asymmetry is severe:

- A loud failure costs **an hour** — read the error, classify the change, apply the migration or the backfill, re-run.
- A silent absorption costs **a quarter** — because that is how long it takes for someone to notice, plus the time to work out when it started, plus the credibility you spend explaining it.

And you pay the second cost with interest, because by the time it is found, decisions have been made on the wrong numbers.

L09 · Make Schema Change a Controlled Event

The type-widening trap deserves its own note, because it is the one that feels unfair. Nobody changed the model. Nobody changed the source. The build simply went red on the second run because a union leg started carrying rows. The lesson is that **a resolved type is not a stable type** — if you did not write the type down, the engine inferred it from data, and inferred types change when data changes. Pinning the cast is how you convert an inference into a declaration.

What breaks if you don't: a widened column shifts totals quietly, and finance finds it long before engineering does.

On Apparel Group

Oracle `NUMBER` is the one to watch. It carries no fixed precision or scale unless the DDL specified one, which means every JDBC read has to decide a width, and different readers on different days can decide differently.

SOURCE	THE SCHEMA-CHANGE RISK	WHAT TO DO ABOUT IT
Oracle Retail RMS	<code>NUMBER</code> columns on cost, price, margin. Decades of patches add columns.	Pin every amount to <code>NUMERIC(38,4)</code> at the staging edge. Expect additive columns; fail on anything else.
Oracle SIM	<code>NUMBER</code> quantities; unit-of-measure changes ripple into meaning.	Same cast. Treat a UoM change as a migration, never as additive.
Oracle XStore	The largest surface: transaction, line, tender, tax columns.	Same cast. Because it is the giant, a full refresh here is expensive — get the types right before first load, not after.
Magento	Ships features regularly; new order/customer attributes appear without notice.	Fail on drift, then add the column deliberately. This is the source most likely to trip the policy.
MoEngage	Event payloads gain attributes as marketing configures campaigns.	Same. Treat "a new attribute appeared" as an ordinary, expected, <i>reviewed</i> event.
Epsilon	Loyalty attributes; PII classification can change with the schema.	A new column may be PII. Classification is part of accepting the change, not a follow-up.
Vemco / Irisys Footfall	Small and stable, but vendor file formats change on upgrade.	Fail on drift. A silently-absorbed footfall column is a silently-wrong conversion rate.

The practical Apparel Group rule: **pin RMS, SIM and XStore amounts to `NUMERIC(38,4)` at staging, before the first union is written.** Do it once, at the edge, on every leg. It costs nothing and it removes the single most common cause of a model that builds fine on Monday and goes red on Tuesday.

Checklist

- ☐ Fail-on-drift is the project-level default, with a comment saying why
- ☐ The ingestion writer guards drift and raises an error naming the table and columns
- ☐ I classified this change as additive or migration **before** writing code
- ☐ An added column is backfilled before anything reads it
- ☐ Merge-schema is enabled only on full-rewrite paths, and a test pins that
- ☐ Dropped columns are refused by name, not absorbed as a union
- ☐ Every union leg is explicitly cast to the same width — no inferred numeric types
- ☐ The partition spec was chosen at creation and reviewed as part of the spec
- ☐ I know that editing partitioning on a live table does nothing, and I have not tried it
- ☐ A migration has a plan, a window and a verification query before it runs

L09 · Make Schema Change a Controlled Event

You've got it when you can...

...classify any proposed schema change as **additive (safe)** / **non-additive (a migration)** in one sentence; explain the choice of fail-over-absorb using the real failure shape — *"the stale table stays readable and keeps serving*

pre-change numbers"; state that partitioning is fixed at creation and that editing the spec later does nothing; and describe the type-pinning rule as "**pin every union leg to the same type so the resolved type cannot drift between runs.**"

Choose Set-Based or Ordered

Default to set-based work. Use an ordered fold only when meaning depends on row order.

1 · THE DECISION

Can this transformation be written as a join?

If yes, it stays set-based. If no, an ordered fold is a choice you must defend.

DO · SET-BASED FIRST

AVOID · ORDERED BY HABIT

2 · DO THIS

THE SETUP · FIVE STEPS

- 1 Write it as joins and aggregates first — always try this route.
`set-based is the default for every model you build`
- 2 Apply one test: does row N mean anything without rows 1...N-1?
`answer no → ordered fold · answer yes → keep the join`
- 3 Group on the smallest key that holds all the state, then fold in order.
`groupBy(receipt_keys).applyInPandas(fold)`
- 4 Make that group key equal the identity you emit downstream.
`split a group → two rows claim one identity`
- 5 Keep the fold pure — list in, lists out — and unit-test its edge cases.
`glue_engine/abap/baskets.py · pure core, no Spark, no AWS`

3 · WHY

Set-based work is declarative — the engine picks the plan and scales it for you. An ordered fold costs a shuffle and a Python hop per group.

IF YOU DON'T

Order-dependent logic squeezed into a join returns wrong numbers, not error messages.

4 · ON APPAREL GROUP

XSTORE · RMS · SIM

XStore POS transaction logs are the one real candidate — a receipt is an ordered block of rows.
RMS and SIM are pure set-based. Keep them joins.

IN PLAIN ENGLISH

A formula that works on any row in any order is set-based. A running balance — where each line needs the ones above it — is not.

L10 · Choose Set-Based or Ordered, Deliberately

Slide: [_render/L10-set-based-or-ordered.html](#)

The decision

Can this transformation be written as a join?

Almost everything you will build on the new platform can. Dimensions, facts, aggregates, conformed lookups — all of it is set-based work: you declare the result, the engine decides how to compute it, and it scales without you touching a config file.

A small number of things genuinely cannot. Some source formats are not tables at all; they are **ordered blocks of rows**, where a row's meaning depends on the rows that came before it. POS receipt logs and event streams are the usual suspects. For those you need an **ordered per-group fold**, and reaching for one is a deliberate, defended decision — not a first instinct.

The choice is binary and you should be able to state which side you are on before writing a line of code:

	SET-BASED	ORDERED FOLD
Shape	joins, aggregates, window functions	<code>groupBy(...)</code> then a per-group loop
Cost	planned and optimised for you	a shuffle + a serialisation hop per group
Test	compare result sets	unit tests on a pure function
Use when	row meaning is independent	row <i>N</i> needs rows <i>1...N-1</i>

Do this

1. **Write it as joins and aggregates first.** Set-based SQL or DataFrame code is the default for every model on the platform. Do not start by asking whether ordering would be easier.
2. **Apply one test before you deviate:** *does row N mean anything without rows 1...N-1?* If the answer is yes, it stays set-based — no exceptions, no "but it would be simpler as a loop".
3. **If it fails the test, group on the smallest key that still holds all the state**, then fold that group in order. The whole group is materialised in memory on one worker, so the key must be small enough that its largest group comfortably fits. A receipt is tens of rows — fine. A store-month is not.
4. **Make the group key equal the identity you emit.** If the fold groups on one set of columns and emits a row identified by another, a single logical entity can be split across two groups and each half will emit its own record under the same identity. Grain damage, no error.
5. **Keep the fold pure and unit-test it.** A list of dicts in, lists out. No Spark session, no cloud SDK, no I/O inside the function. That is what makes the edge cases — incomplete blocks, repeated items, sign flips, uncommitted records — testable in milliseconds instead of in a 40-minute cluster run.
6. **Write down what you deliberately did not port.** A deferral you can name is a plan. A deferral you cannot name is a bug waiting to be discovered by a business user.

Worked example of the pattern: `tamimi-lakehouse/src/glue/glue_engine/abap/baskets.py` is the pure fold (no Spark, no AWS), and `.../abap/ops.py` is the thin Spark wrapper that groups, folds and caches around it. Read them in that order — the pure core first.

L10 · Choose Set-Based or Ordered, Deliberately

Why

Set-based work is **declarative**: you state the result and the engine chooses the plan, the parallelism and the join strategy. It scales as the data grows without you re-tuning anything. That is a large amount of free engineering, and you give it up the moment you write a loop.

An ordered fold buys you something real — arbitrary sequential logic in plain code — and charges you for it: every row shuffles across the network to its group's partition, each group is serialised into the fold's runtime and the results serialised back. Worth it for a receipt. Ruinous on a key whose largest group is millions of rows.

What breaks if you don't: order-dependent logic squeezed into a set-based join does not raise an error — it returns numbers that are quietly wrong.

The converse costs you too. Logic that *is* naturally set-based, written as a fold out of habit, turns a query the optimiser would have handled into a shuffle you now own, maintain and pay for.

On Apparel Group

SOURCE	VERDICT
Oracle XStore	The one real candidate. POS transaction logs are ordered blocks — a header, its line items, its tender rows, a commit — and nothing should be emitted until the block completes. Fold this one, on the smallest receipt-identifying key.
Oracle Retail RMS	Pure set-based. Master and transaction tables, joins all the way.
Oracle SIM	Pure set-based. Inventory positions, high churn, still just joins.
Magento	Set-based. Orders and line items are already relational.
Epsilon · MoEngage	Set-based. Campaign and loyalty extracts arrive as records, not sequences.
Vemco · Irisys Footfall	Set-based. Small per-store counts.

So: **one fold on the whole platform, and eight sources of joins**. If a second candidate appears, that is the signal to re-run the test in step 2 rather than to assume folds are now normal.

Apparel-specific caution on the XStore fold: a receipt in an apparel business routinely contains the same style in two sizes, or the same SKU scanned twice. Decide *deliberately* whether the fold merges those into one line or keeps them separate, and unit-test whichever you chose — that is a rule, not an implementation detail.

Checklist

- ☐ I tried to write it as joins before considering anything else
- ☐ I can state, in one sentence, why row order does or does not carry meaning here
- ☐ If ordered: the group key is the smallest one that holds all the state
- ☐ If ordered: the group key **equals** the identity the fold emits
- ☐ If ordered: the fold is a pure function — no Spark, no SDK, no I/O
- ☐ If ordered: edge cases have unit tests (incomplete block, repeat, sign flip)
- ☐ The largest expected group fits comfortably in one worker's memory
- ☐ Anything deliberately not implemented is named in the docstring

L10 · Choose Set-Based or Ordered, Deliberately

You've got it when you can...

...be handed an unfamiliar source table and say, within a minute, **"this is a join"** or **"this needs an ordered fold"** — and, if it is the fold, immediately name the group key, the identity it emits, and the one edge case you are

going to write a test for first.

Reproduce Source Rules Faithfully

Get the rule, copy it exactly — quirks included — and prove parity before anyone builds on it.

1 · THE DECISION

Re-implement the source rule, or re-derive it?

Only one answer is safe: get the rule from the source and copy it.

DO · REPRODUCE EXACTLY

AVOID · IMPROVING IT

2 · DO THIS

THE SETUP · FIVE STEPS

- 1 Get the rule as an artefact: source code, a spec, or a signed-off statement.
`no artefact → no model · inference is not a rule`
- 2 Reproduce it exactly, including quirks the business already reconciles to.
`a shipped quirk is the accepted answer, not a defect`
- 3 Prove parity with a reconciliation query before anyone builds on it.
`source total vs lakehouse total, same window, signed off`
- 4 Refuse to copy a quirk that drops rows — that one needs a deviation.
`lost rows are unrecoverable once gold stops reading bronze`
- 5 Record every intentional deviation in the docstring and in a test.
`glue_engine/abap/helpers.py · deviations named in code`

3 · WHY

The lakehouse must agree with the system of record before it is allowed to be better than it. Parity first; improvements are a later, named change.

IF YOU DON'T

“We improved it while porting” is how a lakehouse quietly disagrees with the system of record.

4 · ON APPAREL GROUP

ORACLE RETAIL RMS

Oracle Retail carries decades of embedded pricing and costing rules. Get them from the people who own them — do not infer them from the output.

IN PLAIN ENGLISH

Porting a rule is translation, not editing. You may disagree with the original — but in writing, and only once the numbers match.

L11 · Reproduce Source Business Rules Faithfully

Slide: [_render/L11-reproduce-business-rules.html](#)

The decision

Do you re-implement the source rule, or re-derive it yourself?

When a number in the source system is produced by logic — a VAT split, a landed cost, a margin, a store classification, a discount hierarchy — you have two ways to get that number into the lakehouse. You can obtain the actual rule and reproduce it. Or you can look at the inputs and outputs and work out what the rule must be.

Only the first is safe. The second produces something that agrees with the source on the rows you sampled and disagrees on the rows you didn't.

There is a third thing people do that is worse than either: reproduce the rule, notice it does something odd, and fix it on the way through. That is how a lakehouse quietly stops agreeing with the system of record — and nobody finds out until a report is challenged in a meeting.

Do this

1. **Get the rule as an artefact before you write anything.** Source code, a functional spec, or a written statement signed off by whoever owns the number. One of those three. If you cannot obtain any of them, the model is blocked — escalate it as blocked. *Inference is not a rule.*
2. **Reproduce it exactly, including the quirks.** If the source's calculation has an off-by-one, an odd rounding order, or a field it never inspects, and the business already reconciles against that output — reproduce it. A

- shipped quirk is the accepted answer, not a defect. The pack sizes on the shelf labels and the totals in the existing reports came out of that quirk.
3. **Prove parity with a reconciliation query before anyone builds on it.** Same window, same filters, source total versus lakehouse total, with the acceptable tolerance written down. Get it signed off. This step is the entire point of the lesson — a "faithful port" nobody reconciled is just an assertion.
 4. **Refuse to copy a quirk that loses rows.** This is the one deviation you take without hesitation. A rule that silently drops records — an inner join to a lookup that has no matching row, a default filter that excludes a whole document class — is not a rule you can reproduce, because downstream layers stop reading the raw data and the loss becomes unrecoverable. Keep the rows, move the *scoping* decision downstream where it is visible and reversible.
 5. **Do not mistake a source idiom for a bug.** Some conventions read as defects to an outsider: an empty selection range that means "no restriction" rather than "no rows", a screen default that is a suggestion rather than a filter. Encoding one of those as a hard filter drops real data. When something looks wrong, ask before you port it.
 6. **Record every intentional deviation in two places: the docstring and a test.** The docstring says what you diverged from and why. The test fails if someone later "fixes" it back. One without the other does not hold.
 7. **Fail loudly on unknown inputs.** A ported rule that meets a value it does not recognise should raise, not silently fall back to a default. A default is how a typo restores a bug you already removed.

Worked examples of the pattern: in [tamimi-lakehouse/src/glue/glue_engine/abap/helpers.py](#) you can see all three outcomes side by side — a rule reproduced exactly (including its documented off-by-one), a rule that is a derivation rather than a lookup, and a deliberate refusal to reproduce source behaviour that dropped rows. Each one states which it is in its docstring.

L11 · Reproduce Source Business Rules Faithfully

Why

The lakehouse's first obligation is **agreement with the system of record**. Until the numbers match, every improvement you make is indistinguishable from a defect — because the only test anyone can apply is "does it match the system we already trust?"

Parity first. Improvements afterwards, as a named change with its own sign-off and its own communication. That ordering is not bureaucracy; it is what makes an improvement *visible* rather than a mystery discrepancy.

What breaks if you don't: "we improved it while porting" is how a lakehouse quietly disagrees with the source — and the disagreement surfaces in a meeting, not in a test.

There is one asymmetry worth internalising, because it decides every hard case:

Does the divergence change a number people already reconcile against, or does it lose data you can never get back?

Reproduce the first kind. Refuse the second kind. Everything else is a judgement call you write down.

On Apparel Group

Oracle Retail RMS is decades of embedded business logic. Pricing, promotions, markdowns, landed cost, intercompany transfer valuation — these are not columns, they are calculations, and many of them have been amended repeatedly by people who have since left. This is exactly the environment where inference feels reasonable and is not.

Concretely, before building any priced or costed model:

- **Get the pricing and costing rules from the people who own them** — the Oracle Retail functional team or DBAs — as code or as a signed statement. Book that conversation in week one; it is a long-lead item, not a

detail.

- **Reconcile against RMS itself**, at a stated grain and window, before any downstream model consumes the output.
- **Watch for rules that are time-dependent.** Tax rates, cost methods and promotional logic change on effective dates. A flat constant applied to all history silently misprices every row from before the change.
- **XStore, SIM and Magento each carry their own version of "the price"** — ticket price, promotional price, price actually paid. Name which one each model means, in the model header. In apparel these differ constantly because markdown is the business model.
- **Epsilon loyalty accruals are rules too** — points earned, tier thresholds, expiry. Same treatment: get the rule, reproduce it, reconcile it.

Checklist

- ☐ I have the rule as code, a spec, or a signed-off written statement
- ☐ I reproduced it exactly, quirks included
- ☐ A reconciliation query exists, with a stated window, grain and tolerance
- ☐ The reconciliation has been run and signed off *before* downstream work started
- ☐ Any quirk I refused to reproduce is one that **loses rows**, and that is stated
- ☐ Every intentional deviation is in the docstring **and** guarded by a test
- ☐ Unknown / unmapped input values raise rather than silently defaulting
- ☐ Time-dependent parameters (rates, costs) are looked up by date, not hardcoded

L11 · Reproduce Source Business Rules Faithfully

You've got it when you can...

...hold both sentences at once — "we copied the source's quirk on purpose" and "we refused to copy the source's quirk" — and state the rule that decides between them: *does the divergence change a number people*

already reconcile against, or does it lose data we can never get back?

Write Spark That Scales

Five rules of thumb that decide whether a job finishes in minutes or dies at the timeout.

1 · THE DECISION

Why is this job slow — and what do you change first?

Almost always one of five things. Measure to find out which one.

DO · MEASURE FIRST

AVOID · BLIND TUNING

2 · DO THIS

THE SETUP · FIVE RULES OF THUMB

- 1 Cache any frame you consume more than once. A plan re-runs per action.
`df.cache()` before `count` / `agg` / `write`
- 2 Count the actions. `count`, `collect`, `first` and `write` each re-read the source.
`3 actions, no cache = 3 full pulls from Oracle`
- 3 Partition the read. Size for roughly 5–10 M rows per partition.
`partitions × concurrent tables ≤ source connection ceiling`
- 4 Never collect to the driver. Aggregate in the cluster and write out.
`collect()` on a fact table = one JVM, one heap
- 5 Look for skew before you touch config. A stage waits on its slowest task.
`one busy executor → skew, or a single-connection read`

3 · WHY

Spark is lazy: a DataFrame is a plan, and every action runs that plan from the top again.
Tuning without a measurement is superstition.

IF YOU DON'T
The job does not fail fast — it burns hours, then times out with nothing written.

4 · ON APPAREL GROUP

XSTORE IS THE GIANT

Size XStore first — it is the largest read by far.
RMS and SIM masters need almost no parallelism.
Budget connections per source, not per job.

IN PLAIN ENGLISH

A Spark job is a recipe, not a meal. Reading it out three times cooks dinner three times — unless you write the result down once.

L12 · Write Spark That Scales

Slide: [_render/L12-spark-that-scales.html](#)

The decision

Why is this job slow — and what do you change first?

Almost every slow or unstable Spark job on a lakehouse comes down to one of five things: an uncached frame read more than once, an unpartitioned source read, a shuffle you didn't budget for, work pulled back to the driver, or skew. The skill is not knowing more knobs; it is **identifying which of the five you have** before touching any of them.

The one thing you must not do is tune by guesswork. Doubling executors, raising `numPartitions` or increasing memory in response to a symptom you have not measured is how a two-hour job becomes a two-hour job that also costs more.

Do this

The five rules of thumb

1. **Cache any frame you consume more than once.** Spark is lazy: a DataFrame is a *plan*, and every action executes that plan from the top. A frame you count, then aggregate, then write, is read from the source **three times** unless you cache it.

```
df = read_source(...)
df.cache()           # one line, before the first action
n    = df.count()
```

```
hwm = df.agg(F.max("updated_at")).first()[0]
write(df)           # still one source read, not three
```

`MEMORY_AND_DISK` spills rather than failing, so caching is safe on the giants.

2. **Count the actions.** `count`, `collect`, `first`, `show`, `write` execute the plan. `select`, `filter`, `join`, `withColumn` only build it. Count the actions in a function and you have counted the source reads. This is the single highest-value habit in the lesson.
3. **Partition the read, and size the partitions.** A JDBC read with no partition column or predicates is **one connection**, however much parallelism the config claims. Target roughly **5–10 M rows per partition**, then check the number that actually matters:

$$\text{partitions per table} \times \text{concurrently-running tables} \leq \text{the source's connection ceiling}$$

Too few partitions means timeouts and memory pressure. Too many means you exhaust the source database's connection headroom and *every* read fails at once — which looks like a database outage, not a tuning problem. Add a guard that refuses to start when a table declares a large expected row count and has no partitioned read configured; a job that won't start beats a job that times out.

4. **Never collect to the driver.** `collect()` on a fact table moves the whole result into one process's heap. Aggregate in the cluster, write to storage, and read the small result back if you need it locally. The same applies to `toPandas()` on anything unbounded.
5. **Look for skew before you touch config.** A stage finishes when its *slowest task* finishes. One store-day with ten times the volume holds the entire stage, and no `numPartitions` setting fixes that. Salt the key, filter the outlier out and handle it separately, or change the grain.

L12 · Write Spark That Scales

Reading the symptom

WHAT YOU OBSERVE	WHAT IT USUALLY IS
Job time $\approx n \times$ a single read	a missing <code>.cache()</code> — n actions on one plan
One executor busy, the rest idle	skew, or a single-connection source read
OOM inside a Python worker	an <code>applyInPandas</code> group that is too big
Every source read fails at once	too many total connections, not too few
Fast in dev, dies in prod	volume-dependent: partitioning or skew

The order of operations

Measure → **identify which of the five** → **change one thing** → **measure again**. Read the Spark UI stage timeline before you read the config file.

Worked example of the pattern: `tamimi-lakehouse/src/glue/glue_engine/sources/sap_hana.py` shows the cache-before-multiple-actions shape and the partitioned-read predicate builder; `.../abap/ops.py` shows a frame cached because two downstream filters consume it.

Why

Spark’s laziness is the source of most of its surprises. A DataFrame is a description of work, not the work’s result, so “I already read that” is not true unless you said so explicitly. Once you internalise *plan, not data*, rules 1 and 2 stop being tips and become obvious.

The connection-ceiling arithmetic matters because the constraint lives outside your cluster. You can scale executors freely; you cannot scale the source database’s connection limit. Parallelism that ignores that

boundary converts a slow job into a source-wide outage.

What breaks if you don’t: the job does not fail fast — it burns hours of cluster time and then times out with nothing written, so you pay full cost for zero output, every cycle, until someone looks.

On Apparel Group

XStore is the giant. Size it first. POS transactions across the estate dwarf everything else on the platform, and it is the table where partitioning, caching and skew all bite at once. Get its read strategy right before optimising anything else — the other seven sources together will not cost what this one does.

- **XStore:** the largest read by a wide margin. Partition on an indexed, well-distributed column, size for 5–10 M rows per partition, and expect skew by store and by trading day. Sale weekends and season launches are real outliers, not noise.
- **RMS masters** (style, colour, size, season, supplier): small. Almost no parallelism needed. Partitioning these buys nothing and spends connections you need elsewhere.
- **RMS transactions and SIM:** middle-weight, high churn. Partition modestly; re-check as volumes grow.
- **Magento:** middling and spiky — e-commerce peaks are sharper than store peaks.
- **Epsilon · MoEngage:** API/file sources. The constraint is the vendor’s rate limit and page size, not Spark. Parallelism here is throttling, not tuning.
- **Vemco · Irisys Footfall:** tiny. Never worth partitioning.

Budget connections per source, not per job. All the Oracle sources may sit behind shared infrastructure and a shared ceiling. Work out the total concurrent connection count across every table running in the same window and keep it comfortably under the limit — then cap job concurrency to enforce it, rather than trusting each spec to be individually reasonable.

L12 · Write Spark That Scales

Checklist

- ☐ Every frame consumed more than once is cached, on its own line, before the first action
- ☐ I have counted the actions in this function and know the source-read count
- ☐ Large table reads are partitioned; partition size targets ~5–10 M rows
- ☐ partitions × concurrent tables is written down and is under the source ceiling
- ☐ No `collect()` / `toPandas()` on anything unbounded

- ☐ I checked the stage timeline for skew before changing any setting
- ☐ A guard fails the job at config time if a big table has an unpartitioned read
- ☐ I measured before tuning, and after

You've got it when you can...

...look at an unfamiliar Spark function and answer three questions without running it: **how many times does this read the source, where is the shuffle, and what is the biggest single group** — then point at the `.cache()`, the partitioning and the group key that each decide one of those answers.

Build Incremental Models Correctly

A merge is only as good as its key, its reprocess window, and the refresh scheduled behind it.

1 · THE DECISION

View, table or incremental — and merged on which key?

Facts go incremental with a merge. The key is the grain — and the grain must be unique.

DO · MERGE ON A REAL KEY

AVOID · NULL IN A KEY

2 · DO THIS

THE SETUP · FIVE STEPS

- 1 Write the grain in the model header, then make the merge key equal it.
`unique_key = the grain, column for column`
- 2 Prove the key is unique in the source before you trust it in a merge.
`too narrow → cardinality error · too wide → rows re-insert`
- 3 Use a sentinel, never NULL, in any column that forms part of the key.
`'__ALL__' instead of NULL on synthetic rollup rows`
- 4 Set the reprocess window wider than the source's late-arrival habit.
`reprocess_days as a variable, never a literal in the model`
- 5 Schedule a periodic full refresh. The window cannot repair older history.
`dbt run --select model --full-refresh · monthly cadence`

3 · WHY

NULL never equals NULL in a merge predicate, so a NULL-keyed row matches nothing and re-inserts itself on every single run, forever.

IF YOU DON'T

The table grows quietly, every total drifts upward, and no run ever reports an error.

4 · ON APPAREL GROUP

CONFIRM ORACLE PKs FIRST

Confirm the true primary key on every RMS, SIM and XStore table before you write its spec. Composite keys rarely match the documentation.

IN PLAIN ENGLISH

A merge must recognise a row it has seen before. A blank in the key is an unreadable name badge — it gets a new seat every time.

L13 · Build Incremental Models Correctly

Slide: [_render/L13-incremental-models.html](#)

The decision

View, table, or incremental — and merged on which key?

- **View** — cheap to build, computed on every query. Right for thin staging layers over the lake.
- **Table** — rebuilt in full each run. Right while a model is small, and right forever for small dimensions.
- **Incremental + merge** — built once, then only new and changed rows are merged in. Right for facts, and it is the default for every fact on the platform.

Choosing "incremental" is really choosing **four** things, and each has its own failure mode:

PROMISE	FAILS WHEN
the merge key	it isn't genuinely unique, or it contains a NULL
the reprocess window	a correction lands outside it
the schema-change policy	column types drift between runs
the refresh cadence	nobody scheduled the full refresh

The key is the grain. Get that one right and the rest are configuration.

L13 · Build Incremental Models Correctly

Do this

1. Write the grain in the model header, in one sentence, then make the merge key equal it — column for column.

```
-- Grain: one row per store × business date × style × season × scenario
{{ config(
    materialized = 'incremental',
    incremental_strategy = 'merge',
    unique_key = ['store_id', 'business_date', 'style_id', 'season_id', 'scenario'],
    on_schema_change = 'fail'
) }}
```

If the header sentence and the `unique_key` disagree, one of them is wrong and you do not yet know which.

2. Prove the key is unique in the source before you trust it in a merge. Run the duplicate-check query, on real volumes, over a real window:

```
SELECT store_id, business_date, style_id, season_id, scenario, COUNT(*)
FROM    {{ ref('stg_sales') }}
GROUP BY 1,2,3,4,5 HAVING COUNT(*) > 1
```

Too narrow and several source rows match one target row — the engine raises a cardinality error. **Too wide** and the match never fires, so rows re-insert instead of updating. The first is loud; the second is silent, which makes it worse.

3. Use a sentinel, never NULL, in any column that forms part of the key. If a model carries synthetic rollup rows — an "all departments" or "all styles" total — give that column a real marker value:

```
SELECT store_id, business_date, style_id, season_id, scenario, COUNT(*)
FROM    {{ ref('stg_sales') }}
GROUP BY 1,2,3,4,5 HAVING COUNT(*) > 1
```

```
'__ALL__' AS style_id      -- NOT: NULL AS style_id
```

This is the single most expensive mistake in the lesson, and it is invisible: see **Why**.

4. Set the reprocess window wider than the source's late-arrival habit, and make it a variable.

```
vars:
    reprocess_days: 45      # widen per-run with --vars; never a literal in the model
```

Find out empirically how late corrections actually arrive from each source, then set the window comfortably past that. Ops can widen it for a single run; they cannot widen it retroactively.

5. Schedule a periodic full refresh, and write down its cadence.

```
dbt run --select fct_sales --full-refresh      # monthly, and after any source period re-open
```

The window is a backstop for routine lateness, not a substitute for a rebuild. Anything older than the window is *only* recoverable this way.

6. Mind the shape the incremental predicate forces on your SQL. The predicate is appended as a `WHERE` on the final `SELECT`, and `GROUP BY` immediately followed by `WHERE` is a syntax error. Put aggregation in a CTE and end the model with a bare `SELECT * FROM ...`.

7. Pin column types when unioning legs. With `on_schema_change='fail'`, a first build that materialises a column as one numeric type and a later run that widens it will fail. Pre-cast **every** leg of a UNION to the same explicit type so the resolved type cannot drift between runs.

8. Guard the silent drops. If a filter sits on the right side of a LEFT JOIN, unmatched rows vanish without an error. Keep the join shape, and add a test that asserts nothing is being dropped.

L13 · Build Incremental Models Correctly

Worked example of the pattern: `tamimi-lakehouse/src/dbt/dbt_project.yml` sets the incremental + merge + `on_schema_change: fail` defaults and the reprocess-days variable; `src/dbt/macros/scenario_helpers.sql` builds the windowed predicate; `src/dbt/models/marts/gold/unified_sales.sql` shows the declared grain, the matching `unique_key` , and the sentinel on the rollup legs.

Why

NULL never equals NULL in a merge predicate. This is the whole lesson in one line. A row whose key column is NULL matches *nothing* in the target — including the identical row inserted by yesterday's run — so the merge treats it as new and inserts it again. And again. Every run, forever.

The consequence is uniquely nasty because nothing complains. No error, no failed test, no log line. The table grows a little each night, every total that includes those rows drifts upward, and the drift looks like business growth. By the time someone questions the number, months of reporting have been built on it.

The reprocess window has the mirror-image property: a correction that lands *outside* the window is never re-read, so the model keeps the stale value silently. Both failure modes are quiet, which is why both need a scheduled counter-measure rather than vigilance.

What breaks if you don't: the table grows quietly, every total drifts upward, and no run ever reports an error.

On Apparel Group

Confirm the true primary key on every Oracle table before you write its spec. Do not take it from the data dictionary, the vendor documentation, or an assumption — run the duplicate check yourself, on production-like volumes. Composite keys in Oracle Retail rarely match what the documentation claims, and a key that is unique in a sample is not necessarily unique across the estate.

Per source:

- **RMS** — composite keys are the norm and often include an organisational or set-of-books column that is invisible in a single-entity extract. Verify with that column and without it.
- **SIM** — inventory positions churn heavily; the key almost always includes a location and a snapshot dimension. Decide whether you are storing a snapshot per day or a current position, and make the key say so.
- **XStore** — receipt identity usually needs store *and* till *and* date *and* transaction number. A transaction number alone is not unique across the estate, and the store column is what people forget.
- **Magento** — order line IDs are usually genuinely unique; confirm rather than assume, especially across store views.
- **Epsilon · MoEngage** — API extracts often re-send the same record with an updated timestamp. The natural key is the customer or campaign identifier, not the delivery batch.
- **Footfall (Vemco · Irisys)** — store × interval. Small, but the interval boundary convention differs by vendor; make it explicit in the key.

Apparel-specific grain warning. Style, colour, size and season multiply. A fact declared at "style" grain and merged on a key that omits colour or size will collapse rows that are genuinely distinct — and a fact declared at SKU grain whose key omits season will collide across seasons that reuse a SKU. Write the grain sentence first; it prevents both.

Late arrivals: ask each source owner how late a correction realistically lands. Store-level POS adjustments, supplier invoice restatements and returns all have different tails, and the widest one sets your window.

L13 · Build Incremental Models Correctly

Checklist

- ☐ The grain is stated in one sentence in the model header
- ☐ `unique_key` equals that grain, column for column
- ☐ The duplicate-check query returns zero rows on production-like volumes
- ☐ No key column can ever be NULL; synthetic rows use a sentinel
- ☐ The reprocess window is a variable, and wider than the source's real late-arrival tail
- ☐ A full refresh is scheduled, with a documented cadence and owner

- ☐ Aggregation is in a CTE; the model's final statement is a bare `SELECT *`
- ☐ Every leg of any UNION is pre-cast to an explicit, identical type
- ☐ A uniqueness test on the merge key runs on every build

You've got it when you can...

...answer "*the fact table shows the wrong number for a day last quarter*" with the right first question — **is that day inside the reprocess window?** — and then name the three settings that could each have caused it: the merge key, the window, and the last time anyone ran a full refresh.

Guarantee Correctness by Construction

The dangerous failure is not an error. It is a plausible number that is quietly double.

1 · THE DECISION

How do you know the numbers are right?

Not by looking at them. By building tables whose grain cannot be violated.

DO · DECLARE THE GRAIN

AVOID · TRUSTING TOTALS

2 · DO THIS

THE SETUP · FIVE GUARANTEES

- 1 State the grain of every table in one sentence, in the file header.
`one row = one store × day × style × season`
- 2 Ask of every join: is my key the whole of the right side's primary key?
`if not, restrict the remaining key columns before joining`
- 3 Constrain language- and type-keyed lookups so they cannot fan out.
`filter the right side to exactly one row per join key`
- 4 Exclude synthetic rollup rows from every aggregate crossing that axis.
`WHERE dept != 'All Dept' – in every roll-up, every time`
- 5 Add a uniqueness test on every merge key, a reconciliation test on totals.
`rollup row = SUM(detail rows), asserted on every build`

3 · WHY

A fan-out or a double-counted subtotal does not raise an error. It returns a number that looks entirely normal and is exactly twice the truth.

IF YOU DON'T
You hear about it from a business user months later, after decisions were made on it.

4 · ON APPAREL GROUP

STYLE · COLOUR · SIZE · SEASON
Apparel dimensions multiply fast: one style becomes hundreds of SKUs. Grain discipline matters more here, not less.

IN PLAIN ENGLISH

A number that looks wrong gets fixed in an hour. A number that looks right but is double gets quoted in a board pack.

L14 · Guarantee Correctness by Construction ★

Slide: `__render/L14-correctness-by-construction.html`

The decision

How do you know the numbers are right?

Not by looking at them. A doubled total looks exactly like a good year. A fanned-out join looks exactly like more transactions. The dangerous failure on a lakehouse is **not an error** — it is a plausible number, produced by a job that succeeded, that is quietly twice the truth.

You cannot inspect your way out of this. The answer is to build tables whose grain *cannot* be violated, and then to assert the invariant on every build. Correctness by construction, backed by tests — not correctness by review.

Two failure shapes account for most of it, and they are worth naming because they need different fixes:

SHAPE	MECHANISM	FIX
Fan-out	a join returns more rows than the driving table has	restrict the join so it cannot
Two grains in one table	detail rows <i>and</i> their own subtotals coexist	discipline + a test; the extra grain is wanted

Do this

1. State the grain of every table in one sentence, in the file header

```
-- Grain: one row per store × business date × style × colour × size × season
```

If you cannot write that sentence, you do not yet know what the table is, and no test you write will mean anything. Write it before the SQL, not after.

2. Ask of every join: is my key the whole of the right side's primary key?

This is the question that prevents fan-out, and it takes ten seconds. If your join key is only *part* of the right-hand table's primary key, that table has multiple rows per key — and the join will multiply your driving rows by however many there are.

```
-- right side keyed (language, dept_code) – joining on dept_code ALONE
-- matches the English row AND the Arabic row: every fact row comes out twice
```

If the key is not the whole PK, **restrict the remaining key columns before joining**.

3. Constrain language- and type-keyed lookups so they cannot fan out

Description and code-decode tables are the classic offenders: they are keyed by *(code, language)* or *(code, type, version)*, and the column you actually want to join on is only one part of that.

```
LEFT JOIN dim_style_desc d
  ON d.style_id = f.style_id
  AND d.language = 'EN'      -- pins the remaining key column
  AND d.is_current          -- ...and any version/validity column
```

Two further rules for these joins:

L14 · Guarantee Correctness by Construction ★

- **Prefer LEFT over INNER when restricting.** Filtering a lookup to one language and keeping an INNER join silently drops every fact row whose code has no row in that language. Losing rows is worse than a NULL description — keep the fact, let the label be NULL, and make the *scoping* decision downstream where it is visible.
- **Project away what you don't need.** Select only the label columns from the right side, so a stray key column cannot re-introduce duplicates later.

4. Exclude synthetic rollup rows from every aggregate that crosses that axis

A fact table that deliberately contains both per-style rows **and** an "all styles" rollup has two grains on purpose — usually so BI measures resolve directly. That is a legitimate design *if it is written down*. But every roll-up over that axis must exclude the synthetic rows:

```
WHERE style_id != '__ALL__'      -- in every roll-up. Every time. No exceptions.
```

Forget it once and the KPI is roughly double. Put the reason in a comment next to the filter, not in a wiki.

Related trap: subtotals are not always summable. A distinct-count (customers, baskets, transactions) at the rollup level is **not** the sum of the per-dimension distinct counts, because one basket touches several styles. Take that measure from a source computed at the rollup grain, not by summing detail.

5. Add a uniqueness test on every merge key and a reconciliation test on every total

```
# uniqueness — the grain holds
- dbt_utils.unique_combination_of_columns:
  combination_of_columns: [store_id, business_date, style_id, season_id, scenario]
```

```
-- reconciliation — the rollup row equals the sum of its detail rows
SELECT store_id, business_date
FROM   ...
GROUP  BY 1, 2
HAVING ABS(MAX(rollup_value) - SUM(detail_value)) > 1.0
```

Run both on every build. And **test the config, not just the data** — assert that the join type is declared explicitly and that the language filter in the model matches the one the spec applies, so a typo cannot quietly restore a fan-out.

6. Know which invariants you cannot assert

Some relationships *look* like they must hold and don't — a per-dimension distinct count summing to at least the store-level count, for instance. If the source doesn't honour it, asserting it produces a permanently red test, and a permanently red test is no signal at all. Write down the ones you deliberately do not assert, and why.

Worked examples of the pattern: `tamimi-lakehouse/src/glue/specs/transform/zsdcc.yaml` shows a lookup join constrained by an explicit filter and an explicitly declared join type; `src/dbt/models/marts/gold/unified_sales.sql` declares its grain and its legs in the header; `src/dbt/tests/assert_all_dept_reconciles_to_per_dept.sql` is the reconciliation test in both directions.

Why

A fan-out or a double-counted subtotal **does not raise an error**. The job succeeds. No NULLs appear. Row counts go up, which looks like growth. The number is returned to a dashboard looking entirely normal — and being exactly twice the truth.

That is why review does not catch it and monitoring does not catch it. The only things that catch it are structural: a join that *cannot* multiply, and an assertion that runs on every build.

L14 · Guarantee Correctness by Construction ★

What breaks if you don't: you hear about it from a business user months later, after decisions were made on it — and then you have to explain not just the bug, but why nothing detected it.

The cost asymmetry is the real argument. A wrong number that looks wrong gets fixed in an hour. A wrong number that looks right gets quoted in a board pack, drives a buy, and destroys trust in the platform for a year. The whole discipline in this lesson exists to make the second kind impossible rather than unlikely.

On Apparel Group

Apparel dimensions multiply faster than almost any other retail vertical, so grain discipline matters more here, not less.

One "product" in apparel is a hierarchy: **style** → **colour** → **size** → **season**, plus fit, and often a pack or set. A single style becomes hundreds of SKUs. Every level of that hierarchy is a place where a join can fan out and a rollup can double.

Specific things to get right on this platform:

- **Name the level in every fact's grain sentence.** "Product" is not a grain. *Style*, *style-colour*, or *SKU* is. Half the arguments about a number are actually disagreements about which of those three it is at.

- **Style description tables are language-keyed and season-keyed.** The Middle East footprint means English and Arabic descriptions coexist, and the same style code can be re-issued in a later season. Both are extra key columns; both fan out if you join on the style code alone. Pin the language, pin the season, use LEFT.
- **Size is where the biggest multiplication lives.** A size-scale lookup joined without pinning the scale — or the size-range version — multiplies every sales row by the number of sizes in the scale. Devastating and completely plausible-looking.
- **Season overlaps are normal, not an anomaly.** Two seasons trade at once during transition. A fact that omits season from its grain will merge rows across seasons and appear to be working perfectly.
- **Distinct counts across style are not summable.** A basket containing a shirt, trousers and a belt is one transaction and three style rows. Take transaction and customer counts from a source at the right grain; never sum them across the style axis.
- **Returns and exchanges are a sign trap.** Confirm whether a return is a negative row or a separate row with a flag, and make sure your aggregate doesn't count both the original and its reversal as sales.
- **Cross-source overlap.** XStore, Magento and RMS can each describe the same sale. Decide which is authoritative for each measure and state it, or a "total sales" figure will double at the source level before any join gets a chance to.

L14 · Guarantee Correctness by Construction ★

Checklist

- Every table's grain is stated in one sentence in its header
- For every join, I can name the right side's full primary key from the code
- Every lookup join keyed by language / type / version pins those columns
- Restricted lookup joins are LEFT, not INNER, and the join type is declared explicitly
- Right-side projections drop any column that could re-introduce duplicates
- Every table with two grains says so in the header
- Every roll-up excludes synthetic rows, with the reason in a comment beside it

- Distinct-count measures come from a source at the rollup grain, never summed
- A uniqueness test exists on every merge key
- A reconciliation test exists on every total, in both directions
- Invariants deliberately **not** asserted are written down, with the reason

You've got it when you can...

...do two things on sight: point at any join and state the right-hand table's primary key from memory, and point at any `SUM()` over a fact table and say **which grain it is summing** — including whether that table contains its own subtotals.

Design the Schedule and the Dispatcher

One cron. One cycle id. One function that decides tonight's work from config and state.

1 · THE DECISION

Where is “what runs tonight” actually decided?

Eight sources, dozens of tables each. Something has to choose, and you should be able to read it.

× a cron rule per table

ONE CRON · ONE DISPATCHER

2 · DO THIS

ONE CRON · ONE DISPATCHER · ONE CYCLE ID

- 1 One schedule rule fires one dispatcher. The cron carries no instructions.
- 2 Stamp one cycle id per night. Every run row, lock and alert joins on it.
- 3 Write the cycle manifest first, create-only, before any job starts.
- 4 Build tonight's list from config plus state — never a hardcoded schedule.
- 5 Pass references, not rows. Orchestration inputs have a hard size ceiling.

DECLARE THE RUN MODE — NEVER INFER IT

INITIAL

complete history, once

INCREMENTAL

nightly, from watermark

RERUN

redo today's failures

BACKFILL

date range, own cycle

3 · WHY

CONFIG SCALES · CODE DOES NOT

- Config is reviewable; 200 cron rules are not.
- One decision function you can unit-test.
- State decides what is due, not the clock.
- Small payloads survive orchestration limits.

WHAT BREAKS IF YOU DON'T

Per-table schedules drift apart, and nothing can answer “what ran tonight, and why that list?”

4 · ON APPAREL GROUP

ON APPAREL GROUP

One nightly cycle covers all eight: RMS · SIM · XStore · Epsilon · MoEngage · Magento · Vemco · Irisys.
XStore is the giant — give it chunked initial windows; the two footfall feeds ride the same fan-out.

IN PLAIN ENGLISH

The alarm clock does not know your to-do list. It just rings. One person then reads the list and hands out the jobs.

L15 · Design the Schedule and the Dispatcher

Slide: `_render/L15-schedule-and-dispatch.html` → `L15-schedule-and-dispatch.png`

The decision

Where is *"what runs tonight"* actually decided?

You have two ways to answer it. You can put the answer in the schedule — a cron rule per table, each with its own time, its own target and its own idea of what "yesterday" means. Or you can put the answer in **one function** that wakes up once, reads the config and the current state, and works out the night's plan before a single job starts.

Choose the second. The cron becomes a doorbell: it carries no instructions at all.

Do this

- One schedule rule → one dispatcher.** A single rule fires once per day at a fixed wall-clock time. It passes an empty payload. Everything the run needs is derived, not carried. If your team has to open the scheduler to answer "why did this table run?", the design is wrong.
- Stamp one cycle id for the whole night.** Use the UTC date for a scheduled run (`<YYYY-MM-DD>`) and a prefixed id for a scoped one (`init-...` , `bf-...` , `rebaseline-...`). Every run row, every lock, every alert and every lineage edge carries it. One id is what turns thousands of scattered rows into one answerable question: *what happened last night?*
- Write the cycle manifest first — and make it create-only.** Before you start any work, write one record that says: this cycle exists, it started at *T*, and these are the units of work it expects. Use a conditional write

(`attribute_not_exists(pk)` or the equivalent) so a double-fire of the schedule cannot resurrect a finished cycle or reset the start clock. Write it *before* dispatch, or a fast source can reach the first hand-off before the manifest it needs is there.

- Build tonight's list from config plus state — never from a hardcoded schedule.** The dispatcher reads the source/table config (which tables are enabled, which source they belong to, what load strategy each uses) and the current watermark state (has this table completed its initial load yet?). It groups by source and produces one launch plan per source. Adding a table is a config row; it is never a change to the dispatcher.
- Make the expected list a promise, not a wish.** A table only enters `expected[]` if it was actually launched. A table that is enabled in config but has no runnable target must be reported as a failure now, not silently waited on forever by a downstream gate.
- Declare the run mode. Never infer it.** Four modes, chosen by the payload, all handled by the same function:

MODE	PAYLOAD SHAPE	WHAT IT MEANS
<code>initial</code>	<code>{"initial_load": true, "source": ...}</code>	complete history, once, before CDC starts
<code>incremental</code>	<code>{}</code> (the scheduled path)	tonight's delta, from the stored watermark
<code>rerun</code>	<code>{"rerun": true, "cycle_id": ...}</code>	redo today's failures inside the same cycle
<code>backfill</code>	<code>{"rerun": true, "date_from": ..., "date_to": ...}</code>	a dated re-pull in its own cycle, watermark untouched

Route on the payload in one place — a handful of `if` s at the top of the handler is the entire public API of your pipeline. Write it so a reviewer can read all four branches on one screen.

L15 · Design the Schedule and the Dispatcher

7. **Keep the orchestration payload small: pass references, not data.** Every orchestration engine has a hard input-size ceiling (Step Functions: 262,144 bytes). A large initial load walks straight into it. Two habits keep you clear of the wall: - Each work item carries *identifiers* — source, table, mode, date window — and nothing the downstream stage can look up for itself. Strip every field that belongs to a later phase. - Chunk a long initial load into calendar-aligned windows, and make the chunk size a **parameter of the invocation**, not a constant in the code. Wider chunks mean fewer job startups but bigger, riskier pulls; narrower chunks mean resumability but startup cost dominates. You want to be able to change your mind without a deploy.
8. **Do not window a table that has no usable date column.** If a table's only strategy is full-refresh, a date predicate matches everything — so each "window" re-reads the entire table. Give those tables one un-windowed chunk. A table opts *into* chunking only when its watermark is a genuine date.
9. **Make dispatch itself idempotent.** Derive the execution name from `<source>-<cycle_id>` so a duplicate fire collides and no-ops. Then handle the trap: many engines reserve execution names long after the execution has ended, so "name already exists" does **not** mean "still running". Read the prior execution's status; if it is terminally dead, re-dispatch under a uniquified name.

Worked example (Tamimi): `src/lambda/dispatcher/handler.py` — `_dispatch` is the launch plan, the mode routing is five `if` s near the top, `_month_windows` / `_chunk_windows` is twenty lines that define what a chunk is, and `_make_table_item` documents exactly which fields a work item is *not* allowed to carry. Read it as a shape to copy, not as history.

Why

- **Eight sources times dozens of tables is too many cron rules to keep correct by hand.** A schedule spread over hundreds of rules has no single place where its logic can be reviewed, tested, or explained.
- **A dispatcher is a pure-ish function, so you can unit-test the decision.** Feed it a config fixture and a watermark fixture; assert on the launch plan. You can never write that test against a scheduler.
- **State — not the clock — decides what is due.** A table that has not finished its initial load must not join the nightly delta cycle. Only something that reads state can enforce that.
- **Small payloads survive the orchestration layer.** Size limits apply to what you send *and* to what a parallel map aggregates on the way back; pass references in both directions and neither can bite you.

What breaks if you don't: per-table schedules drift apart until no two tables share a definition of "today", and nothing in the system can answer *"what ran tonight, and why that list?"*

L15 · Design the Schedule and the Dispatcher

On Apparel Group

- **One nightly cycle covers all eight sources** — Oracle Retail (RMS), Oracle SIM, Oracle XStore, Epsilon, MoEngage, Magento, Vemco Footfall, Irisys Footfall — with one dispatcher fanning out to one execution per source. Sources run in parallel; tables within a source are the dispatcher's concern, not the scheduler's.
- **XStore is the giant.** Its initial load is the one that will hit the payload ceiling first, so it is the one to chunk into calendar windows, and the one whose chunk size you will tune. Size it before you size anything else.
- **RMS masters are small and mostly full-refresh** — one un-windowed chunk each. Do not let them inherit XStore's chunking scheme just because they share a dispatcher.
- **Epsilon and MoEngage have their own cursors**, not date watermarks. Their work item carries a cursor reference; the connector resolves it. The dispatcher does not need to know the difference — that is what LO2's source contract bought you.
- **Vemco and Irisys are small file drops.** They ride the same fan-out as everything else. Resist the urge to give a small source a special path; two special paths become eight.
- **Pick the cycle-id timezone deliberately.** Gulf retail operates on a fixed offset with no DST, so a UTC-based cycle id maps to a stable local hour. Write that decision down once, in the config, and never recompute it per table.

Checklist

- ☐ Exactly one schedule rule per environment, and it passes an empty payload

- ☐ `cycle_id` is generated in one place and stamped on every downstream record
- ☐ The cycle manifest is written **before** the first job starts, with a create-only condition
- ☐ The launch plan is derived from config + watermark state; no table names in code
- ☐ `expected[]` contains only work that was actually launched
- ☐ All four run modes are named, routed in one function, and documented in the runbook
- ☐ Chunk size is an invocation parameter, not a constant
- ☐ Full-refresh-only tables are dispatched un-windowed
- ☐ Work items carry identifiers only — you have measured the serialized size of a worst-case plan
- ☐ Execution names are deterministic, and "name exists" is resolved by reading the prior status
- ☐ Dev's schedule rule is **disabled**: Dev is manual-only

You've got it when you can...

...take a payload you have never seen — say `{"initial_load": true, "source": "xstore", "chunk_months": 3}` — and predict, before running it: which cycle id it creates, roughly how many jobs it will produce, whether the orchestration input will fit inside the size ceiling, and which hand-off will decide when it is finished.

Make Phase Hand-offs Safe

Parallel sources finish together. Two callers will race the same hand-off — exactly one must act.

1 · THE DECISION

How does the next phase start — and start only once?

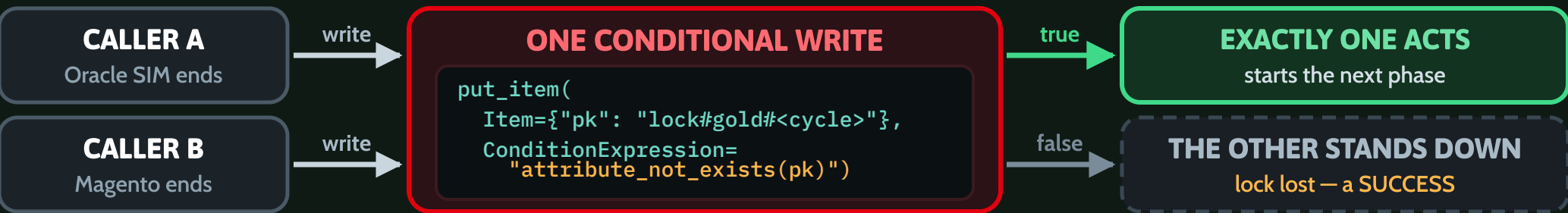
Every source ends at the same hand-off, and on a busy night they arrive together.

× poll, or “is anything running?”

ONE CONDITIONAL WRITE

2 · DO THIS

GATE EVERY PHASE WITH A SINGLE-FLIGHT LOCK



- 1 Hold before you claim — every “not ready yet” return sits above the lock.
- 2 Decide on **TERMINAL** states, never on “is it running”. A cycle advances.
- 3 Losing the lock is **SUCCESS**. Half a healthy night's calls end that way.

3 · WHY

LET THE STORE DO THE ORDERING

- One item, one writer: the database serialises it.
- No queue, no election, no lease to tune.
- A phase advances: running → built → concluded.
- The lock is what makes a re-check free and safe.

WHAT BREAKS IF YOU DON'T

Two builders publish the same day twice — or every hand-off after the first quietly stops firing.

4 · ON APPAREL GROUP

ON APPAREL GROUP

All eight reach the hand-off together, and RMS, SIM and XStore rarely finish in the same order twice.
One lock item per phase per cycle — landing, bronze, silver, gold. Identical code four times, on purpose.

IN PLAIN ENGLISH

Two people reach for one latch. It turns once. Whoever turns it does the work — the other walking away is the design working.

L16 · Make Phase Hand-offs Safe ★

Slide: `_render/L16-safe-phase-handoffs.html` → `L16-safe-phase-handoffs.png`

The decision

How does the next phase start — and start **only once**?

Your pipeline has phases: land the raw extract, build bronze, conform to silver, build gold. Each phase fans out across many sources and many tables, and each phase ends at the same question: *is the whole phase finished, and if so, who starts the next one?*

Every source's execution asks that question. On a busy night two of them ask it milliseconds apart, both look at the world, both see the same all-green picture, and both conclude "I should start the next phase now."

Exactly one of them must. Not because of a queue, not because of a leader election, not because of a lock service with a lease you have to tune — but because of **one conditional write to one item**.

Do this

- 1. **End every phase with a barrier, and make it the only thing that advances the cycle.** A barrier is a small function invoked at the tail of a phase. It reads the cycle manifest, decides whether the phase is genuinely complete, and — if it is — starts the next phase exactly once. Individual jobs never start the next phase. Nothing else writes the cycle state.
- 2. **Claim the right to act with a single-flight lock, implemented as a conditional write.**

```
ddb.put_item(  
  TableName=coordination_table,  
  Item={"pk": f"lock#{gold#{cycle_id}}", "claimed_by": request_id, ...},
```

```
  ConditionExpression="attribute_not_exists(pk)",  
)
```

The store serialises writes to a single item. Whichever request it orders first finds the item absent and succeeds; the second gets a conditional-check failure. One caller acts, the other does not. No timing assumptions, nothing to tune.

- 3. **One lock item per phase per cycle — identical code every time.** `lock#landing#<cycle>` , `lock#bronze#<cycle>` , `lock#silver#<cycle>` , `lock#gold#<cycle>` . Four locks, one helper, copied verbatim. Resist the urge to make each phase's claim "smarter"; the value of this pattern is that it is boring in four places.
- 4. **Treat losing the lock as SUCCESS.** The loser returns `{"status": "lock_lost"}` and does nothing at all. That is not an error path — it *is* the mechanism. In a healthy pipeline a large share of barrier invocations end this way. Make sure your metric names, log levels and alarm filters agree with that: a `lock_lost` must never page anyone.
- 5. **Hold before you claim.** Order matters, and this is the rule people get wrong. Every "not ready yet" check must return **above** the claim, mutating nothing. A claimed lock can never be re-claimed, so claiming one and *then* discovering you cannot proceed wedges the cycle permanently. Structure the barrier in exactly this order:

```
1. read the cycle manifest           → no manifest?      return "no_cycle"  
2. is the cycle already concluded?  → yes?           return "already_concluded"  
3. is every expected unit TERMINAL? → no?           return "waiting"  
4. is the phase genuinely COMPLETE (see 7)? → no?          return "waiting"  
5. claim the lock                   → lost?          return "lock_lost"  
6. — only now may you mutate anything —  
7. judge: any failure → mark the cycle skipped, alert, do not proceed  
8. otherwise start the next phase, then record the new cycle state
```

L16 · Make Phase Hand-offs Safe ★

6. **Decide on TERMINAL states, never on "is it running".** A cycle *advances* through states: `running` → `landing_built` → `bronze_built` → `silver_built` → `gold_built` . A barrier that no-ops unless the state equals `running` will stop firing the instant the first barrier advances the state — and every phase after that silently never runs.

Define the stop condition as a **set of concluded states**, not one string:

```
CONCLUDED_CYCLE_STATES = frozenset({
    "complete", "landing_skipped", "bronze_skipped",
    "silver_skipped", "gold_skipped", "gold_built",
})
```

Proceeding on an intermediate state is safe *precisely because* the lock makes it idempotent. The lock is what buys you the right to be relaxed about the state check.

7. **Resolve "is this unit finished?" as the latest attempt, and demand positive evidence.** One table produces several run rows per cycle — one per stage, plus one per retry. "The table's status" is only well defined as *its most recent attempt*, resolved by comparing `started_at` (ISO-8601 sorts lexically, so a string compare is a chronological one). Never rely on the store's natural query ordering.

Two refinements that matter as soon as you have more than one stage: - **Terminal is not the same as done.** If terminality can be satisfied by *any* stage's row, a table whose bronze step never wrote a row at all still resolves

to its successful landing row and reads as finished. Gate on **positive evidence for this phase**: a succeeded `bronze` row for every expected table. - **Latest is not the same as complete.** A chunked table is many runs sharing one name; collapsing it to its latest run passes a table whose other windows were dropped. Carry an expected-chunk count **on the manifest, counted off the items you actually dispatched**, and gate on `succeeded ≥ planned` .

8. **Release only when you caused nothing.** If the claim succeeded but the *start* then failed, delete the lock and re-raise so a retry or the sweeper can re-attempt. If the start **succeeded** and only the bookkeeping write failed, do **not** release — releasing would risk a second build.

9. **All-or-nothing is the verdict, not the gate.** Once you hold the lock: any failed unit in `expected[]` → set the cycle `*_skipped` , publish an alert, and do not start the next phase. A partial raw landing must never load into bronze; a partial bronze must never build gold.

10. **A barrier error must never fail a good ingestion.** Wrap the barrier task with a retry and a catch-to-succeed. The work already landed; a flaky tail call should not mark the whole source failed. The sweeper (L18) is what re-checks a barrier that died.

Worked examples (Tamimi): `src/shared/shared/control_plane/coordination.py` — `try_claim_gold_lock` is sixteen lines and its three siblings are deliberately identical; `CONCLUDED_CYCLE_STATES` is the terminal-set predicate; `latest_run_by_table` is the one-status-per-table resolver. `src/lambda/gold_barrier/handler.py` is the canonical barrier in the order listed above.

L16 · Make Phase Hand-offs Safe ★

Why

- **One item, one writer.** The database already serialises writes to a single item. You are borrowing a guarantee that is stronger than anything you would build, and it costs one request.
- **No queue, no leader election, no lease to tune.** There is no clock skew to reason about and no lease to expire at the wrong moment. The failure mode of a lock service — "the lease lapsed mid-work" — does not exist here.
- **A phase advances, so the gate must be a set.** The only durable way to express "do not proceed" is to enumerate the states from which nobody may proceed.
- **The lock is what makes re-checking free.** Because a second decider does nothing, you can safely nudge a barrier from a timer, from a retry, or from an operator's console. Everything in L18 depends on that.

What breaks if you don't: two builders publish the same day twice — or, with a naive `== running` gate, every hand-off after the first one quietly stops firing and the report simply never updates. Both failures are silent.

On Apparel Group

- **All eight sources reach each hand-off together.** RMS, SIM and XStore are Oracle pulls of wildly different sizes; Epsilon and MoEngage are API pulls with their own pacing; Magento, Vemco and Irisys are small and fast. They will not finish in the same order twice, and two of them *will* land in the same millisecond.
- **Four lock items per cycle, one per phase** — landing, bronze, silver, gold. Same helper, four call sites. Review them as one change, not four.
- **The small sources are the dangerous ones.** Vemco and Irisys finish in seconds and will usually be the callers racing the barrier while XStore is still running — so the "waiting" path is the one that executes most often.

Make sure it is cheap and mutates nothing.

- **Chunked XStore initial loads need the chunk count on the manifest.** Its windows finish out of order, and no single window can conclude "the load is done" — only the phase can, and only the barrier sees the whole phase.
- **Wire `lock_lost` out of alerting on day one.** With eight sources you will see it several times a night per phase. If it pages, the team will start ignoring the channel in week one.

Checklist

- ☐ Every phase ends at a barrier; no job starts the next phase directly
- ☐ The claim is a conditional write on a single item, per phase, per cycle
- ☐ `lock_lost` returns success, is logged at info, and is excluded from alarms
- ☐ Every "waiting" return sits **above** the claim and mutates nothing
- ☐ The stop condition is a set of concluded states, not `!= "running"`
- ☐ Per-unit status is resolved as the latest attempt by timestamp, not by query order
- ☐ The gate requires positive evidence for *this* phase, not any terminal row
- ☐ Chunked work carries a planned-count on the manifest, taken from what was dispatched
- ☐ Start-failure releases the lock; post-start bookkeeping failure does not
- ☐ Any failed unit → cycle marked skipped + alert; the next phase does not start
- ☐ The barrier task is retried and caught, so a barrier error cannot fail a good ingestion

L16 · Make Phase Hand-offs Safe ★

You've got it when you can...

...answer *"two jobs raced the hand-off — which one wins, and why is that safe?"* with "whichever one the store orders first on that single item; the other gets a conditional-check failure, returns `lock_lost` , and does

nothing — which is the design working" — and then name the two *other* checks (latest-status-per-unit and phase completeness) that must pass before anyone is allowed to reach for the lock at all.

Design the Control Plane

Four record types, four questions. Every operator answer should be a lookup, not a log grep.

1 · THE DECISION

What does the platform write down about itself?

Jobs are stateless and executions forget. If you did not record it, it did not happen.

× grep the logs afterwards



RECORD IT AS IT HAPPENS

2 · DO THIS

FOUR RECORD TYPES · ONE QUESTION EACH

RUNS	What happened?	one row per stage; index by cycle and by table
WATERMARKS	How far did we get?	one position per table; failures never advance it
PIPELINE STATE	Is it healthy now?	live health, freshness and alarms for the console
LINEAGE	What came from what?	deterministic edge id — re-runs never grow it

Rule: never infer success from an orchestrator's execution status — read the run rows.

3 · WHY

IF IT IS NOT RECORDED, IT IS GONE

- Executions expire; the record is the memory.
- A dashboard is a read over these four types.
- Gates and sweepers judge on rows, not logs.
- Every runbook step becomes a keyed lookup.

WHAT BREAKS IF YOU DON'T

Every incident becomes archaeology, and no one can build an operator view over data you never kept.

4 · ON APPAREL GROUP

ON APPAREL GROUP

An Oracle RMS table and a MoEngage API pull produce the same four record shapes — one console for all eight.
Stamp source, cycle and run mode on every row, and the Retail IQ operations view is already queryable.

IN PLAIN ENGLISH

A ship's log. Nobody reads it on a good day — but on a bad one it is the only thing that says where you were, and when.

L17 · Design the Control Plane

Slide: `_render/L17-design-the-control-plane.html` → `L17-design-the-control-plane.png`

The decision

What does the platform write down **about itself**?

Compute is stateless. An orchestrator forgets an execution's payload the moment it ends. Logs expire, and even while they live they are prose, not data. Nothing in a pipeline remembers anything by itself — so the memory is something you design, on purpose, before you need it.

The test to apply to every design choice here: *can an operator answer this question with a keyed lookup, instead of a log search?*

Do this

Build **four record types**. Each answers exactly one question. Learn them that way and nobody ever has to guess where to look.

RECORD TYPE	THE QUESTION IT ANSWERS	KEY
<code>runs</code>	<i>What happened?</i>	<code>run_id</code> + <code>stage</code>
<code>watermarks</code>	<i>How far did we get?</i>	<code>source_id</code> + <code>table_name</code>
<code>pipeline_state</code>	<i>Is it healthy right now?</i>	<code><kind>:<id></code>
<code>lineage_edges</code>	<i>What came from what?</i>	<code><upstream>\ <downstream>\ <edge_type></code>

1 · `runs` — what happened

- **One row per stage, not one row per table.** Give it a composite key of `run_id` + `stage` so a table's `bronze` and `silver` steps are separate rows sharing one `run_id`. Do not overwrite one row as the table advances — you lose the history exactly when you need it.
- **Index it two ways, and both indexes are sparse:**
 - `by_table` = (`table_name` , `started_at`) — one table's history, for monitoring and re-run lookups.
 - `by_cycle` = (`cycle_id` , `started_at`) — one whole cycle's runs. **Every barrier queries this one.**
- **Every row carries the same envelope:** `status` , `rows_in` / `rows_out` , `error_message` , `run_kind` (`scheduled` / `initial` / `cdc` / `rerun` / `backfill`), `started_at` , `ended_at` , and a TTL (90 days is a good default). `error_message` is the exact blocker for that unit — it is the first field anyone reads on a failure, so make sure the job writes something useful into it.
- **Assume the index is eventually consistent.** A barrier invoked microseconds after the last write can legitimately fail to see it. Put a short explicit wait before the final gate call, and let single-flight make the redundant call a harmless no-op.
- **Add a crash net.** A job that dies hard cannot write its own terminal row, and a missing row is indistinguishable from a running one. Have the orchestrator (or a small status function) guarantee a terminal row exists for every dispatched unit, so the table stays trustworthy.
- **Many-to-many stages need their own `run_id`.** A single warehouse build produces many models; no one `run_id` spans it. Give those rows a per-model `run_id` and correlate back through `cycle_id`. Their *lineage* belongs in `lineage_edges` , not here.

L17 · Design the Control Plane

2 · watermarks — how far did we get

- **One row per source table.** Read at job start, written at job end.
- **A failed run must never advance it.** That single invariant is the entire reason incremental loading is recoverable. Write the watermark on success only, in the same place, once.
- **Store the position in the shape the source uses** — a timestamp for a database, a cursor or token for an API, a key or filename for a file drop. If a source needs more than one position (a high-water date *and* a change-stream token), give it named fields rather than overloading one.
- **Carry an init-state machine on the same row:** `pending → initial_loaded → cdc`, plus a `needs_reinit` branch for "the change pointer expired, force a re-baseline". This is what enforces the rule that a table may not join the nightly delta cycle until its one-time initial load has completed.
- **Flip to `initial_loaded` from the barrier, not the job.** A chunked initial load finishes out of order, so no single chunk can conclude the load is done — only the phase can, and the barrier is the only actor that sees the whole phase.

3 · pipeline_state — is it healthy right now

- **One table, several widget kinds**, keyed polymorphically as `<kind>:<id>` where kind is `pipeline | alarm | freshness | reconciliation`. One API surface for the operator console; one place to add a widget.
- **Buy back the schema you gave up.** A polymorphic key weakens per-kind enforcement, so validate each kind against its own model with a discriminated union on `kind`. You get one table and four real schemas.

- **Enable a change stream with new-image only** if the console wants live updates; it does not need deltas.
- **Be careful with nullability in health fields.** "No comparable number" and "zero drift" are different answers. A non-optional `drift_pct` with a `>= 0` constraint forces a caller to clamp its *cannot-compute* sentinel to `0.0` — which renders a broken check as a perfect one. Make the field optional and let null mean null.

4 · lineage_edges — what came from what

- **Make the edge id deterministic:** `f"{upstream}|{downstream}|{edge_type}"`. Re-emitting the same edge overwrites the same item, so repeated runs never grow the graph.
- **Enumerate `edge_type`** — `source_to_bronze | bronze_to_silver | silver_to_gold | gold_to_view` — so impact analysis can walk one layer at a time.
- **Store a `transform_id`**: the model name, job ARN or function name that drew the edge. That is the handle an engineer uses to navigate from a lineage edge back to the *code*.
- **Write it at completion, from the transform itself.** Lineage inferred later is lineage you cannot trust.

5 · Two rules that make the whole thing usable

- **Never infer success from an orchestrator's execution status.** A fan-out execution can report success over children that all failed. Query the run rows.
- **Design the console query at the same time as the record.** If you cannot write, on a whiteboard, the exact keyed read that powers the widget, you have not finished designing the record.

L17 · Design the Control Plane

Worked examples (Tamimi): `src/shared/shared/control_plane/` — `runs.py` (composite key + both indexes), `watermarks.py` (`InitState` and the multi-position fields), `pipeline_state.py` (the discriminated union and the nullability lesson), `lineage_edges.py` (`_compute_edge_id`). The tables themselves are one Terraform module, `infra/modules/control_plane/main.tf` .

Why

- **Executions expire; the record is the memory.** Everything that survives the night is something you chose to write.
- **A dashboard is nothing more than a read over these four types.** Design the records and the operator view is almost free; skip them and no dashboard is possible at any price.
- **Gates and sweepers judge on rows, not logs.** L16's barriers and L18's sweeper both read `runs` by cycle. The control plane is not documentation — it is load-bearing.
- **Every runbook step becomes a keyed lookup.** "Did it run?" "How far did it get?" "Is it alerting?" "Where did this column come from?" — four questions, four tables, no searching.

What breaks if you don't: every incident becomes archaeology, and nobody can build an operator view over data you never kept. You will discover this on the first bad night, which is the worst possible time to start writing records.

On Apparel Group

- **One control plane spans all eight sources.** An Oracle RMS table and a MoEngage API pull produce the same four record shapes, so one console covers RMS, SIM, XStore, Epsilon, MoEngage, Magento, Vemco and Irisys without per-source special cases.
- **Stamp `source_id` , `cycle_id` and `run_kind` on every row.** Those three fields are what make "show me last night, by source" a query instead of a project — and they are the fields the Retail IQ / Quick operations view will group by.
- **Model the source-specific position properly, once.** Oracle tables get a date/SCN watermark; Epsilon and MoEngage get a cursor or token plus a freshness timestamp so you can alarm on token age; footfall file drops get the last processed key. Same table, named fields, no overloading.
- **Epsilon is PII.** The control plane records *metadata* — counts, timings, statuses — never sample rows or key values from a customer table. Decide that rule before the first run row is written, and encrypt these tables with the same class of key you use for audit data.
- **Freshness SLA per table belongs here,** not in a spreadsheet: it is what L23's lateness alarm reads and what the console's red/amber/green is computed from.
- **Retention:** 90-day TTL on run rows keeps the table cheap and still covers a full quarter-end investigation. Lineage edges have no TTL — they are the current shape of the graph, not history.

L17 · Design the Control Plane

Checklist

- ❑ Four record types exist, and each one has a written down "the question it answers"
- ❑ `runs` is keyed per stage, with a by-cycle and a by-table index
- ❑ Every run row carries status, row counts, `error_message` , `run_kind` and a TTL
- ❑ A crash net guarantees a terminal row for every dispatched unit
- ❑ Watermarks advance on success only — verified by a deliberate failure test
- ❑ An init-state machine gates entry into the nightly delta cycle
- ❑ `pipeline_state` kinds validate against their own models

- ❑ Health fields distinguish "unknown" from "zero"
- ❑ Lineage edge ids are deterministic, and edges are written by the transform at completion
- ❑ Every widget on the intended console maps to one keyed read you have written down
- ❑ The runbook says "query the run rows", never "check the execution status"

You've got it when you can...

...be handed a question — *"did it run?", "how far did it get?", "is it alerting?", "where did this column come from?"* — and name the record type, the key and the index you would query, without looking anything up.

Design for Failure Before You Ship

At-least-once delivery means a job will run twice. Build for the second run before the first ships.

1 · THE DECISION

Is re-running the normal move, or the desperate one?

Retries, duplicate triggers and overlapping windows are ordinary events. Design for them.

× hope it never runs twice

MAKE IT SAFE TO RUN TWICE

2 · DO THIS

IDEMPOTENT STAGES · A SWEEPER · NAMED RECOVERY MODES

- 1 Make every write repeatable: overwrite a snapshot, or merge on a real key.
- 2 Claim work with a create-only write, so a duplicate trigger simply no-ops.
- 3 Look for an in-flight job before you start one — start APIs rarely dedupe.
- 4 Add a sweeper that re-checks stuck cycles on a timer, not on an event.

NAME THE RECOVERY MODE — AND WHO MAY RUN IT

RERUN

today's failed tables
on-call engineer

BACKFILL

old data restated
owner signs off

RE-BASELINE

change pointer lost
platform team

AUTO SWEEP

stuck, nobody asked
every 15 minutes

3 · WHY

THE SECOND RUN IS NOT RARE

- At-least-once is the normal delivery contract.
- A timeout hides a side effect you already caused.
- An event-triggered gate that dies is never retried.
- Age alone is not evidence a cycle is dead.

WHAT BREAKS IF YOU DON'T

One duplicate trigger doubles a day's numbers, and a wedged cycle blocks every night that follows it.

4 · ON APPAREL GROUP

ON APPAREL GROUP

Eight sources, eight ways to fail: Oracle sessions drop, Epsilon and MoEngage throttle, footfall files land late.
Write the recovery table and name an owner per mode before go-live — the on-call rota starts on day one.

IN PLAIN ENGLISH

Assume the delivery van comes twice. Design the doorstep so the second box is harmless — not a second charge on the card.

L18 · Design for Failure Before You Ship

Slide: `_render/L18-design-for-failure.html` → `_L18-design-for-failure.png`

The decision

Is re-running the **normal** move, or the desperate one?

You get to choose. If every write in the pipeline is safe to repeat, a re-run is a shrug: it is either identical work or a no-op. If they are not, every recovery becomes a negotiation about what might get double-counted, and the team stops touching the pipeline at exactly the moment it needs touching.

Design for the second run **before** the first one ships. At-least-once delivery is the normal contract of every queue, every retry policy and every timeout you will use — so a job *will* run twice. Plan the doorstep, not the apology.

Do this

1 · Make every stage idempotent, at five independent layers

Belt and braces on purpose — no single mechanism is enough, and you want the design to survive any one of them being wrong.

- 1. **Deterministic dispatch names.** `<source>-<cycle_id>` collides on a duplicate fire and no-ops. Handle the trap: engines commonly reserve execution names long after the run ended, so "already exists" is not proof of "still running". Read the prior status; re-dispatch a terminally dead one under a uniquified name.
- 2. **A create-only manifest.** `put_cycle` writes with `attribute_not_exists(pk)` , so a re-fire cannot overwrite a concluded cycle or reset `started_at` (which would restart the sweeper's clock).
- 3. **Single-flight locks.** One conditional write per phase; the loser does nothing (L16).

- 4. **Repeatable writes.** Landing overwrites its per-cycle snapshot; bronze/silver `MERGE` on a real natural key, or overwrite whole partitions. Re-running a table never duplicates rows. (This is LO7's rule, and this is why it matters.)
- 5. **Resolution by latest attempt.** A gate judges each unit by its most recent row, so a fresh success supersedes an old failure without anyone deleting anything.

2 · Look before you leap: check for an in-flight job

Most job-start APIs have **no client idempotency token**. That makes this sequence possible:

```
03:12:04  StartJob → the service ACCEPTS the run, r-8821 starts
03:12:34  the API response never comes back (socket timeout)
03:12:35  the task retries the barrier → a fresh lock claim succeeds
          → a SECOND build for the same day
```

The lock does **not** save you here. Locks protect against *concurrent deciders*; they do not protect against *an ambiguous side effect you already caused*.

So before starting, query recent job runs and look for one whose cycle argument matches this cycle and whose state is `STARTING` / `RUNNING` / `WAITING` / `SUCCEEDED` . If you find one, treat the day as already built, reconcile the cycle state, and return `already_running` .

Fail open. Any error inside the pre-check — a throttle, a missing permission — returns "unknown" and the caller proceeds. This guard is an optimisation over correctness, not the correctness mechanism itself, so it must never be worse than not having it.

3 · Add a sweeper — the level-triggered net

Barriers are **edge-triggered**: invoked once, at the end of a phase. If a barrier dies, nothing re-evaluates it and the cycle sits forever. So add a sweeper on a timer (15 minutes is a good default) doing two different jobs:

L18 · Design for Failure Before You Ship

- **RESUME — every pass, no deadline.** For any non-concluded cycle, re-invoke the barrier its state implies: `running` → landing barrier, `landing_built` → bronze barrier, and so on. Barriers are self-gating and single-flight, so a nudge costs nothing unless the gate is genuinely satisfiable. This is the whole payoff of L16's lock.
- **CONCLUDE — only when overdue *and* no longer moving.** Force-fail every expected unit that is still non-terminal (writing a synthetic failed row for a unit with no row at all), then re-invoke the final barrier so it claims the lock and concludes the cycle with an alert.

Three clocks, not one, and each has a distinct job:

CLOCK	DEFAULT	WHAT IT MEANS
deadline	12 h	<i>opens the question</i> — this cycle is overdue
stall	3 h with no new run row	<i>answers it</i> — nothing is progressing
ceiling	96 h	absolute backstop, so "never hangs forever" still holds

Age alone is not evidence of death. A long initial load is slow, not dead. A sweeper that concludes on age alone will force-fail healthy work — and the cost of that is a whole night's re-run.

Two subtleties worth building in from the start:

- **The sweeper must ignore rows it wrote itself.** A force-fail pass stamps timestamps on every non-terminal unit; counting those lets the sweeper read its own writes back as proof the cycle is alive and refuse to conclude for another stall window.
- **Exclude earlier-phase rows from the force-fail pass.** A successful *landing* row means the raw extract arrived, not that the table reached bronze. Counting it makes a table with no bronze row look terminal, so it is never force-failed, the completeness gate has nothing to break it, and the cycle hangs instead of concluding.

4 · Isolate per item, but gate all-or-nothing

Two rules that look contradictory and are not:

- **Per-item isolation.** In the fan-out, a failed table catches to a *succeed* state, so one bad table never stops the other fifty from processing. Its run row is already failed — the job self-reports before raising.
- **All-or-nothing at the gate.** Any failed unit in `expected[]` → the next phase does not start. A partial raw landing must not load into bronze; a partial bronze must not build gold.

Isolation maximises how much useful work one cycle completes. All-or-nothing stops a half-complete picture reaching a report. The run rows are what reconcile the two.

L18 · Design for Failure Before You Ship

5 · Define the recovery modes — and who may run each

MODE	USE WHEN	WHAT IT DOES	WHO RUNS IT
rerun	a table failed in today's cycle	reopens the cycle (<code>state</code> → <code>running</code>), releases the downstream locks, restarts the source under a unique name. Empty <code>tables</code> = re-run what failed; name a table to force a green one to re-run	on-call engineer, self-serve
rerun (landing only)	the raw pull failed	re-pulls raw only; never starts the next phase — the barrier does that once the gate re-passes. Must also re-point any planned-chunk count, because a re-pull dispatches one un-windowed item per table and the old multi-window plan would be permanently unsatisfiable	on-call engineer
backfill	old data was restated at source	its own scoped <code>bf-...</code> cycle, a bounded <code>BETWEEN</code> read, and the incremental watermark is untouched so tomorrow's normal run is unaffected. Force a full refresh downstream, or a window older than the incremental predicate lands in silver and never surfaces	data owner signs off
re-baseline	the change-stream pointer expired	a bounded re-pull over the CDC gap in its own <code>rebaseline-...</code> cycle; on success <code>init_state</code> → <code>initial_loaded</code> so the next run can reseed a fresh token. The dispatcher should queue this automatically when it sees <code>needs_reinit</code>	platform team
auto sweep	a cycle is stuck and nobody asked	resume, or conclude with an alert	the system, every 15 min

Two rules people get wrong: - **Derived / conformed tables are never named in `tables`** — they have no source pipeline. Name their *inputs*, and the conform phase rebuilds them. - **Auto-detect never re-runs a green table.** An empty `tables` means "re-run the failures". To redo something that succeeded you must name it.

6 · Write the recovery table before go-live

Put it in the runbook with copy-paste payloads, and put a name next to each mode. A recovery procedure discovered at 3 AM is a recovery procedure that will be got wrong.

Worked examples (Tamimi): `src/lambda/cycle_sweeper/handler.py` — its docstring is the design rationale for edge- vs level-triggered, then `_last_activity` (don't read your own writes), `_should_conclude` (the three clocks) and the force-fail pass. `src/lambda/gold_barrier/handler.py` — the in-flight pre-check, its fail-open contract, and release-on-start-failure vs never-release-after-success. `docs/OPERATIONS-RUNBOOK.md` §4 is the operator-facing recovery table.

Why

- **At-least-once is the normal delivery contract.** Retries, redeliveries and duplicate triggers are not exotic — they are the guarantee you actually bought.
- **A timeout hides a side effect you already caused.** The absence of a response is not the absence of an effect, which is why locks alone are not enough and the in-flight check exists.
- **An event-triggered gate that dies is never retried.** Edge-triggered logic needs a level-triggered counterpart, or one dead invocation costs you a night.
- **Age alone is not evidence a cycle is dead.** Slow and stuck look identical from a distance; only *activity* tells them apart.

L18 · Design for Failure Before You Ship

What breaks if you don't: one duplicate trigger doubles a day's numbers, and a single wedged cycle blocks every night that follows it — with the report showing yesterday's figures the whole time.

On Apparel Group

- **Eight sources, eight ways to fail.** Oracle sessions drop mid-read on RMS, SIM and XStore; Epsilon and MoEngage throttle and return 429s; Magento's API pages out; the Vemco and Irisys footfall files simply arrive late. Every one of those is a *normal* night, and every one is fixed by a re-run — provided you built for it.
- **XStore's initial load is where the sweeper deadline will bite first.** It runs for many hours legitimately. Tune the stall window against XStore, not against RMS, or the sweeper will force-fail your largest and most expensive load.
- **The footfall feeds are the "arrived late" case, not the "failed" case.** Decide up front whether a missing optional source blocks the cycle. If it does not, it must not be in `expected[]` — a unit that can never run makes the gate wait forever.
- **Name an owner per mode before go-live.** `rerun` is self-serve for on-call. `backfill` needs a business owner's sign-off because it changes history. `re-baseline` is platform-team only. Put these three names in the runbook on day one; the on-call rota starts with the first production cycle, not after it.
- **Test the recovery path in Dev deliberately.** Point a spec at a nonexistent column, watch the run row go failed, the cycle conclude skipped and the alert arrive; then fix it, `rerun`, and watch the cycle reach built. Doing that once makes the whole lesson obvious — and it is the single best onboarding exercise for a new engineer on the platform.

Checklist

- ☐ Every write in the pipeline is repeatable — snapshot overwrite or merge on a real key
- ☐ Dispatch names are deterministic, and "name exists" is resolved by reading the prior status
- ☐ The cycle manifest is create-only
- ☐ An in-flight pre-check runs before any job start, and it fails open
- ☐ A sweeper runs on a timer and can both resume and conclude
- ☐ The sweeper has three clocks (deadline / stall / ceiling), not one
- ☐ The sweeper ignores rows it wrote itself, and excludes earlier-phase rows from force-fail
- ☐ Per-item failures are isolated in the fan-out; the phase gate is all-or-nothing
- ☐ All recovery modes are named, documented with copy-paste payloads, and have an owner
- ☐ Backfill leaves the incremental watermark untouched and forces a downstream full refresh
- ☐ A deliberate failure and recovery has been rehearsed in Dev by someone who will be on call

You've got it when you can...

...be handed a red cycle at 3 AM and, without guessing: name the failed stage and its `error_message` from the run rows, say whether the cycle will conclude by itself or needs the sweeper, choose the right recovery mode and explain in one sentence why the other three are wrong — and state which of the five idempotency guarantees means your re-run cannot make things worse.

Structure Terraform for Three Environments

One shelf of environment-blind modules, three thin compositions — and state that cannot reach across.

1 · THE DECISION

Copy the stack three times — or build one shelf of modules and vary only values?

Then answer the second question, which sets your blast radius: what does Dev still share with Prod?

2 · DO THIS

ONE SHELF · THREE THIN COMPOSITIONS

```
infra/modules/<name>/ → main.tf · variables.tf · outputs.tf · versions.tf
env-blind — no account id, no CIDR, no environment name, ever

infra/env/{dev,qa,prod}/ → main.tf · backend.tf · providers.tf · terraform.tfvars
composition only — module calls plus one file of values
```

- Pin the provider in every module and in every env root.
- Isolate state per environment — its own bucket, its own lock table, its own key.
- prevent_destroy on every stateful resource: buckets, keys, the warehouse namespace.
- Put the environment in every resource name, and pin the account id on the provider.

```
aws = "~> 6.28" · allowed_account_ids · lifecycle { prevent_destroy = true }
```

3 · WHY

BLAST RADIUS

One fix in a module lands in all three envs.
Shared state hands Dev delete rights on Prod.

WHAT BREAKS IF YOU DON'T
A dev mistake can destroy production.

4 · ON APPAREL GROUP

AG · DEV · QA · PROD

One module shelf serves all 8 sources.
Oracle CIDRs, Epsilon endpoints and
MoEngage keys are values, never code.

```
infra/modules/ · infra/env/{dev,qa,prod}/
```

IN PLAIN ENGLISH

Modules are the parts bin; each environment is its own order sheet — and each sheet locks in its own cabinet.

L19 · Structure Terraform for Three Environments

Slide: [_render/L19-terraform-for-three-envs.html](#)

The decision

You need Dev, QA and Prod. You can copy the whole stack three times, or you can build **one shelf of reusable modules** and write three thin compositions that call them with different values.

Choose the shelf. Then answer the harder question, the one that sets your blast radius: **what does Dev still share with Prod?** Sharing modules is the point. Sharing *state* is the danger.

Do this

- 1. Split the tree in two, and keep the halves honest.

```
infra/
  modules/<name>/    main.tf · variables.tf · outputs.tf · versions.tf
  env/dev/           main.tf · backend.tf · providers.tf · terraform.tfvars
  env/qa/
  env/prod/
```

A module takes inputs and knows nothing else. If a file under `modules/` contains an environment name, an account id, a live subnet id or a CIDR, it has stopped being a module. Make that a review rule and grep for it:

```
grep -rniE '"(dev|qa|prod)"|[0-9]{12}|10\.[0-9]+\.[0-9]+\.[0-9]+' infra/modules/
```

- 2. Let the env root wire modules to each other, not to AWS. Pass `module.kms.key_arns["bronze"]` into `module.iam` ; pass `module.vpc.data_subnet_ids` into the warehouse. Terraform derives the apply order from those references, so nobody maintains a dependency graph by hand.
- 3. Pin the provider in every module *and* every env root, not just at the top:

```
terraform {
  required_version = ">= 1.9"
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.28" } }
}
```

Unpinned, a provider release between your plan and a colleague's plan changes what "no changes" means.

- 4. Isolate state per environment — bucket, lock table and key. Three keys in one bucket is not isolation; it means a Dev apply holds `s3:DeleteObject` on the Prod state object.

```
# infra/env/prod/backend.tf
terraform {
  backend "s3" {
    bucket      = "ag-lakehouse-tfstate-prod-<prod-account-id>"
    key         = "prod/terraform.tfstate"
    dynamodb_table = "ag-lakehouse-tflock-prod"
    region      = "me-central-1"
    encrypt     = true
  }
}
```

Verify the values, not the comment. A header block describing per-environment isolation proves nothing; read `bucket` and `dynamodb_table` on all three and confirm the three strings differ.

- 5. prevent_destroy on everything stateful — data buckets, KMS keys, the warehouse namespace, the state bucket itself:

```
lifecycle { prevent_destroy = true }
```

Terraform requires a literal here, so it cannot be driven by a variable. Read the resource, not a variable whose name sounds protective.

L19 · Structure Terraform for Three Environments

- 6. Put the environment in every resource name — `ag-lakehouse-raw-prod-<account>` — so a mis-targeted console action is obvious at a glance, and so a copy-paste between envs collides instead of silently succeeding.
- 7. Two cheap safeties worth adding on day one. - `allowed_account_ids` on the provider, so a plan run with the wrong identity fails before it touches anything. - `-lock-timeout=30m` on every plan and apply, so a second pipeline run queues behind the first instead of dying on the lock.

Why

Modules exist so one fix lands in all three environments. That is the benefit, and it is real.

State isolation exists so a *mistake* does **not** land in all three. Those are opposite requirements, and the layout has to serve both: share the code, never share the state.

What breaks if you don't: a dev mistake can destroy production.

On Apparel Group

- One `modules/` shelf covers all 8 sources. The Oracle sources (RMS, SIM, XStore) reuse the same VPC, KMS, IAM and Glue-job modules as the SaaS sources (Epsilon, MoEngage, Magento) and the footfall feeds (Vemco, Irisys).
- Everything that differs is a **value in** `terraform.tfvars` : the Oracle source CIDRs, the Epsilon and MoEngage endpoints, the per-environment VPC range, the secret names.
- Three environments × three Oracle sources means nine JDBC connection definitions generated from one module — not nine hand-written blocks.

- Name the state buckets before you write a single resource. It is a ten-minute decision on day one and a migration project in week ten.

Worked example of the pattern: the Tamimi repo's `infra/modules/` (29 modules, each with all four files) beside `infra/env/{dev,qa,prod}/` , where the only per-environment difference is the tfvars.

Checklist

- ☐ `infra/modules/` contains no environment name, account id or CIDR
- ☐ Every module and every env root has a `versions.tf` with a pinned provider
- ☐ The three `backend.tf` files name three different buckets **and** three different lock tables
- ☐ `prevent_destroy = true` on every bucket, key and warehouse namespace
- ☐ Every resource name carries the environment
- ☐ `allowed_account_ids` set on each env's provider
- ☐ `-lock-timeout` set on every backend-touching command

You've got it when you can...

- Draw the two-layer picture — env-blind modules on a shelf, three thin compositions calling them — without looking.
- Say which single value you would change to give QA a different VPC range, and name the one file it lives in.
- Open any `backend.tf` and state, from the values alone, whether that environment's state is genuinely isolated.
- Explain in one sentence why `prevent_destroy` cannot be a variable, and what that implies for how you review it.

Set Up CI/CD With Real Gates

No stored keys, one approved plan, and a human standing between the merge and production.

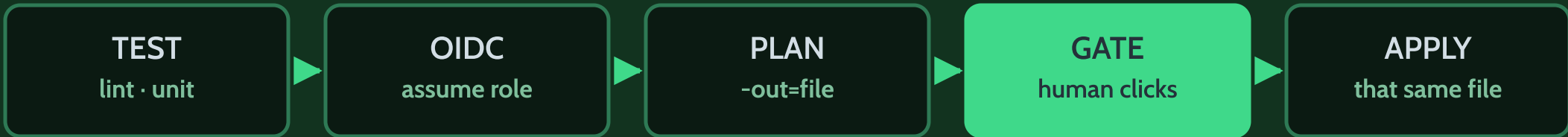
1 · THE DECISION

How does a merge become infrastructure — and who is allowed to stop it?

The pipeline needs AWS credentials it must never store, and an approval that has to mean something.

2 · DO THIS

TEST → ASSUME → PLAN → GATE → APPLY THAT PLAN



- Federate with OIDC — the runner presents a signed token, never a stored key.
- Save the plan to a file, then apply that file. Never re-plan inside the apply.
- Manual gate on QA and Prod. One deploy role per environment, never shared.
- Pin and checksum every tool, driver and layer the runner downloads.
- Match the branch in the OIDC trust policy to the branch you deploy from.

```
plan -out=prod.tfplan → show prod.tfplan → apply prod.tfplan
```

3 · WHY

AN APPROVAL THAT MEANS IT

A gate approves one specific plan.
Re-plan after the click and you ship another.

WHAT BREAKS IF YOU DON'T
The human approved a plan that never ran.

4 · ON APPAREL GROUP

AG · THREE ENVS, THREE ROLES

Dev auto-applies. QA and Prod wait for a person. The Oracle JDBC driver and every SaaS SDK are pinned and checksummed.

one deploy role per env · no cross-env reuse

IN PLAIN ENGLISH

You approve one exact plan, and that exact plan runs. The runner never keeps a key — it shows ID at the door, every time.

L20 · Set Up CI/CD With Real Gates

Slide: `_render/L20-cicd-with-real-gates.html`

The decision

A merge has to become infrastructure. Two things must be true along the way, and both are choices you make in week one:

- 1. The pipeline needs AWS credentials it can never store.
- 2. Someone must be able to stop it — and their approval has to mean something.

A gate that approves one plan and then applies a different one is decoration. Build the gate so that cannot happen.

Do this

- 1. **Federate with OIDC. Never issue the pipeline a static access key.** The CI system mints a short-lived signed token per step; AWS trades it for a one-hour role session. No secret is stored anywhere, and there is nothing to rotate or leak.

```
# the whole exchange — four lines, no credentials
- export AWS_REGION=me-central-1
- export AWS_ROLE_ARN=$AWS_ROLE_PROD_ARN
- echo "$CI_STEP_OIDC_TOKEN" > /tmp/web-identity-token
- export AWS_WEB_IDENTITY_TOKEN_FILE=/tmp/web-identity-token
```

- 2. **Write the trust policy as the real gatekeeper, and keep it in sync with the pipeline.** The trust policy decides which branch may assume which role — the pipeline YAML has no say. Condition on the audience *and* the subject claim, and cap the session:

```
condition {
  test = "StringEquals"; variable = "<provider>:aud"; values = [var.workspace_id]
}
```

```
condition {
  test = "StringLike";   variable = "<provider>:sub"; values = [
    "${var.repo_uuid}:${var.branch_pattern}:*"
  ]
}
max_session_duration = 3600
```

Make `branch_pattern` match the branch the pipeline actually deploys from. Put the two lists side by side — one per environment — and check them. A mismatch shows up as `AccessDenied` on `assume-role`, which reads like a permissions bug and is not one.

- 3. **One deploy role per environment.** `...-deploy-dev` , `...-deploy-qa` , `...-deploy-prod` . Never one shared role with three sets of permissions: the whole point is that a Dev pipeline cannot reach QA or Prod even if someone edits the YAML.

- 4. Plan to a file, then apply that file.

```
terraform plan -out=/tmp/prod.tfplan -lock-timeout=30m
terraform show /tmp/prod.tfplan           # this is what the human reads
# — manual gate —
terraform apply /tmp/prod.tfplan -lock-timeout=30m
```

Generate the plan **inside** the gated step and apply that same artifact. A saved plan fails closed if state moved since it was written — which is exactly the protection you want. `apply -auto-approve` computes a *fresh* plan and rubber-stamps it.

- 5. **Put a manual gate on QA and Prod.** Dev may auto-apply; it is the environment whose job is to be broken. Everything above Dev waits for a person to click.

- 6. **Deploy in the order the dependencies require.** Artifacts before the apply that references them:

```
targeted apply (KMS + S3)  → upload wheels / scripts / specs / zips
                             → full plan → gate → full apply
                             → seed the control plane
```

L20 · Set Up CI/CD With Real Gates

7. **Pin and checksum everything the runner downloads.** The runner holds cloud credentials, so nothing unversioned may land on it: exact versions for the CLI and build tools, exact versions for Lambda-layer dependencies, and a SHA-256 comparison on every JDBC driver jar before it is uploaded.

```
computed=$(sha256sum ojdbc11.jar | cut -d' ' -f1)
test "$computed" = "$OJDBC_SHA256" || { echo "driver checksum mismatch"; exit 1; }
```

8. **Fail closed on missing configuration.** Write required pipeline variables as `${VAR:?message}` so the step dies rather than silently downloading or deploying something unverified.

Why

OIDC removes the thing you would otherwise have to protect forever: there is no long-lived key, so there is nothing to steal, rotate or accidentally print into a log.

The saved plan is what makes approval real. Approval is not "I agree with this change in principle" — it is "I agree with *these* resource diffs".

What breaks if you don't: the human approved a plan that never ran.

On Apparel Group

- Three environments, three deploy roles, three branch patterns. Write the branch↔role table on the wall in week one and check it the first time a deploy fails to see AWS.
- **Dev auto-applies; QA and Prod wait for a person.** With four workstreams (Data Foundation, Price Optimization, Inter-store Transfer, Amazon Quick) pushing in parallel over 12 weeks, the gate is the only

thing keeping an in-progress model out of a client demo.

- The **Oracle JDBC driver** is a third-party binary that ends up on a credentialed runner and then in every Glue job. Pin the version and verify the checksum before upload.
- The Epsilon and MoEngage SDKs go in a Lambda layer — pin them exactly, and build them for the CPU architecture the functions actually run on.
- Concurrent pushes from four workstreams will collide on the state lock. Set `-lock-timeout=30m` so the second run queues instead of failing.

Worked example of the pattern: the Tamimi `bitbucket-pipelines.yml` — OIDC exchange, targeted bootstrap apply, artifact upload, gated `plan -out → apply <file>` , then the control-plane seed.

Checklist

- ☐ No `AWS_ACCESS_KEY_ID` anywhere in the pipeline definition or its variables
- ☐ Trust policy conditions on both `aud` and `sub` ; session capped at 1 hour
- ☐ Branch patterns in the trust policy match the pipeline's deploy branches, environment by environment
- ☐ One deploy role per environment, no sharing
- ☐ `plan -out=<file>` generated inside the gated step; `apply <file>` uses that artifact
- ☐ Manual gate on QA and Prod
- ☐ Every downloaded tool, driver and dependency pinned; drivers checksummed
- ☐ `-lock-timeout` set on every plan/apply

L20 · Set Up CI/CD With Real Gates

You've got it when you can...

- Explain, without notes, that CI never holds an AWS key — it presents a per-step signed token and receives a one-hour session for exactly one environment.
- Name the three things standing between a merge and production: **a human on the gate, a saved plan that fails closed, and a checksum on every binary the runner fetches.**
- Say what happens if the trust policy's branch pattern and the pipeline's deploy branch disagree — and which of the two you check first.
- Describe the failure mode of `apply -auto-approve` after a manual gate in one sentence.

Set Up Security and Governance

One role per workload, one key per layer, and a condition on every service principal.

1 · THE DECISION

Who can read the data — and how narrow can you make that on day one?

Broad grants are easy to add and almost impossible to take back once a dozen jobs depend on them.

2 · DO THIS

SIX RULES, SET ON DAY ONE

Least privilege, per workload

One role per job — no full-access managed policy.

Condition every service principal

aws:SourceAccount = this account, on every key.

One KMS key per data layer

raw · bronze · silver · gold · secrets · audit

Tag-based access for data

Grant on sensitivity=pii, not on table names.

Secrets by reference only

Code holds the ARN; the value never enters state.

Permissions boundary

On any role that is allowed to create roles.

modules/iam · modules/kms · modules/lake-formation · modules/secrets

3 · WHY

EVERY STAR DELIBERATE

Least privilege is not zero wildcards — it is every wildcard deliberate, and commented.

WHAT BREAKS IF YOU DON'T

A job role becomes an account-wide key.

4 · ON APPAREL GROUP

AG · EPSILON IS PII

Epsilon loyalty and MoEngage profiles are personal data. Classify in the spec, mask in Silver — before any report can see it.

```
pii_class: pii · grant on sensitivity=pii
```

IN PLAIN ENGLISH

Give every job the smallest key ring that still opens its own doors — and put a name on the one door you had to leave open.

L21 · Set Up Security and Governance

Slide: `_render/L21-security-and-governance.html`

The decision

Who can read the data — and how narrow can you make that on day one?

Narrow is cheap now and expensive later. A broad grant takes one line to add; taking it back once a dozen jobs quietly depend on it takes a week and a change-freeze. Decide the shape before the first job runs.

Do this

- 1. **One role per workload, scoped to that workload's resources.** A job role gets the buckets it reads, the buckets it writes, the catalog objects it owns, and nothing else.

Never attach a full-access managed policy to a job role. `AmazonS3FullAccess` on an ETL role is an account-wide data-plane grant: that job can read or delete the Terraform state bucket and the audit-log bucket too.

```
resources = [
  "arn:aws:s3:::${local.prefix}-landing-${var.account_id}",
  "arn:aws:s3:::${local.prefix}-landing-${var.account_id}/*",
  "arn:aws:s3:::${local.prefix}-raw-${var.account_id}",
  "arn:aws:s3:::${local.prefix}-raw-${var.account_id}/*",
]
```

- 2. **Condition every service-principal grant in a key policy on the source account.** A key policy that lets a service principal use the key on `Resource: "*"` with no condition is the textbook confused-deputy hole — any resource, in any account, that can induce that service to call KMS could use your key.

```
condition {
  test = "StringEquals"; variable = "aws:SourceAccount"; values = [var.account_id]
```

```
}
```

- 3. **One customer-managed key per data layer** — raw, bronze, silver, gold, secrets, audit — in each environment. The point of separate keys is that compromise of one does not decrypt the others, so **also narrow each key's trusted-service list to the services that layer actually uses**. Six keys sharing one fifteen-principal default list is six copies of the same key policy, not separation.

- 4. **Turn the catalog's default off, then grant explicitly.** Run the data catalog in strict mode: default permissions on a new database or table are NONE, and the "any IAM principal" fallback is disabled. IAM alone then opens nothing; access comes only from explicit, version-controlled grants.

- 5. **Grant on tags, not on names.** Define the vocabulary up front and grant against it:

TAG KEY	VALUES
layer	bronze · silver · gold
sensitivity	public · internal · confidential · pii
domain	rms · sim · xstore · loyalty · ecommerce · shared

A grant on `sensitivity = pii` keeps working when table 90 arrives. A grant on a list of table names does not.

- 6. **Secrets by reference only.** Terraform owns the secret's *name*, its key and its rotation policy — never its value. Write a placeholder once, then pin it:

```
lifecycle { ignore_changes = [secret_string] }
```

Consumers receive an **ARN** and resolve the value at run time. Where the platform can own the credential entirely (a managed admin password, for instance), let it — then no password exists in code or state at all.

L21 · Set Up Security and Governance

7. **Put a permissions boundary on any role that can create roles.** A deploy role that can create roles can create a role more powerful than itself unless a boundary caps it. That is a privilege-escalation path, and the boundary is the fix.
8. **Review rule: every wildcard needs a comment.** Some actions legitimately take no resource ARN. Those stay on `"*"` — with a line above saying why.

```
# lakeformation:GetDataAccess takes no resource ARN; scoping happens in LF grants.
resources = ["*"]
```

Why

Least privilege is not "zero wildcards". It is **every wildcard deliberate, and commented** — so a reviewer can tell the difference between a considered exception and an unconsidered one in five seconds.

What breaks if you don't: a job role becomes an account-wide key.

On Apparel Group

- **Epsilon loyalty data is PII, and so are MoEngage profiles.** Classify at the spec (`pii_class: pii` per column), mask or tokenise in Silver, and grant reporting on `sensitivity = pii` — not by naming tables. Do it before the first report is built, because a report is the hardest consumer to take access away from.
- **Five workload roles, not one.** Ingestion (Oracle JDBC), ingestion (SaaS API), transform, warehouse read, and orchestration. Each sees only its own buckets and catalog objects.

- **Source credentials for RMS, SIM, XStore, Epsilon, MoEngage and Magento are six secrets, six ARNs.** Jobs get the ARN; nobody gets the value.
- Footfall feeds (Vemco, Irisys) are per-store counts, `sensitivity = internal` — but classify them explicitly rather than by omission, so "unclassified" never becomes a permitted state.
- A permissions boundary goes on the deploy role from day one, because that role creates the workload roles.

Worked example of the pattern: the Tamimi `infra/modules/iam` (per-workload roles, bucket ARNs derived from a prefix, boundary attached) and `infra/modules/kms` (one key per layer, `aws:SourceAccount` on the service-principal statement).

Checklist

- ☐ No full-access managed policy on any workload role
- ☐ Every key-policy service-principal statement carries `aws:SourceAccount`
- ☐ One CMK per data layer, each with a narrowed trusted-service list
- ☐ Catalog in strict mode; default permissions on new objects are NONE
- ☐ Tag vocabulary defined; data grants written against tags
- ☐ Secrets created by name only, values written out of band, consumers hold ARNs
- ☐ Permissions boundary on every role that can create roles
- ☐ Every `"*"` in an IAM policy has a comment explaining itself

L21 · Set Up Security and Governance

You've got it when you can...

- State the difference between scoping a role and scoping a key policy, and give the one condition key that fixes the confused-deputy case.
- Explain why strict catalog mode means an IAM grant alone opens nothing.
- Argue for tag-based grants over name-based grants in one sentence, using the table count at week 12 as the argument.
- Point at any "✖" in your IAM code and say why it is there — or file it as a review finding.
- Say where Epsilon PII is classified, where it is masked, and what stops it reaching a report before either happens.

Connect to On-Prem Sources Safely

Private subnets, endpoints instead of NAT, TLS on both switches — and enough IP addresses.

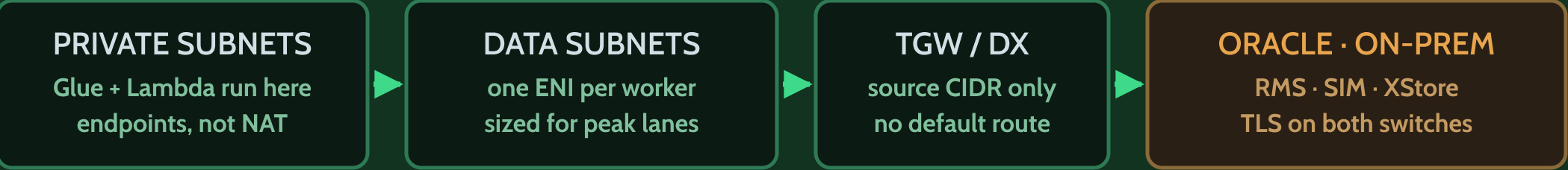
1 · THE DECISION

Your compute is in AWS. Oracle is not. What must be true before a byte moves?

Four things: a route to the source, an encrypted session, a private path to AWS APIs, and a free IP address.

2 · DO THIS

PRIVATE COMPUTE · PRIVATE PATH · ENOUGH ADDRESSES



- Put compute in private subnets; reach AWS APIs through interface endpoints, not NAT.
- Enforce TLS twice — on the JDBC URL and on the connection object.
- Route only the specific source CIDR. A /32 in Prod beats a /16 everywhere.
- Size the subnet to peak ENIs, cap job concurrency to match, and turn on flow logs.

`lanes × runs × workers ≈ peak ENIs · a /26 gives 59 usable addresses`

3 · WHY

THE NETWORK GIVES OUT FIRST

Addresses run out before the database does, and the failure reads as a database problem.

WHAT BREAKS IF YOU DON'T
Jobs die on zero free IPs, not on the query.

4 · ON APPAREL GROUP

AG · THREE ORACLE SOURCES

RMS, SIM and XStore all cross the same link — size it for XStore. Epsilon, MoEngage and Magento are egress, not on-prem.

`one route per source CIDR · TLS everywhere`

IN PLAIN ENGLISH

The pipe to the warehouse is not the problem — the loading bay is. Count the parking spaces before you send the trucks.

L22 · Connect to On-Prem Sources Safely

Slide: [_render/L22-connect-to-on-prem.html_](#)

The decision

Your compute is in AWS. Oracle is not — it is on-premises, behind someone else's firewall, reachable only over a shared link. Four things must be true before a byte moves, and the fourth is the one that surprises application developers:

- 1. The worker sits in a subnet whose **route table** points the source CIDR at the link.
- 2. The link has an **available attachment** to your VPC.
- 3. The session is **encrypted** — and that is two switches, not one.
- 4. There is a **free IP address** in that subnet.

Number four is what caps your download concurrency. Not CPU, not the database.

Do this

- 1. **Three subnet tiers, three different exits.**

TIER	HOLDS	REACHES THE INTERNET	REACHES AWS APIS
Public	NAT and the internet gateway only	yes	—
Private	orchestration Lambdas, transform runner	via NAT	interface endpoints
Data	JDBC worker ENIs, the warehouse	no route	gateway endpoints only

No workload of yours runs in the public tier.

- 2. **Use interface endpoints instead of NAT for AWS APIs.** Secrets Manager, KMS, STS, logs, the catalog, queues, metrics — put an interface endpoint in the private subnets so those calls stay on the AWS backbone.

Know the difference, because it decides what works where: - **Gateway endpoints** (object storage, the key-value store) are route-table entries. Free, and they work in a subnet with no NAT. - **Interface endpoints** are ENIs with a security group, in *named* subnets. A workload in a subnet without one has no path at all — it does not silently fall back.

- 3. **Route only the specific source CIDR.** One route per source, as narrow as the source team will give you. A `/32` to a single Prod database host is better than a `/24` ; a `/16` or a `/12` is a routing decision you will be asked to justify in a security review.

- 4. **Enforce TLS twice.** The URL and the connection object must agree, and the server certificate must be in the connection's truststore or validation fails:

```
jdbc:oracle:thin:@...?ssl_server_dn_match=true
JDBC_ENFORCE_SSL = "true"
```

Grep for both switches across your infrastructure and your job code, and find any place they disagree.

- 5. **Size the subnet against the ENI count, then cap concurrency to match.** This is arithmetic, and you should do it before you pick a CIDR.

```
a /26 subnet      → 59 usable addresses
each worker      → 1 ENI
ENI teardown lags a finished run by 1-2 minutes

peak ENIs = lanes × concurrent runs per lane × workers per run
           = 3     × 2                               × 12
           ≈ 72 ... too many for one /26 — spread across three, or reduce workers
```

L22 · Connect to On-Prem Sources Safely

Then set the orchestrator's fan-out cap equal to the sum of the lanes' per-lane run caps. If those two numbers disagree, a retry stacks an extra run onto a subnet instead of queueing on the pool.

- 6. **Turn off engine-internal retries on the download jobs.** An internal retry starts a second attempt without passing back through the scheduler, so it ignores your concurrency budget and stacks another full set of ENIs. Put retry in the orchestrator, where it re-issues through the same gate.
- 7. **Watch the second, softer ceiling too:** `fan-out cap × max JDBC partitions per table` is your concurrent connection count against the source. Keep it comfortably under whatever the source DBA will admit to.
- 8. **Enable VPC flow logs from day one**, all traffic, with a retention period someone has agreed to. You will want them the first time a connection is refused and both teams believe it is the other's firewall.

Why

The database is rarely the first thing to give out. The subnet is. And when it does, the error surfaces at the driver — a connect timeout, a socket failure — so the first hour goes to the DBA and the second hour goes to the network, and neither of them changed anything.

What breaks if you don't: jobs die on zero free IPs, not on the query.

On Apparel Group

- **RMS, SIM and XStore all cross the same on-prem link.** Size the plan for XStore — POS transactions are the giant — and let the RMS masters ride along.
- **Route each source CIDR separately.** Three narrow routes are easier to defend and easier to revoke than one broad one, and they let you cut a single source without touching the others.
- **Epsilon, MoEngage and Magento are not on-prem** — they are public API egress. That is a different path (NAT or a proxy), a different security group, and a different set of failure modes. Don't design one network rule for both halves of the source list.
- **Do the ENI arithmetic for XStore before you choose subnet sizes.** Changing a subnet CIDR after Redshift and a dozen Glue connections are pinned to it is not a small change.
- Footfall feeds (Vemco, Irisys) are small file/API pulls — no lanes, no ENI budget, no special routing.

Worked example of the pattern: the Tamimi `infra/modules/vpc` (three tiers, gateway + interface endpoints, flow logs) and its lane definitions, where the per-lane worker counts are written next to the ENI arithmetic that justifies them.

L22 · Connect to On-Prem Sources Safely

Checklist

- ☐ Compute runs in private or data subnets — never public
- ☐ Interface endpoints exist for every AWS API the workload calls from a private subnet
- ☐ Data-tier subnets have no default route, and you know exactly what they can reach
- ☐ Exactly one route per source CIDR, each as narrow as the source allows
- ☐ TLS enforced on both the URL and the connection object; server cert in the truststore
- ☐ Peak-ENI arithmetic written down, and the subnet sized above it
- ☐ Orchestrator fan-out cap equals the sum of per-lane run caps
- ☐ Engine-internal retries disabled on download jobs; retry lives in the orchestrator

- ☐ Flow logs enabled with an agreed retention

You've got it when you can...

- Name the four preconditions for an on-prem pull: **a route on the right table, an available attachment, TLS on both switches, and a free IP.**
- Answer without running anything: a function in a private subnet calls Secrets Manager — does that traffic leave the VPC? Same call from a data subnet — what happens?
- Derive a fan-out cap from first principles, starting at usable addresses per subnet.
- Explain why an internal engine retry breaks a concurrency budget that the orchestrator's retry does not.
- Say, for each of the 8 Apparel Group sources, whether it crosses the on-prem link or goes out to the internet.

Set Up Observability From Day One

Freshness gets a number, lateness gets an alarm, and every alarm reaches a named group.

1 · THE DECISION

When is this table late — and who finds out first, you or the business?

Failure alarms are the easy half. Nothing fires when a job succeeds on time and produces nothing.

2 · DO THIS

A NUMBER · TWO ALARMS · A LIST · A RUNBOOK

pipeline_state · freshness:<table> · last_success_at · expected_within_seconds

ALARM A — a run ended FAILED.

ALARM B — nothing succeeded in time.

Both route to the same list. Both link the stage runbook.

- Give every table a freshness SLA — a number in the control plane, not a habit.
- Alarm on failure AND on lateness. The silent success is the dangerous one.
- Route alerts to a distribution list, never to one person's inbox.
- Write the stage runbook in the same pull request as the pipeline.
- Build the operator view on the control plane, not on log searches.

3 · WHY

SILENCE IS NOT HEALTH

A green run with zero rows looks exactly like a healthy day — until month-end.

WHAT BREAKS IF YOU DON'T

The business tells you the data is stale.

4 · ON APPAREL GROUP

AG · ONE SLA PER TABLE

XStore lands overnight; footfall files arrive hourly; Epsilon pages on its own cursor. One number per table, not one per platform.

expected_within_seconds · one row per table

IN PLAIN ENGLISH

A smoke alarm nobody can hear is decoration. Say in seconds when each table is late, and send that to a list — not to a person.

L23 · Set Up Observability From Day One

Slide: `_render/L23-observability-from-day-one.html`

The decision

When is this table late — and who finds out first, you or the business?

Failure alarms are the easy half; every platform ships with them. The hard half is that **nothing fires when a job succeeds on time and produces nothing**. That is the failure that reaches a report, and it is the one you have to design for.

Do this

- 1. **Give every table a freshness SLA, expressed as a number the platform can evaluate.** Not "daily by 07:00" in a document — a row in the control plane:

```
entity:  freshness:<table>
fields:  last_success_at      (written by the pipeline on success)
         expected_within_seconds (the SLA, per table)
```

The pipeline writes the first field. You choose the second, per table, when you onboard the table — the same pull request, not later.

- 2. **Alarm on failure AND on lateness.**

	FIRES WHEN	ANSWERS
Alarm A — failure	a run reached a terminal FAILED state	"something broke"
Alarm B — lateness	<code>now - last_success_at > expected_within_seconds</code>	"nothing broke, and nothing arrived"

Alarm B is the one that catches a wedged watermark, an upstream export that stopped being written, and a source that returns an empty page forever.

- 3. **Route both alarms to a real distribution list.** Not one engineer’s inbox, not a channel nobody has joined. Write down who is on the list and confirm at least two of them can act at the hour the alarm can fire.

- 4. **Make the alert carry the evidence, not just the news.** A useful alert names the **stage**, the **table**, the **cycle**, the recorded error message verbatim, a console link, and the runbook. The recipient should be able to start diagnosing from the email alone.

- 5. **Write the runbook in the same pull request as the pipeline.** One page per stage: what this stage does, the three most likely causes, the exact command or payload for each recovery, and who is allowed to run it. A runbook written six months later is written from memory and is wrong.

- 6. **Build the operator view on the control plane, not on log searches.** If the platform records runs (per stage), watermarks, cycle state and lineage, then every operator question is a lookup. Design for these four questions:

- Did today's cycle complete, and which stage is it on?
- Which table failed, and what did it say?
- Did this table's watermark advance, and is the new value plausible?
- Is this table inside its freshness SLA right now?

- 7. **Alarm on the things that are not runs, too:** cost against budget, queue depth or dead-letter counts on anything asynchronous, and the age of the oldest in-flight cycle.

Why

Monitoring is not a dashboard; it is a **promise with a number attached and a person on the other end**. A run that ends green tells you the code did not crash. It does not tell you the data arrived — only a freshness number can do that.

L23 · Set Up Observability From Day One

What breaks if you don't: the business tells you the data is stale.

On Apparel Group

- **One SLA number per table, not one for the platform.** XStore POS lands overnight and has hours of slack; footfall files arrive hourly and go stale in one; Epsilon and MoEngage page against their own cursors and can quietly return nothing. A single platform-wide threshold is wrong for all three.
- **Set the number when you onboard the table** — it is step 6 of L24's procedure, not a follow-up ticket.
- **The SaaS sources need lateness alarms most.** A JDBC pull that cannot reach Oracle fails loudly; an API that returns an empty page succeeds quietly. Alarm B is the only thing watching Epsilon and MoEngage.
- **Route to the workstream, not the individual.** Four workstreams over 12 weeks means the person who wrote the pipeline may not be the person on duty when it breaks.
- Write eight runbooks — one per source — as you onboard, and keep them beside the specs.

Worked example of the pattern: the Tamimi control plane's `pipeline_state` freshness entity (`last_success_at` vs `expected_within_seconds`), the per-stage failure notifier that emails the run row itself, and `docs/runbooks/` with one file per stage.

Checklist

- ☐ Every table has an `expected_within_seconds` value, chosen deliberately
- ☐ A failure alarm and a lateness alarm exist, and both have been tested by firing them
- ☐ Both route to a distribution list with at least two people who can act
- ☐ The alert body names stage, table, cycle, error text, console link and runbook
- ☐ A stage runbook shipped in the same pull request as the stage
- ☐ The operator view answers the four questions above from stored state, not log search
- ☐ Budget, dead-letter and stuck-cycle alarms exist

You've got it when you can...

- Explain the difference between Alarm A and Alarm B, and name a failure that only Alarm B catches.
- State the freshness SLA of any table on the platform, in seconds, and say who chose it.
- Show, from stored state alone, whether today's cycle completed and which stage it is on.
- Explain why a green run with zero rows is more dangerous than a red one.
- Say why "an alarm that reaches nobody is not monitoring" is a statement about people, not about tooling.

The Standard Procedure for Adding a Source

Nine steps, in order, worked once against a real table — then it stops being a project.

1 · THE DECISION

Is a new source an engineering project, or a configuration change?

You will run this loop eight times in twelve weeks. Decide now what one new table is allowed to cost.

2 · DO THIS

NINE STEPS, IN ORDER — NONE OF THEM INSIDE THE ENGINE

- 1

CLASSIFY THE TABLE
full master · incremental fact · windowed giant
- 2

CONFIRM THE PK AND THE WATERMARK
read the constraint catalog — never assume
- 3

CONNECTOR — ONLY IF NEEDED
a branch in the JDBC class, not a new project

CONDITIONAL
- 4

WRITE THE DOWNLOAD SPEC
projection, watermark column, parallelism
- 5

WRITE THE BRONZE SPEC
merge_key, plus the same schema mirrored
- 6

ADD THE CATALOG ROW
one row per table, one connection per source
- 7

STAGING MODEL + TESTS
a view, plus not_null and unique on the key
- 8

GOLD MODEL — IF IT REPORTS
otherwise stop — runs already prove the pull

CONDITIONAL
- 9

DEPLOY THROUGH THE GATE
sync specs · seed catalog · smoke-test in Dev

3 · WHY

ONE SOURCE, ONE PULL REQUEST

Engine, barriers and control plane stay untouched — only config moves.

WHAT BREAKS IF YOU DON'T Every new table becomes its own project.

4 · ON APPAREL GROUP

AG · ORACLE RETAIL RMS · ITEM_MASTER

```
download/rms_item_master.yaml · watermark: LAST_UPDATE_DATETIME
bronze/rms_item_master.yaml · merge_key: [ITEM]
shape refs: src/glue/specs/{download,bronze}/sap_mara.yaml
```

IN PLAIN ENGLISH

Nine steps, none of them inside the engine. Do this once for RMS. Then it's a habit.

L24 · The Standard Procedure for Adding a Source

Slide: `__render/L24-adding-a-source-sop.html`

The decision

Is a new source an engineering project, or a configuration change?

Everything in this module has been building to one answer. You will run this loop **eight times in twelve weeks** at Apparel Group. Decide now what one new table is allowed to cost — because if the answer is "a sprint", the answer for eight sources is "the whole engagement".

Make it a checklist. Nine steps, in order, and none of them inside the engine.

Do this

The whole procedure, once, worked concretely against **Oracle Retail RMS** `ITEM_MASTER` — the first table your team will onboard.

1 · Classify the table

Before you write anything, put it in exactly one bucket:

CLASS	LOOKS LIKE	LOAD SHAPE
Full-only master	small, changes slowly, no reliable change column	full refresh every cycle
Incremental fact	large, append-mostly, has a trustworthy change column	initial load, then deltas
Windowed giant	very large, changes across a range	date-window pulls, chunked

`RMS.ITEM_MASTER` is a **master with a change stamp** — one complete initial load, then incremental. Write the class down; steps 4 and 6 both depend on it.

2 · Confirm the primary key and the watermark

Two facts, both verified against the source, neither assumed:

- **Primary key** — read it out of the source's constraint catalog. Do not infer it from a column name, and do not accept "it's the item number, obviously".
- **Watermark** — a column that is monotonic, indexed, non-null and actually maintained by the application. If no such column exists, say so and declare the table full-refresh. **Faking CDC is worse than not having it.**

For `ITEM_MASTER` : PK `ITEM` ; watermark `LAST_UPDATE_DATETIME` .

These two lines are the highest-stakes output of the whole procedure. Everything downstream inherits them.

3 · Connector — only if the protocol isn't already satisfied

The engine never imports a concrete connector; it asks a registry for one by source type. If an existing connector already satisfies the protocol for this protocol shape, **you write no code at all**.

Oracle over JDBC is the same code path as any other JDBC engine. That makes RMS, SIM and XStore *one branch on an engine literal*, not three connectors:

```
_SUPPORTED_ENGINES = {"postgresql", "sqlserver", "oracle"}  # ← the whole change
```

Choose deliberately between widening an existing connector and registering a sibling class: a genuinely new source-type string also has to be added to the spec's validated literal, which is a wider change than the branch.

Epsilon and MoEngage will **not** fit the JDBC shape. They need paging, cursors, token refresh and rate-limit handling — and raw-first landing, because you often cannot re-request an old page. Say that out loud when you plan them; do not discover it in week 9.

L24 · The Standard Procedure for Adding a Source

4 · Write the download spec (source → raw)

Shape reference: [src/glue/specs/download/sap_mara.yaml](#)

```
# specs/download/rms_item_master.yaml
table: rms.item_master
source_type: rds_jdbc

source_config:
  cadence: daily
  jdbc_schema: RMS
  jdbc_table: ITEM_MASTER
  watermark_column: LAST_UPDATE_DATETIME    # monotonic, indexed, IN THE PROJECTION
  jdbc_hash_field: ITEM                      # PK member, high cardinality
  jdbc_hash_partitions: "8"                 # parallelism is mandatory, not optional

landing_prefix: "rms/rms.item_master"        # RELATIVE to the raw bucket root
format: parquet

schema:
  - { name: ITEM,      type: string, pii_class: internal, description: "Item number (PK)" }
  - { name: ITEM_DESC, type: string, pii_class: internal }
  - { name: DEPT,      type: string, pii_class: internal }
  - { name: CLASS,     type: string, pii_class: internal }
  - { name: SUBCLASS,  type: string, pii_class: internal }
  - { name: LAST_UPDATE_DATETIME, type: string, pii_class: internal, description: "WATERMARK" }

partition_by: []
```

Three decisions live in this file:

- **The watermark column must be in the projection.** The new watermark is computed as `MAX(<watermark_column>)` over what you pulled. If it isn't selected, you find out *after* paying for the whole pull.

- **Parallelism has no safe default.** Declare exactly one of hash field or hash expression, never both. Without it the read runs on one executor.
- **Curate the projection; never `SELECT *`** . Declare the columns you consume. A wide table with hundreds of columns should arrive as the twenty you use, so a source-side schema change cannot silently leak untyped columns into Bronze.

5 · Write the bronze spec (raw → table format)

Shape reference: [src/glue/specs/bronze/sap_mara.yaml](#)

```
# specs/bronze/rms_item_master.yaml
table: rms.item_master
source_type: s3_landing      # reads back what step 4 landed – no creds, no DB hit

merge_key: [ITEM]           # the VERIFIED natural PK from step 2

schema:                     # MUST mirror the download spec, column for column
  - { name: ITEM, type: string, pii_class: internal, description: "Item number (PK)" }
  # ... identical to download/rms_item_master.yaml

source_config:
  landing_prefix: "rms/rms.item_master"    # the same relative prefix step 4 landed under
  format: parquet
```

`merge_key` is the single highest-stakes line in the procedure: it is what makes replays, overlapping windows and retries land each row exactly once. Validation can check that every member exists in the schema, that there are no duplicates, and that no member is classified PII. It **cannot** check that it is the real primary key — that was step 2's job.

Two specs exist because a re-pull and a re-load are different failures with different costs. Keep their schemas identical; drift between them is a silent correctness bug.

6 · Add the catalog row

One row per **table**, one connection entry per **source**:

L24 · The Standard Procedure for Adding a Source

```
- name: rms.item_master
  source: rms           # → the connection entry carrying source_type + secret ARN
  source_object: ITEM_MASTER
  download_spec: download/rms_item_master.yaml
  bronze_spec:  bronze/rms_item_master.yaml
  driver_by_mode: { full: rds_jdbc, range: rds_jdbc, incremental: rds_jdbc }
  initial_load: once
  cadence_band: daily
  freshness_sla_seconds: 86400    # L23 — choose it here, not later
  enabled: true
```

- `initial_load: once` engages the init-once guard: one complete historical load first, and only then are daily deltas allowed.
- `enabled: false` pauses a table without deleting it — use it while you iterate.
- `driver_by_mode` keeps the option of different connectors for different read modes; for Oracle all three are the same, but declaring it costs nothing.

7 · Staging model + tests

```
-- models/staging/stg_rms_item_master.sql
{{ config(materialized='view') }}

SELECT
  item              AS item_id,
  item_desc         AS item_name,
  dept, class, subclass,
  last_update_datetime AS updated_at
FROM {{ source('silver', 'rms_item_master') }}
```

```
# models/staging/_staging.yml
- name: stg_rms_item_master
  description: "1:1 mirror of silver rms.item_master, snake_case columns."
```

```
columns:
- name: item_id
  tests: [not_null, unique]
```

Staging is a **view** and a rename — no business logic. The `not_null` + `unique` pair on the natural key is the cheapest correctness insurance you will ever buy; add it before you add anything clever. If the staging layer reads an external schema, declare it late-binding.

8 · Gold model — only if something reports on it

If `ITEM_MASTER` is a dimension feeding a fact that already exists, this is a join, not a new model. If it opens a new subject area, write the Gold model and its tests — with L14's checklist in hand: state the grain in one sentence, constrain lookup joins, exclude synthetic rollup rows, uniqueness test on the key, reconciliation test on the totals.

Do not invent a Gold model to prove the pipeline works. The run records and row counts already prove that.

9 · Deploy through the gate

Three things must happen, in this order:

1. **Sync the specs to object storage.** Jobs read the YAML at run time — a spec that isn't synced does not exist.
2. **Sync the transformation project.**
3. **Seed the control plane** from config, hashing the *same spec bytes* that were just uploaded, so the platform can tell whether the deployed spec matches the repo. Run it `--dry-run` first.

The seed runs **after** both the artifact upload and the infrastructure apply, because it hashes uploaded specs and writes rows for resources that must already exist. Then it goes through L20's gate like any other change.

Smoke-test in Dev and watch for four things: the run rows go green stage by stage; the watermark advanced; the new value is *plausible*; the cycle reaches its terminal state. That sequence — not a passing unit test — is "done".

L24 · The Standard Procedure for Adding a Source

Why

The engine, the barriers, the writers, the control plane, the orchestration and the Silver/Gold contracts are **untouched by all nine steps**. Only configuration moves. That is what makes eight sources tractable in twelve weeks: one source, one pull request.

What you never touch: the job code, the writers, the dispatcher, the barriers, the sweeper, the control-plane models, the state machines, the reporting views. If your change needs one of those, stop and ask whether you are solving the wrong problem. The one legitimate exception is step 3's engine branch — and that is four lines in a connector, not the engine.

What breaks if you don't: every new table becomes its own project.

On Apparel Group

Run the procedure once for RMS `ITEM_MASTER` , then fill this table in before you write any more specs. It is the planning artefact for the whole Data Foundation workstream:

#	SOURCE	SHAPE	STEP 3 NEEDED?	WATERMARK	PK
1	Oracle Retail RMS	JDBC	branch only	change stamp per table	from constraint catalog
2	Oracle SIM	JDBC	no — same branch	high-churn position stamp	from constraint catalog
3	Oracle XStore	JDBC	no — same branch	transaction timestamp	from constraint catalog
4	Epsilon	SaaS API	yes — paging + cursor	API cursor / token	vendor id, confirm
5	MoEngage	SaaS API	yes — paging + rate limits	API cursor / token	vendor id, confirm
6	Magento	DB or API	decide the protocol first	<code>updated_at</code>	entity id
7	Vemco Footfall	file / API	small connector	file date	store + interval
8	Irisys Footfall	file / API	small connector	file date	store + interval

Three of the eight are one connector branch. Two need real connector work. Plan the two early — an API connector written under deadline is where paging bugs come from.

L24 · The Standard Procedure for Adding a Source

Checklist

- ☐ Table classified: full / incremental / windowed
- ☐ PK read from the source's constraint catalog, not assumed
- ☐ Watermark column verified monotonic, indexed, non-null — or the table declared full-refresh
- ☐ Connector work confirmed necessary before any is written
- ☐ Download spec: curated projection, watermark column present in it, parallelism declared
- ☐ Bronze spec: `merge_key` = the verified PK, schema mirrors the download spec exactly
- ☐ Catalog row added, with the freshness SLA chosen now
- ☐ Staging view + `not_null` and `unique` on the natural key
- ☐ Gold model written only if something reports on it
- ☐ Specs synced, control plane seeded (dry-run first), change taken through the gate
- ☐ Dev smoke test: all stages green, watermark advanced and plausible, cycle terminal

- ☐ No file outside specs, config and models was modified

You've got it when you can...

- List the nine steps in order, from memory, and say which two are conditional.
- Explain why adding Oracle Retail is a branch in one connector rather than a new pipeline.
- Explain `merge_key` to a room in one sentence, and name what must be verified before you choose it.
- Say why the download spec and the bronze spec both exist, and what goes wrong when their schemas drift.
- Point at every file that must change — and every file that must not.
- Explain why a spec that isn't synced does not exist as far as the platform is concerned.
- Say the sentence that closes this module: **do this once for RMS. Then it's a habit.**

Do this once for RMS. Then it's a habit.